

Cloud Computing

Exercises Part 2 (Platforms)

Stuttgart University

WS 2026/27

09-12-2026

Prof. Dr. Kristof Kloeckner

GM and CTO, IBM GTS (retired)

kristof.kloeckner@iaas.uni-
stuttgart.de

Gerd Breiter

DE, IBM (retired)

gbreiter58@gmail.com

Cloud in Practice

Working with AWS Part 2



Cloud Platforms

- Service-Oriented Architectures and Microservices
 - Compute Services discussed in other sections
 - Integration Services (SQS, SNS)
- Event-Driven and Serverless (Function as a Service)
 - Lambda, API Gateway
- Containers (ECS and EKS)
 - Fargate serverless containers
- Serverless Workshops
 - Streams, Analytics
 - Step Functions
 - EventBridge

App Developer Platform Vision



Onsi Fakhouri

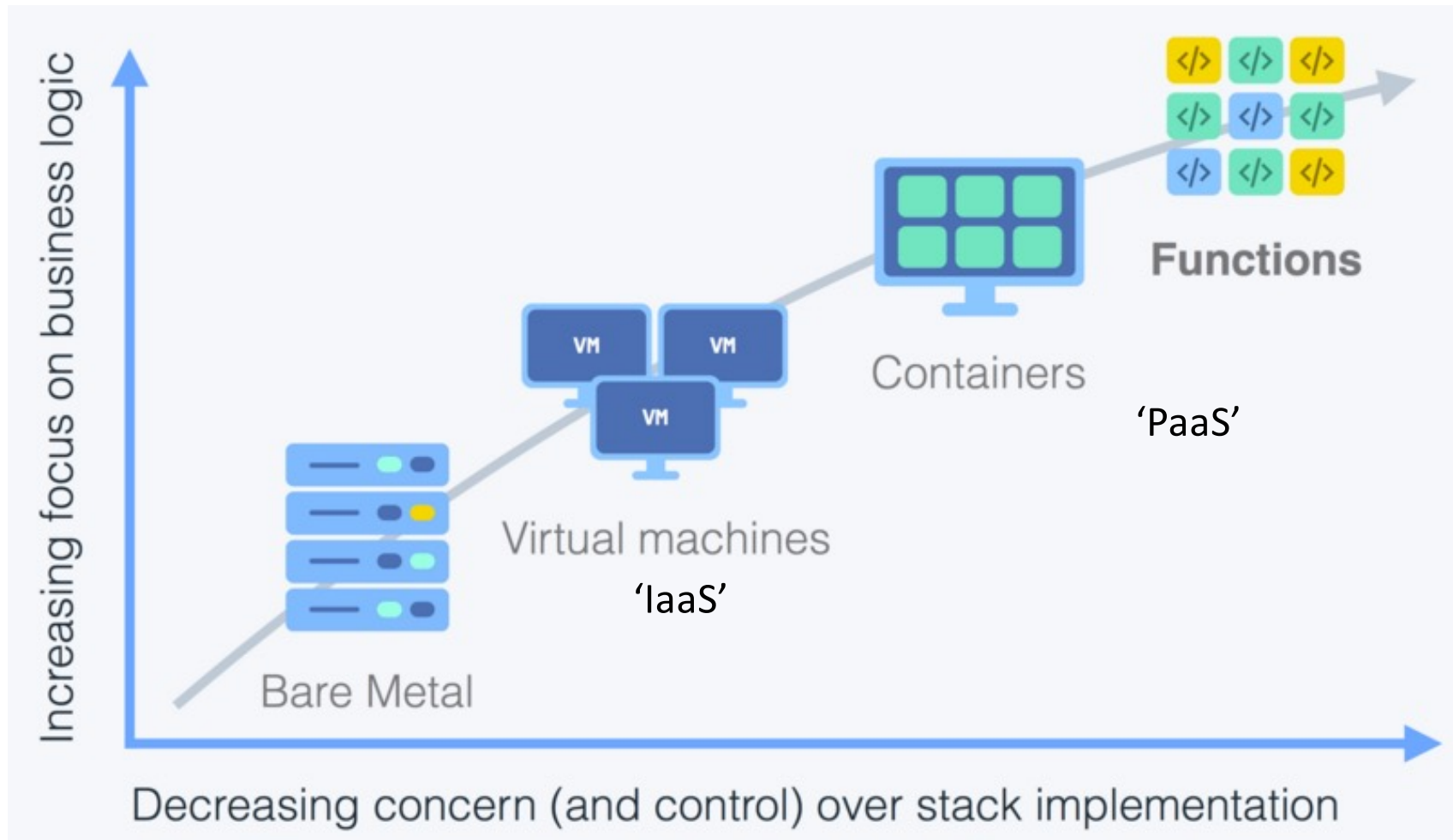
@onsijoe

cf push haiku

here is my source code
run it on the cloud for me
i do not care how

11:18 PM - 12 May 2015

Levels of Control



Module 6

Cloud Platforms: SOA & Microservices



Service Oriented Architectures and Microservices

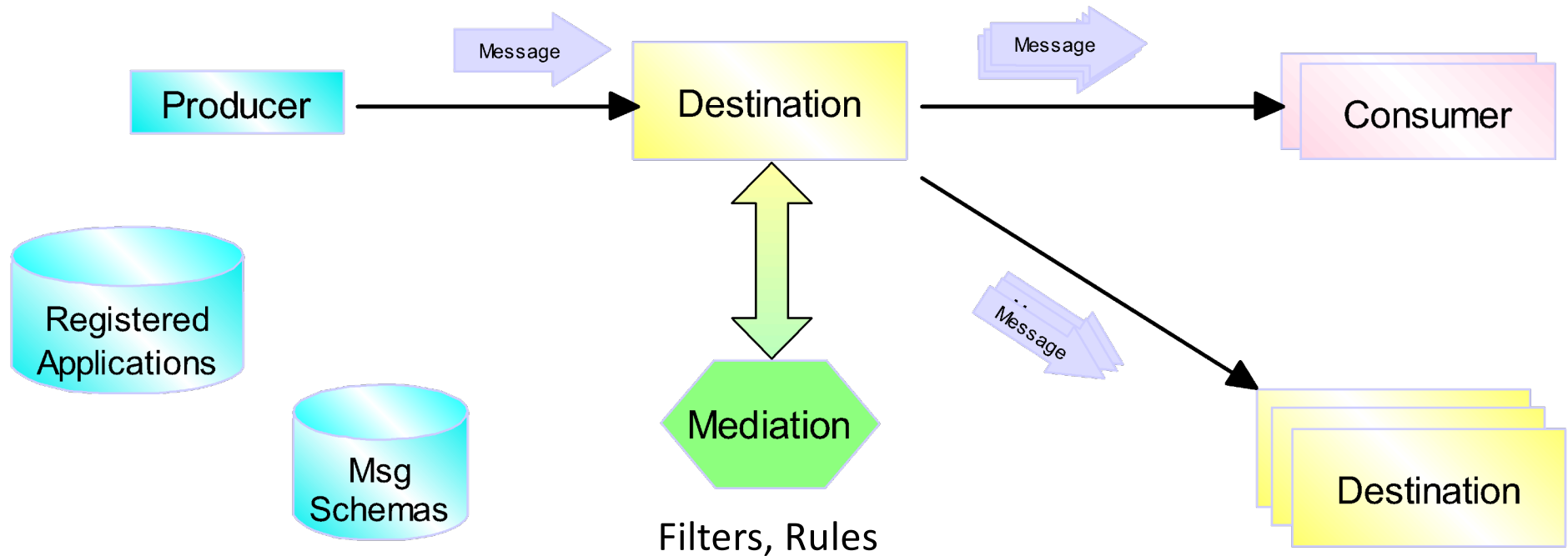
Asynchronous Messaging and Loose Coupling

In the 1990s, an increasing need to integrate diverse applications to deliver 'composite' business services gave rise to message-based integration:

- 'Autonomous' applications communicate asynchronously through messages, i.e. exchange of information in a structured format
- They have no other dependencies on each other, the messages they can send and receive describe their entire interface
- This style of integration is also called '**loose coupling**'
- Loose coupling allows flexible composition of '**application networks**'
- Messaging comes in many flavors, among them
 - **Guaranteed delivery** (at-least-once)
 - **Once-and-once-only** (allows transaction tiering)
 - **Publish-and-subscribe** (consumers subscribe to topics)
 - **Message brokers/message buses** (intermediaries that can route and transform messages)

Core Messaging Model

- End Applications are represented as Message Producers and/or Consumers
 - ▶ have a distribution pattern (one or all), and an option to queue messages
 - ▶ can be connected together by the administrator, or dynamically
- Mediations modify routing, format and/or content of a message
 - ▶ logic hosted in the network rather than at an endpoint



Some AWS Integration Services

- **Amazon Simple Queue Service**
 - Fully managed queuing service
 - Priced per request, first 1M requests free
- Amazon MQ
 - Fully managed message broker service for Apache ActiveMQ
- **Amazon Simple Notification Service**
 - Fully managed publish/subscribe (pub/sub) and mobile notification service
- Amazon Managed Services for Apache Kafka
- Related: Amazon EventBridge
 - Serverless event bus service
- Related: StepFunctions
 - Serverless state-machine-based simple workflow

Amazon Simple Queue Service (SQS)

- From AWS Console, select Simple Queue Service
- Either choose 'Get started', or go to dashboard and 'Create queue'
- Choose a queue name and select the type of queue
 - **Standard queue** delivers '**at least once**' and does not preserve order, very high performance
 - **FIFO queue** preserves order and delivers **exactly once**, performance limited to 300 messages/second
- Note: This behavior is due to replication of queues and how the copies are updated
- You can modify settings (like retention period) or 'quick-create'
- After creation, the queue shows up in the SQS console. It has a URL.
- Send a message via 'queue actions' and dialog box
 - Messages can have **attributes** (structured metadata) to inform the subscriber about context
- View the message and delete it (see messages through 'start polling')
 - The last two actions would normally be done by producer/consumer applications, see developer guide
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>

SQS and the AWS SDK

<https://docs.aws.amazon.com/code-library/latest/ug/what-is-code-library.html>

```
import boto3
# Get the service resource
sqs = boto3.resource('sqs')

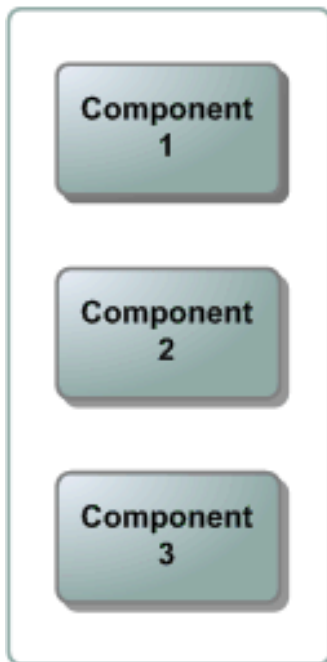
# Get the queue
queue = sqs.get_queue_by_name(QueueName='Orders')

# Create a new message
response = queue.send_message(MessageBody='world')

# The response is NOT a resource, but gives you a message ID and MD5
print(response.get('MessageId'))
print(response.get('MD5ofMessageBody'))
```

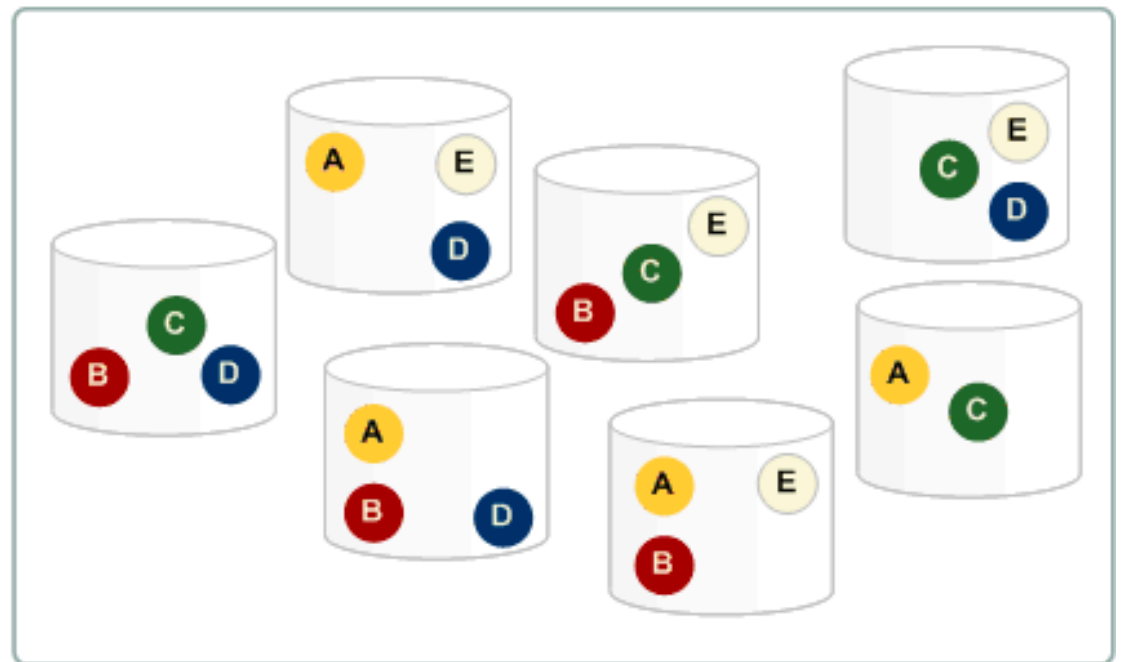

SQS Setup

**Your Distributed
System's
Components**



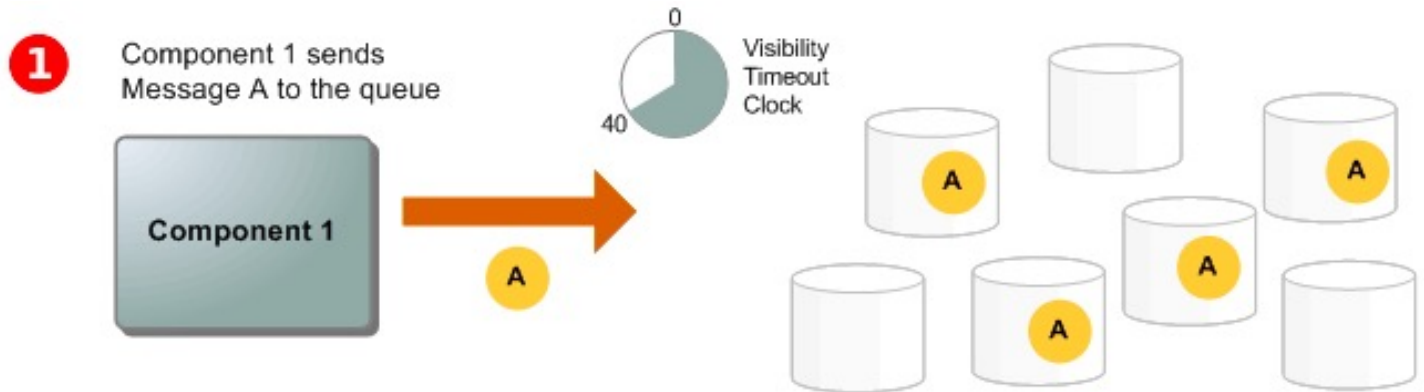
**Your Queue
(Distributed on
SQS Servers)**

Managed by AWS

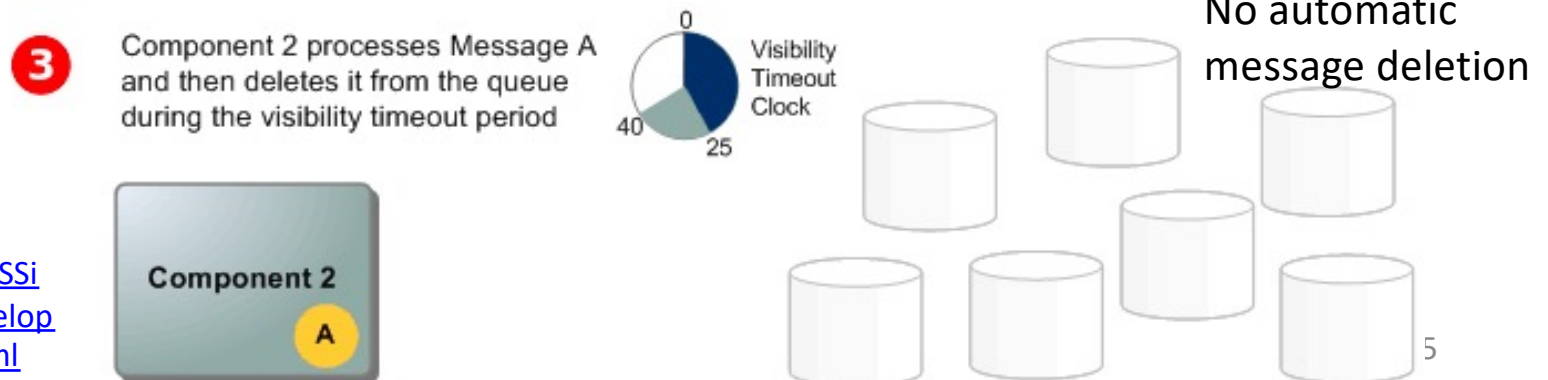
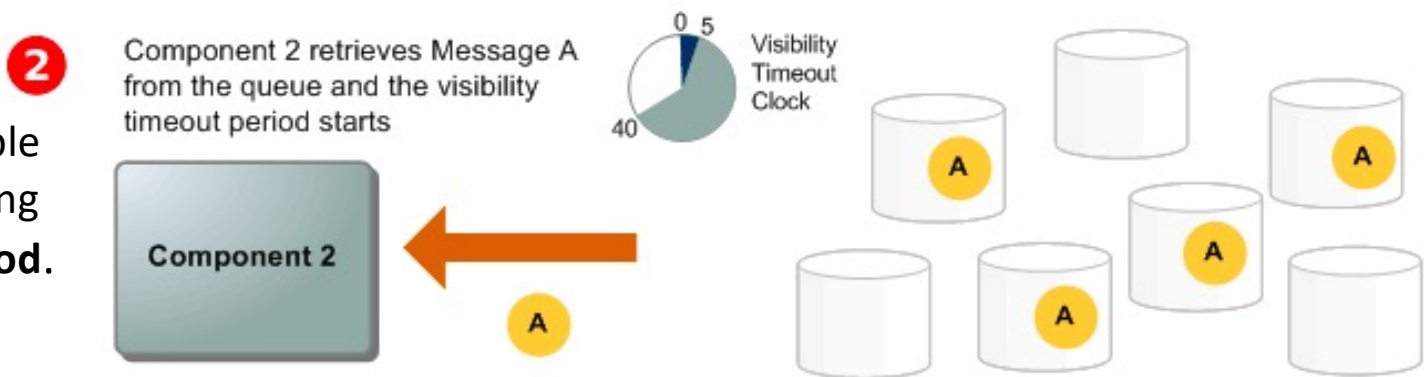


<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-basic-architecture.html>

Message Lifecycle



The message is not available to other components during the **visibility timeout period**.



Amazon Simple Notification Service (SNS)

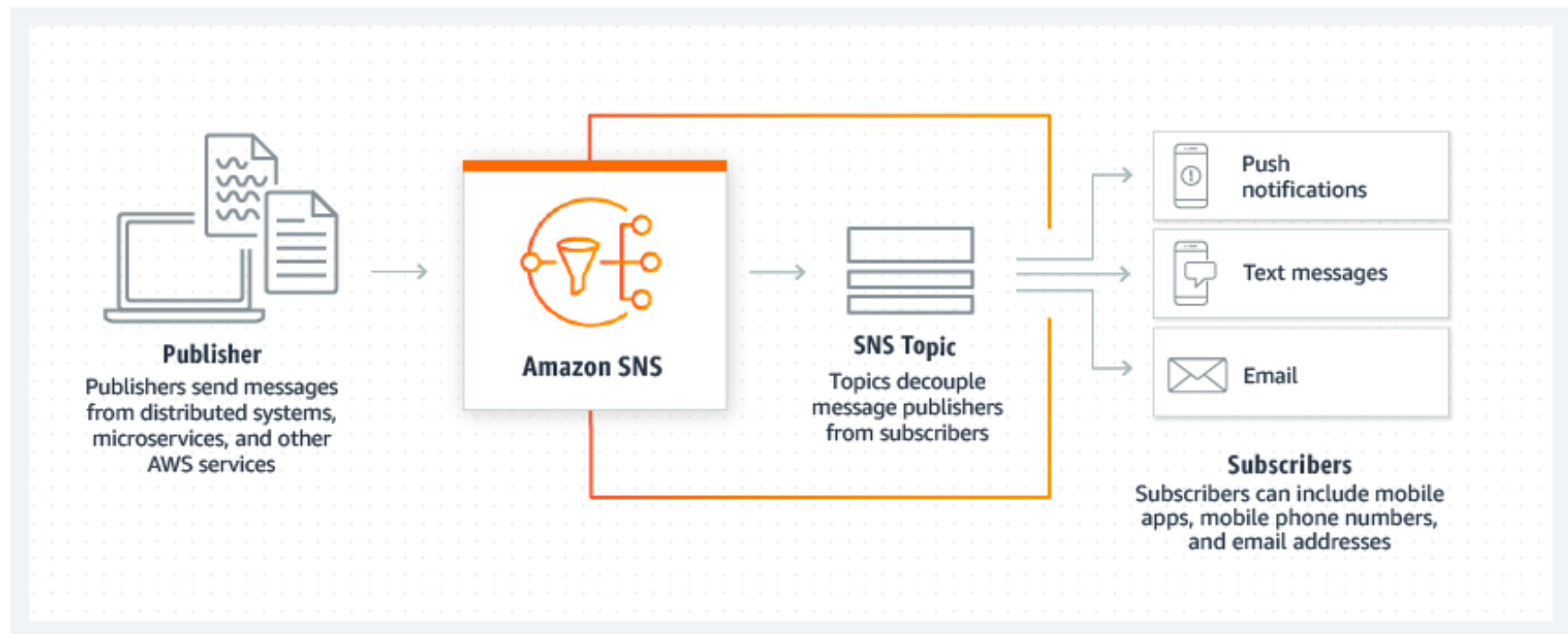
- (Managed) System-to-system messaging decoupling producers and consumers of messages through a publish/subscribe mechanism



Amazon Simple Notification Service (SNS)

User Notifications

- Push notifications to mobile apps, texts to phone or simple email
- Direct push to endpoints or fan-out with pub/sub



Creating Topics and Subscriptions

- Create a Topic
 - Select name
 - Select access policy (basic/advanced)
 - Default access is owner only
 - You can enable specific accounts, services or everyone
 - Several advanced options available
- Create Subscription
 - Select protocol
 - Select endpoint
 - Subscription is pending until confirmed by endpoint (subscriber)
 - **You can also subscribe directly from a service like SQS**
- Publish a message to the topic and check subscriber
- Allowing e.g. an S3-bucket to publish to a topic:

<https://docs.aws.amazon.com/sns/latest/dg/sns-access-policy-use-cases.html#sns-allow-s3-bucket-to-publish-to-topic>

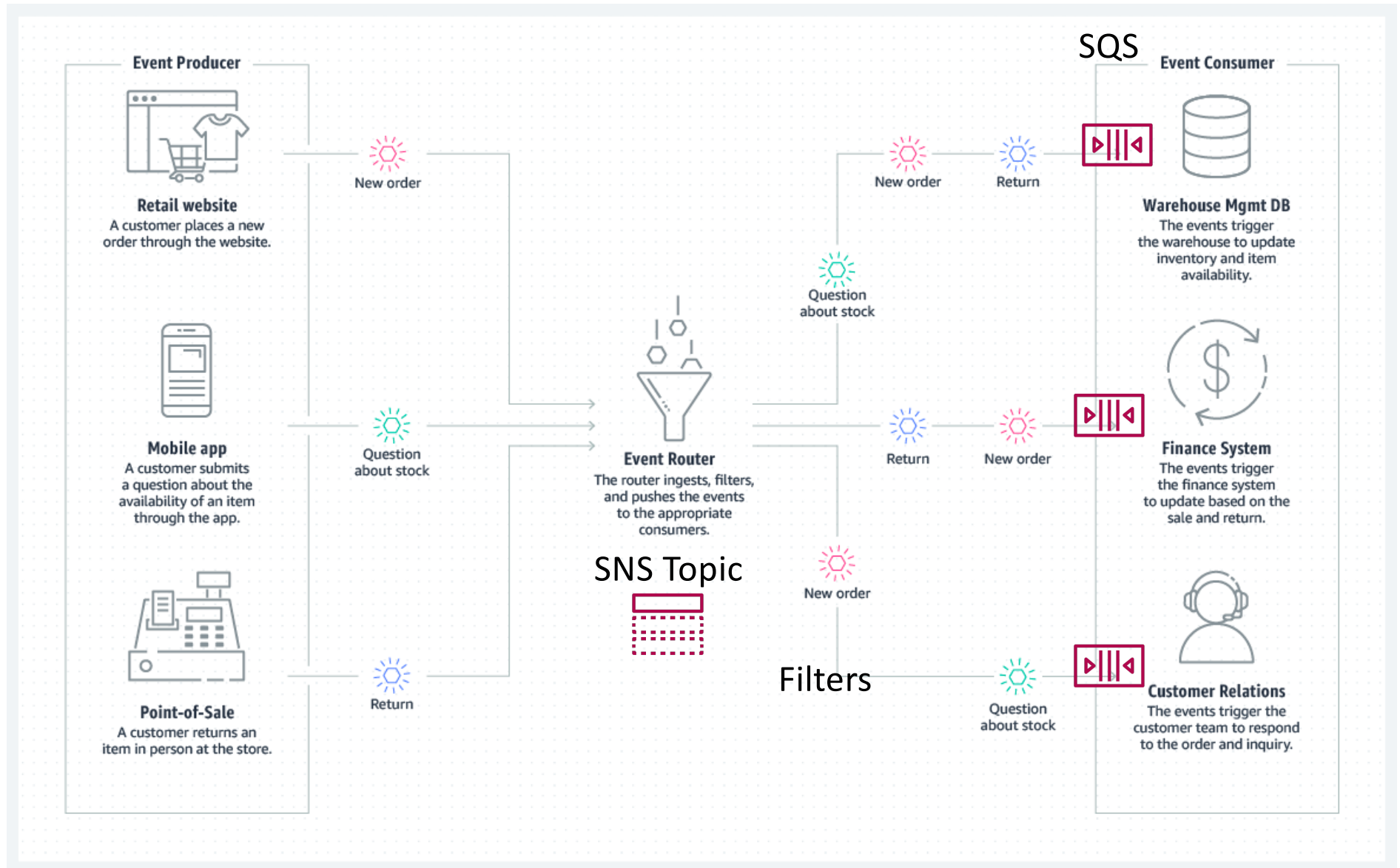
Subscription Filters

Subscriptions can filter what messages they want to receive by establishing a **subscription filter policy**

- Edit the subscription, open the 'subscription filter policy' tab and replace the JSON document with one that describes the required message attributes
- Example: When subscribing to an Insurance-Quote-Request topic, to receive only life insurance requests, specify

```
{"insurance_type": ["Life"]}
```
- When publishing a message to the topic, use the **message attributes** for filtering – the attribute below would pass the filter above
 - Type: String
 - Name: insurance_type
 - Value: Life

Event-Driven Architecture with Topics, Filters and Queues



Message Bus Pattern in AWS

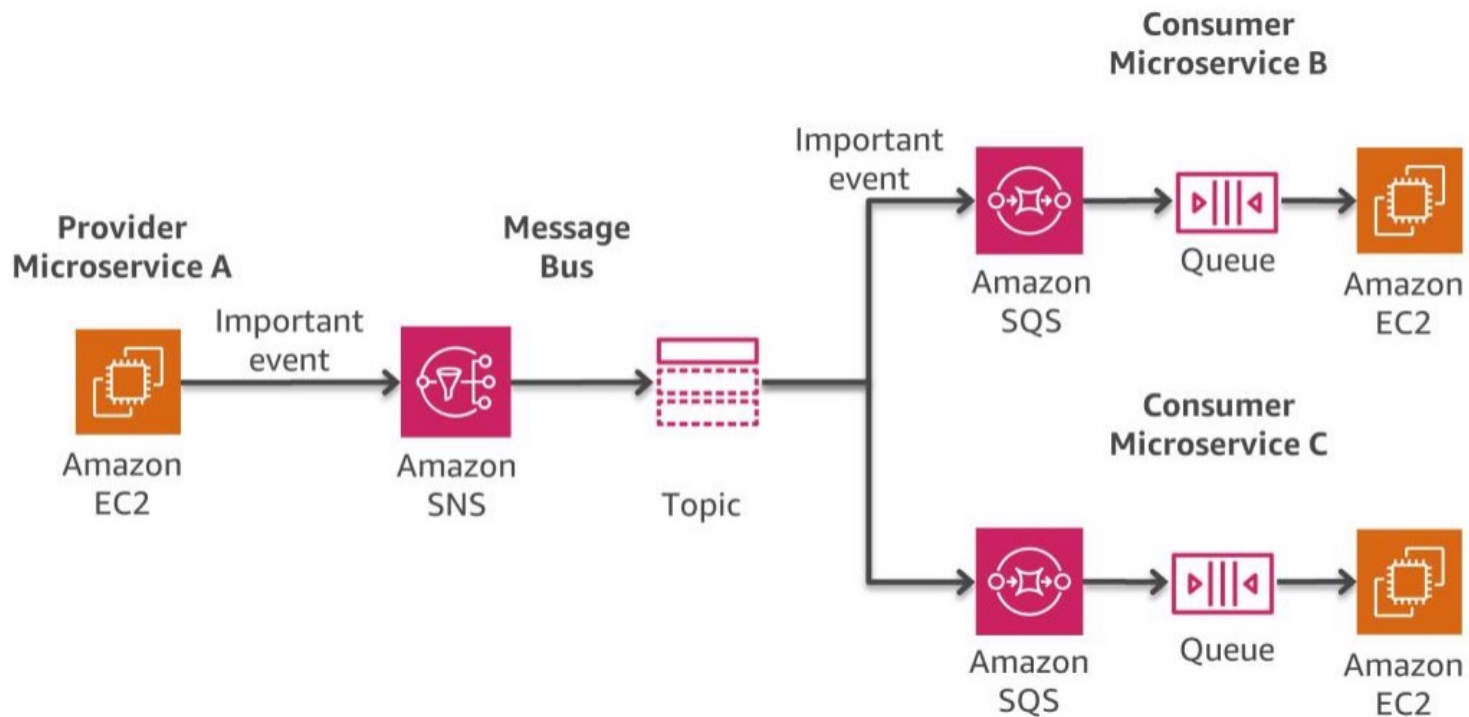
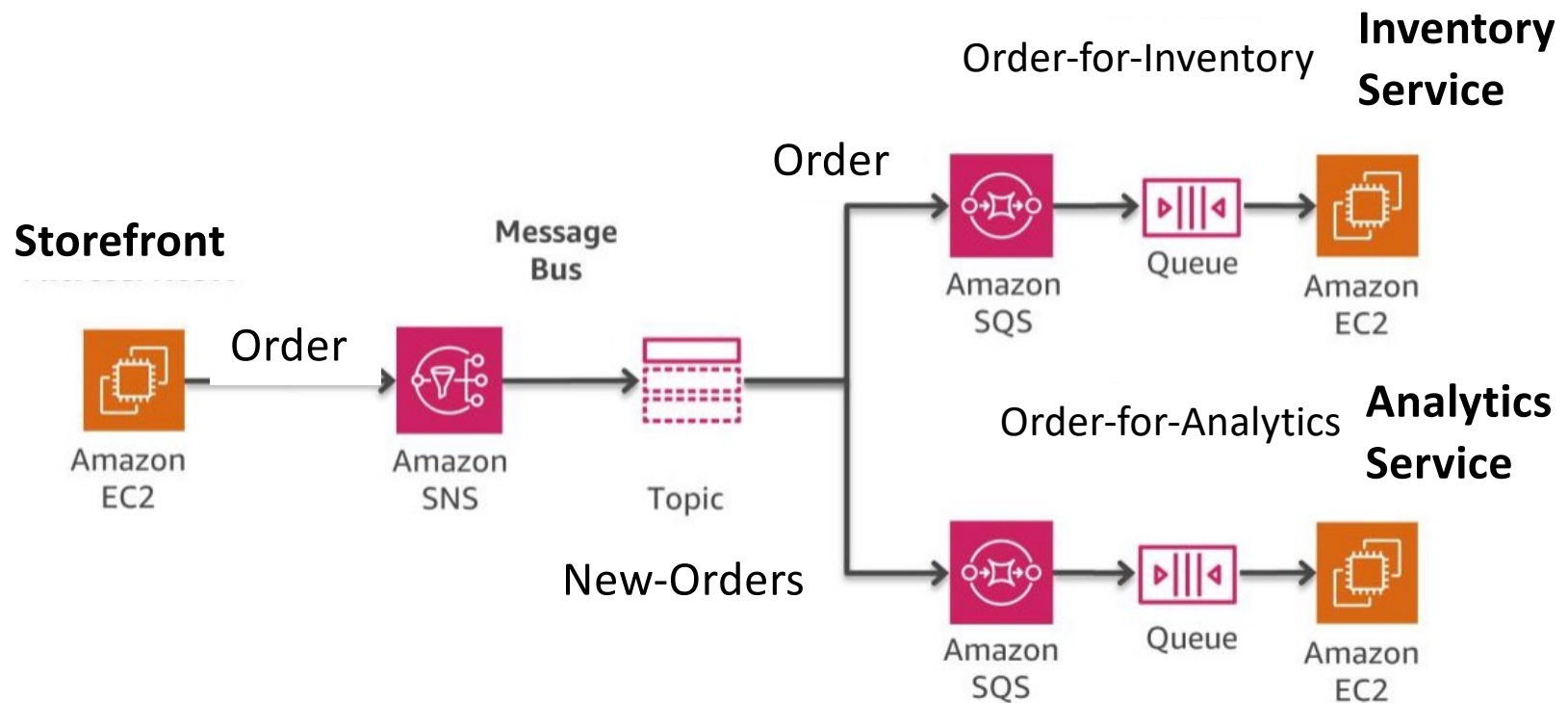


Figure 7: Message bus pattern on AWS

Fan-out of Event Notification to Queues

Connecting a Storefront to Backends



<https://aws.amazon.com/getting-started/hands-on/send-fanout-event-notifications/>

Creating a Fan-Out Structure

- Create topic 'New-Orders'
- Create 2 queues
 - Order-for-Inventory
 - Order-for-Analytics
- **Subscribe the queues to the topic**
- Test by sending a message to the topic

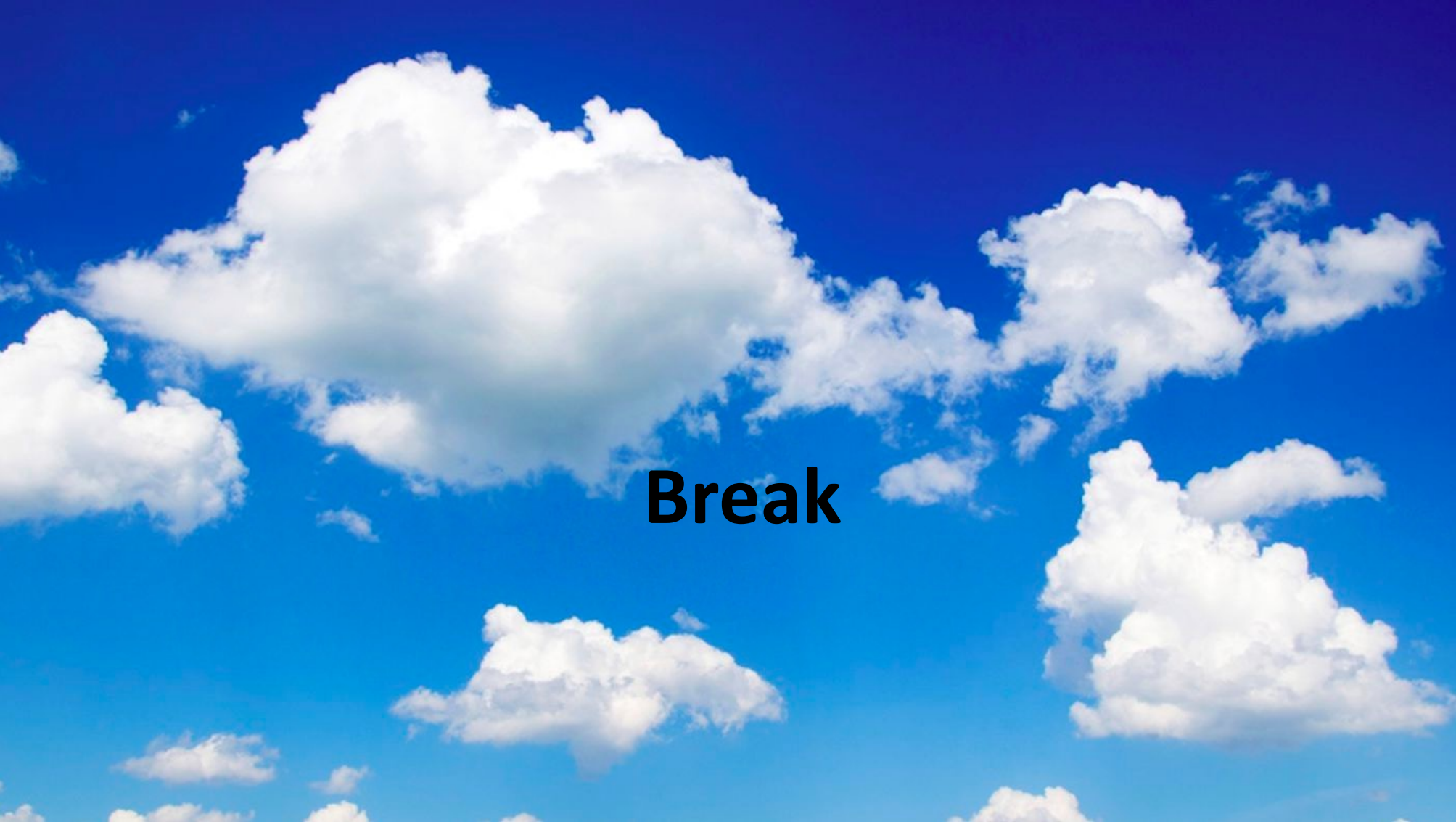
Filtering Messages

<https://aws.amazon.com/getting-started/hands-on/filter-messages-published-to-topics>

- Create topic 'Insurance-Quote-Requests'
- Create 3 queues
 - Vehicle-Insurance-Quotes
 - Life-Insurance-Quotes
 - All-Quotes
- **Subscribe the queues to the topic**
- **Create Filters**
 - Filter by 'insurance-type', e.g
{“insurance-type”: [“vehicle”]}
- Test by sending a message to the topic, specifying the attribute value for 'insurance-type'
 - If the value is a plain string, not an array, no quotation marks
 - Otherwise, use string.array with multiple values

Amazon Managed Streaming Service for Apache Kafka (Amazon MSK)

- Fully managed service for Kafka
- Not available in Learner Labs
- Highly available, self-healing cluster spanning three availability zones (recommended)
- Configurable storage and broker instance sizes
- Integrated to VPC, IAM, Key Management
- Amazon pitches this as alternative to on-premises Kafka installations
- <https://console.aws.amazon.com/msk/home?region=us-east-1#/home>



Break

Service Oriented Architecture (SOA)

Componentized

Standardized services interfaces for applications and resources

Interoperable

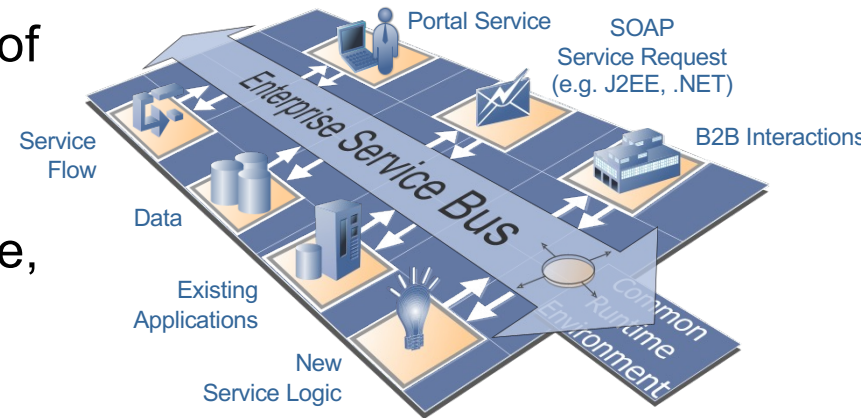
between applications and/or resources through exchange of structured information

Modular

Mix and match, add or remove, business processes and infrastructure

Scaleable

Start with what you have and add additional resources as needed



Enterprise Service Bus (ESB)

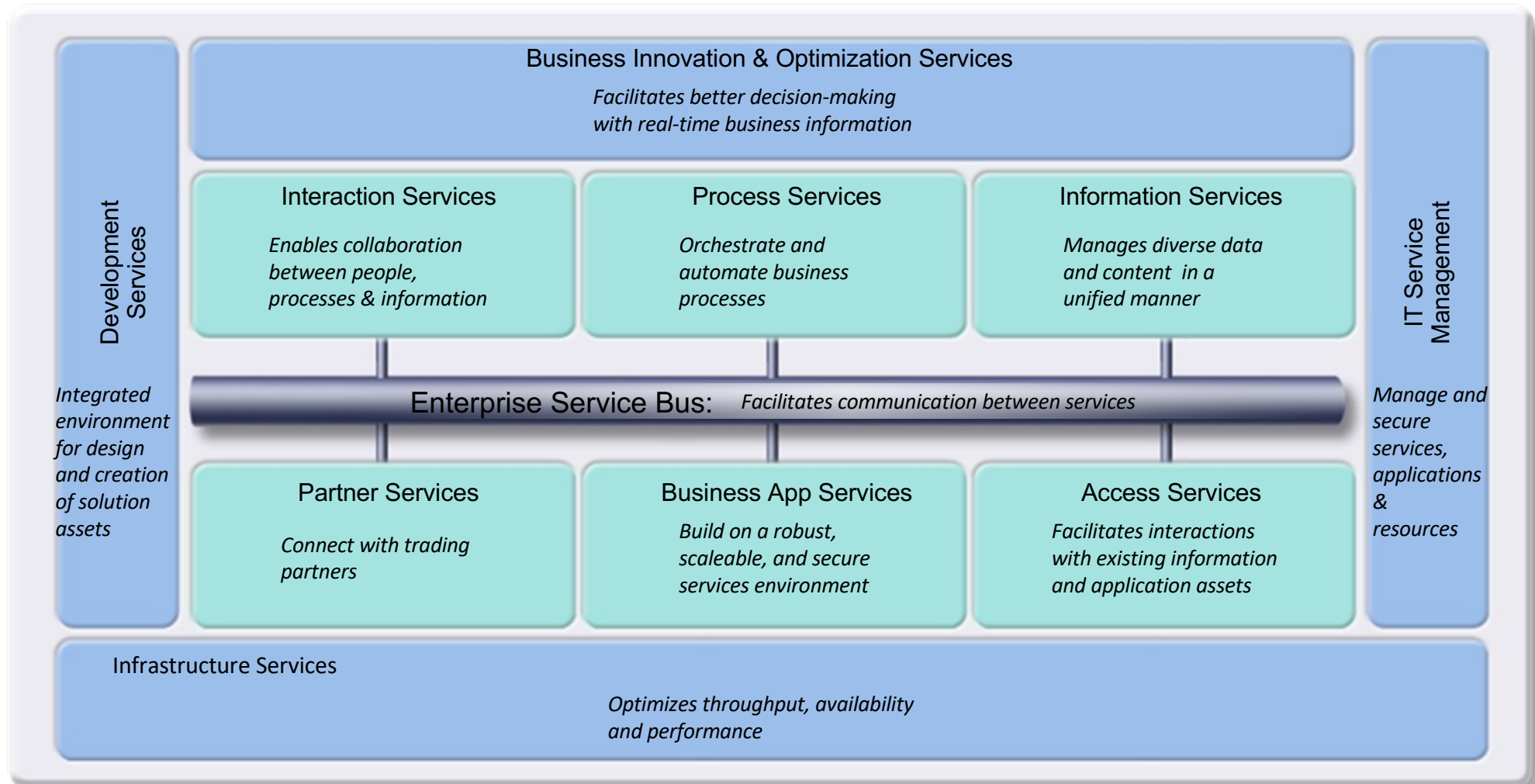
(aka Service Integration Bus)

- An **ESB** enables integration between loosely-coupled applications and services within and across
 - **message driven architectures** - where applications send messages through the ESB to receiving applications (point to point)
 - **event driven architectures** - where applications generate and consume messages anonymously
 - **services oriented architectures** – where distributed applications are composed of granular re-usable services with well-defined, published and standards-compliant interfaces
- **Mediations** within the ESB process the messages in transit
 - At the message producer, the message consumer, or distributed through the infrastructure of the Bus
 - Capabilities include:
 - Simple transformations (e.g. XML to XML translations)
 - Complex transformations (e.g. database lookups or aggregations)
 - Customized logging and content-based routing
 - Autonomic behavior, such as system and business event detection, self configuration, healing, optimization and protection

SOA Reference Architecture

(IBM, ca 2007)

Robust Middleware Capabilities Connected in an Open, Flexible Manner



Microservices

An evolution of service-oriented architectures

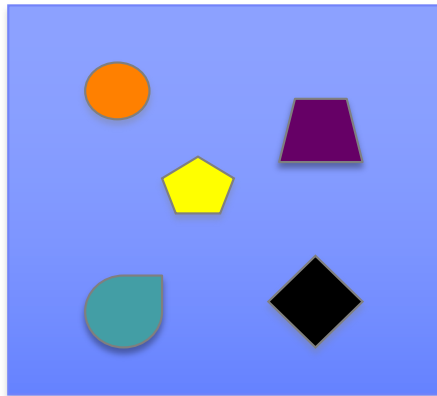
The term "Microservice Architecture" has sprung up over the last few years to describe a particular way of designing software applications as suites of independently deployable services. While there is no precise definition of this architectural style, there are certain common characteristics around organization around business capability, automated deployment, intelligence in the endpoints, and decentralized control of languages and data.

From: <http://martinfowler.com/articles/microservices.html>

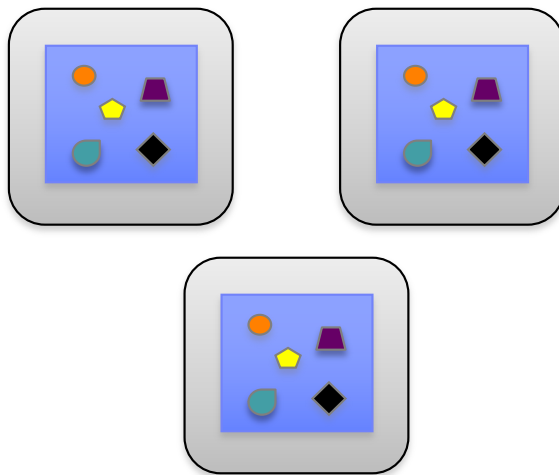
- A good tutorial can be found in:
<https://www.ibm.com/cloud/learn/microservices>
- Werner Vogels describes Amazon's motivation for microservices in his blog
<https://www.allthingsdistributed.com/2019/08/modern-applications-at-aws.html>
- Amazon site: <https://aws.amazon.com/microservices/>

Moving from monolithic applications to **micro-services**

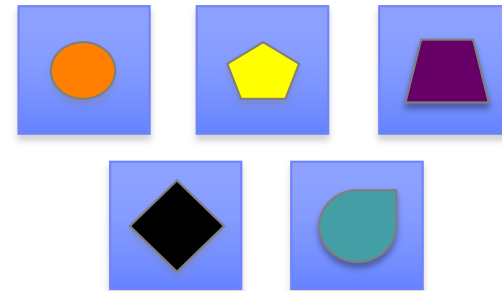
Monolithic app



Scaling

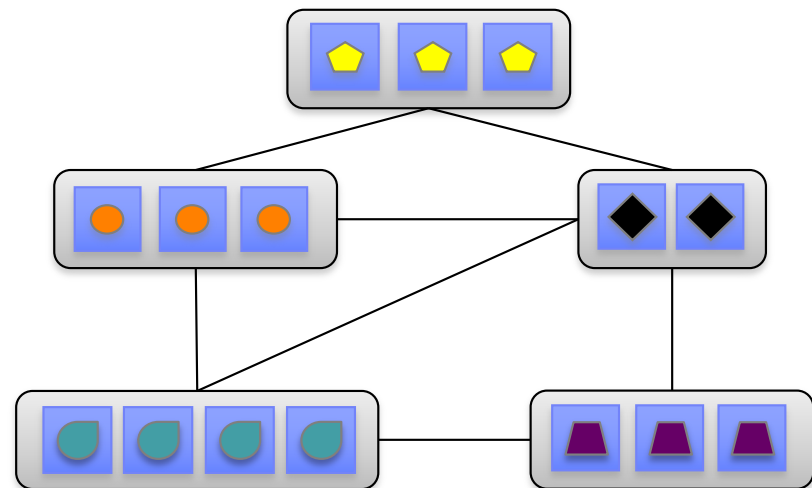


Micro services



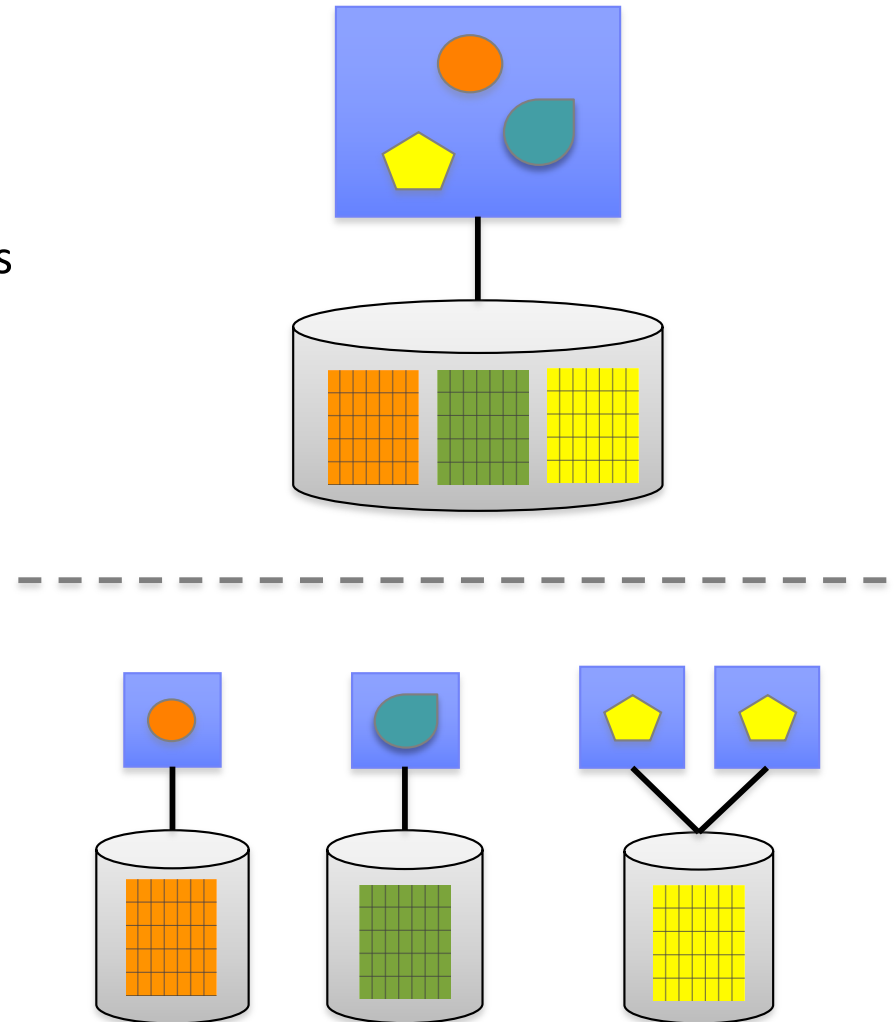
Separation of concerns

Scaling

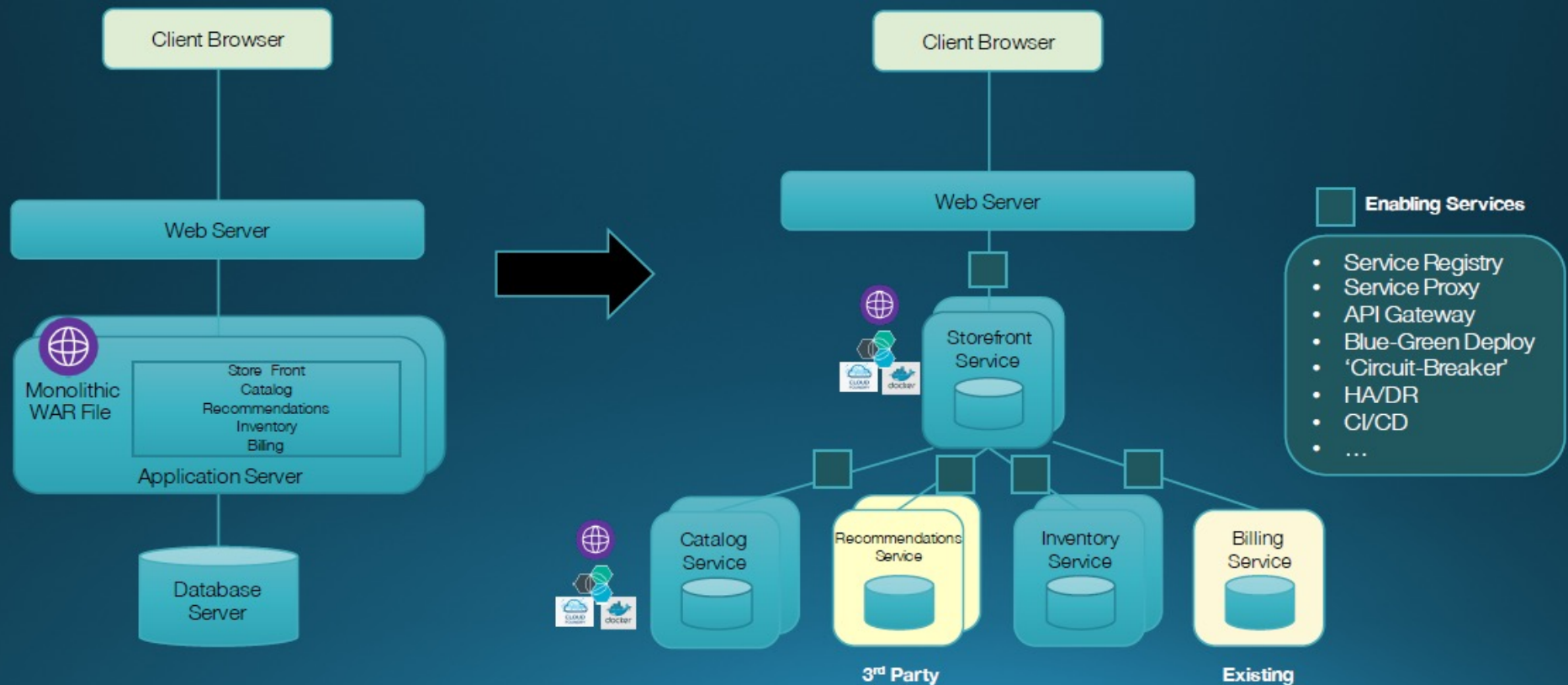


Properties of a micro-service architecture

- Compartmentalized business capability with separation of concerns
- Built and operated by cross-functional teams
- Communication via API ONLY!!
- Use ***messaging*** to remove peer-to-peer dependencies
- REST communication
- Decentralized data
- Design for failure
- Pluggable architecture
- Enables continuous delivery



Example: Evolution to a Microservices Architecture



- The microservices perform distinct business functions
- Enabling services enable communication between microservices

Microservices come with a cost

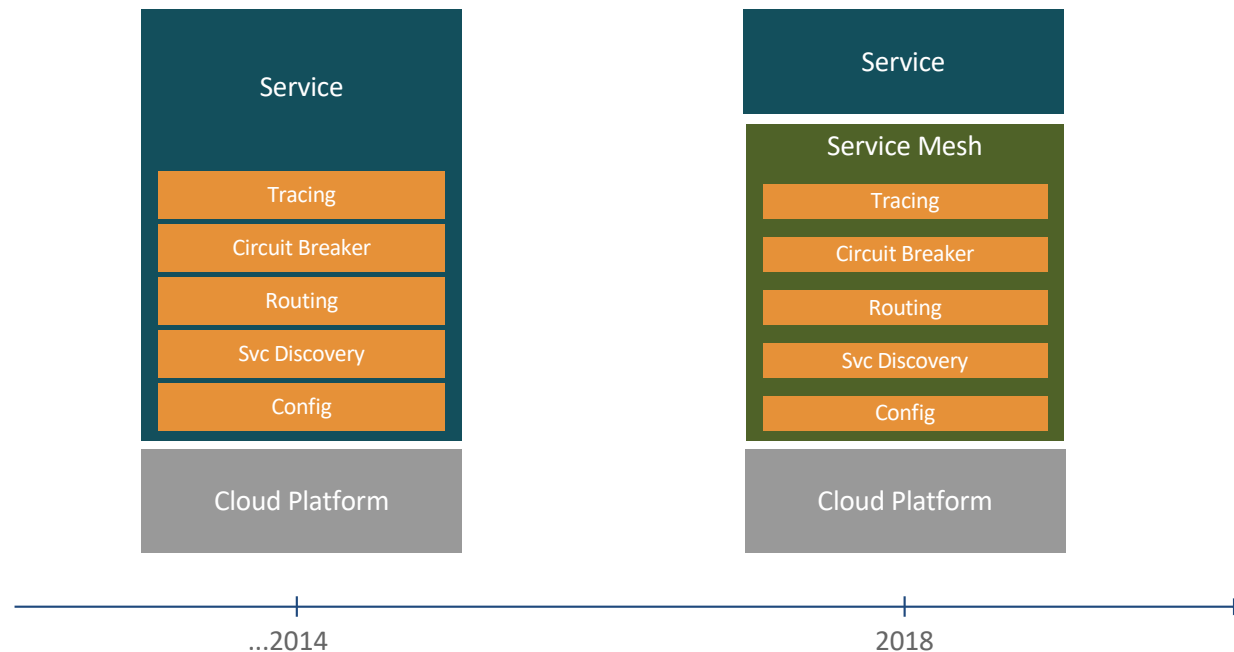
- Simple services but **complex** distributed systems
- **Designing microservices** is hard
 - Avoid heavy communication load
 - What is the right size?
- **Asynchronous communication** is difficult
- **IT overhead**
 - Configuration management
 - HA/DR for each service
 - Capacity
 - High degree of automation needed
 - Low overhead virtualization infrastructures desirable (containers)
- **API management** is a must
 - API Gateways
- **DevOps skills** are a must



Service Mesh

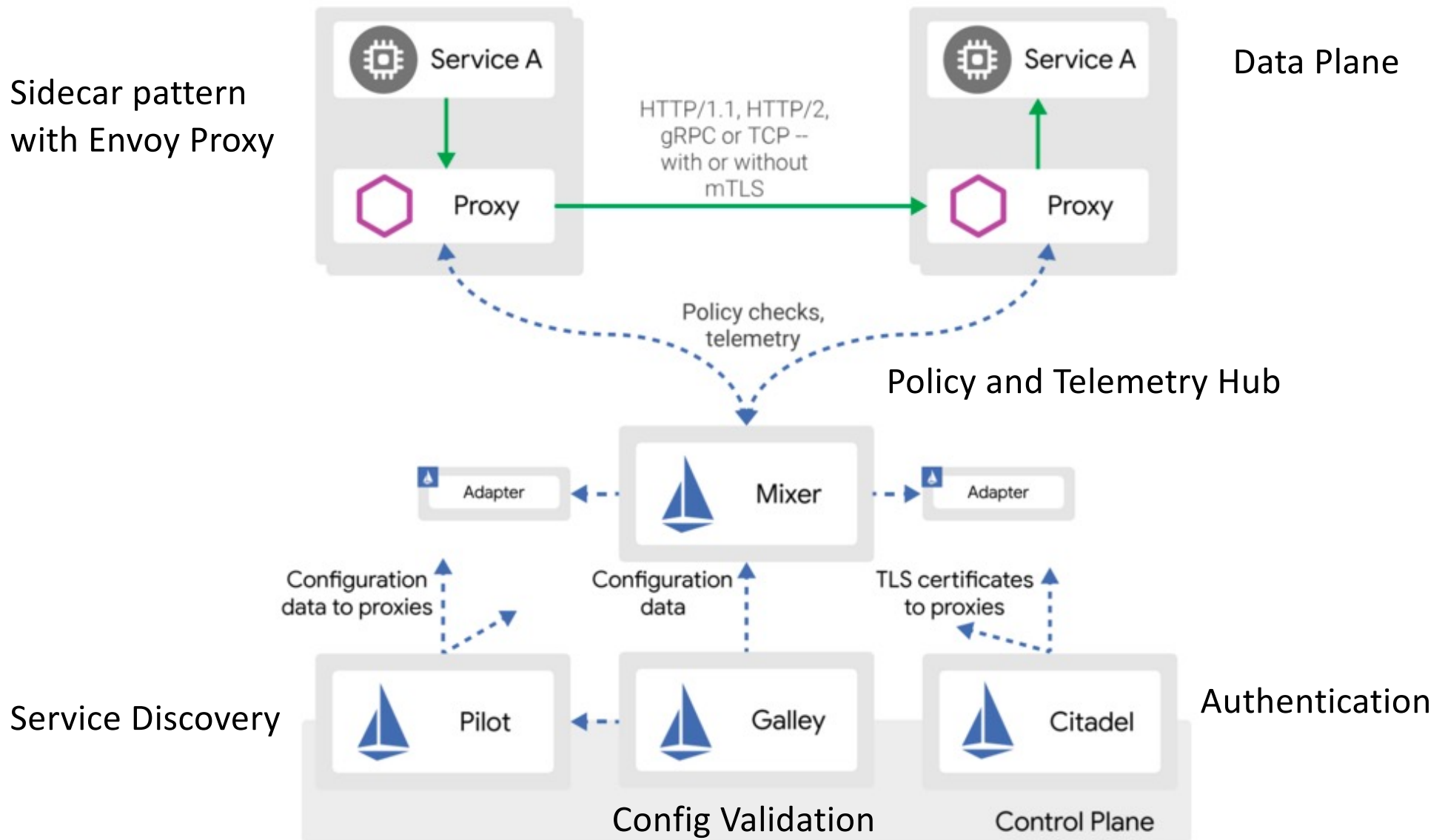
- Infrastructure layer to help manage service-to-service communication, delivering enhanced security and traffic management for microservices applications.
 - Load balancing
 - Routing rules
 - Service monitoring and logging
 - Secure cross-service communications
- Avoids 'bloating' the individual services
- Implementations are in early stages

Offloading Management Functions to a Service Mesh



Istio Service Mesh Architecture

<https://istio.io/docs/ops/deployment/architecture/>

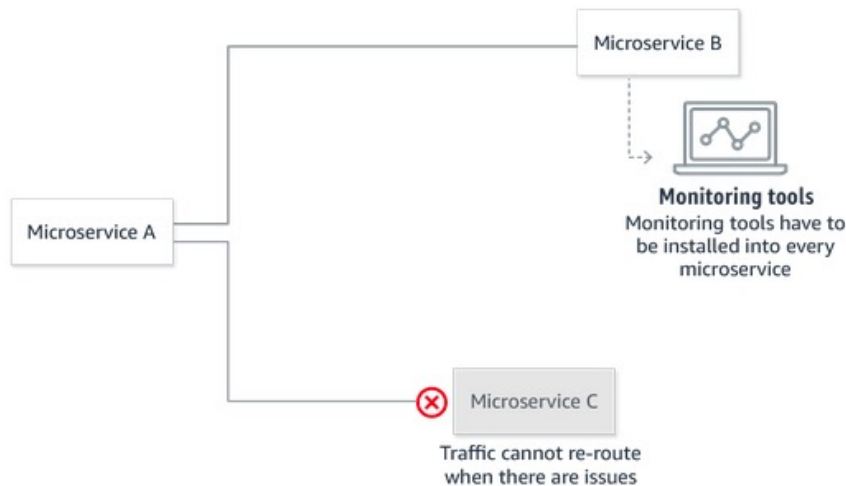


AppMesh is the Service Mesh for AWS

<https://console.aws.amazon.com/appmesh/home?region=us-east-1#>

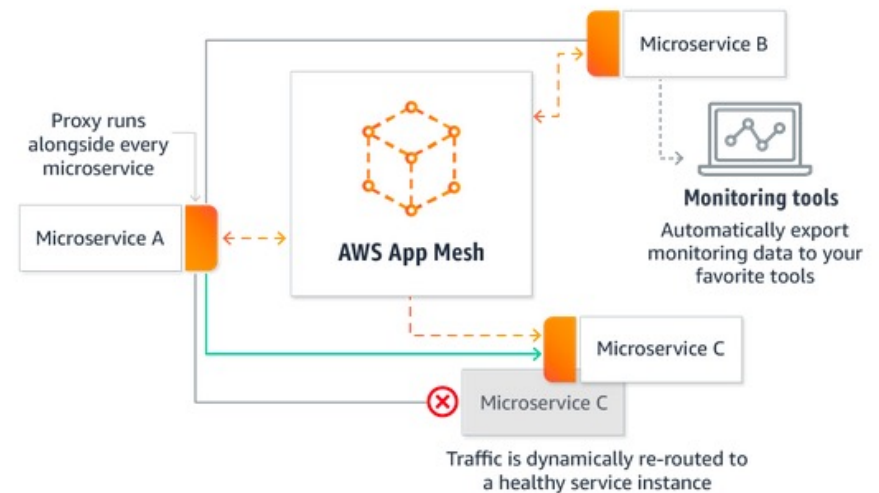
Before App Mesh

Communications and monitoring are manually configured for every microservice.



After App Mesh

App Mesh configures communications and monitoring for all microservices.



Exercises

Do the simple tutorials for SNS and SQS

- SQS

https://aws.amazon.com/getting-started/tutorials/send-messages-distributed-applications/?trk=gs_card

- SNS

https://aws.amazon.com/getting-started/tutorials/filter-messages-published-to-topics/?trk=gs_card

<https://aws.amazon.com/getting-started/hands-on/send-fanout-event-notifications/>

Read some blogs on microservices, for instance

- <https://about.gitlab.com/blog/2019/06/17/strategies-microservices-architecture/>
- <https://cloud.google.com/files/Cloud-native-approach-with-microservices.pdf>

Module 7

Cloud Platforms: Serverless



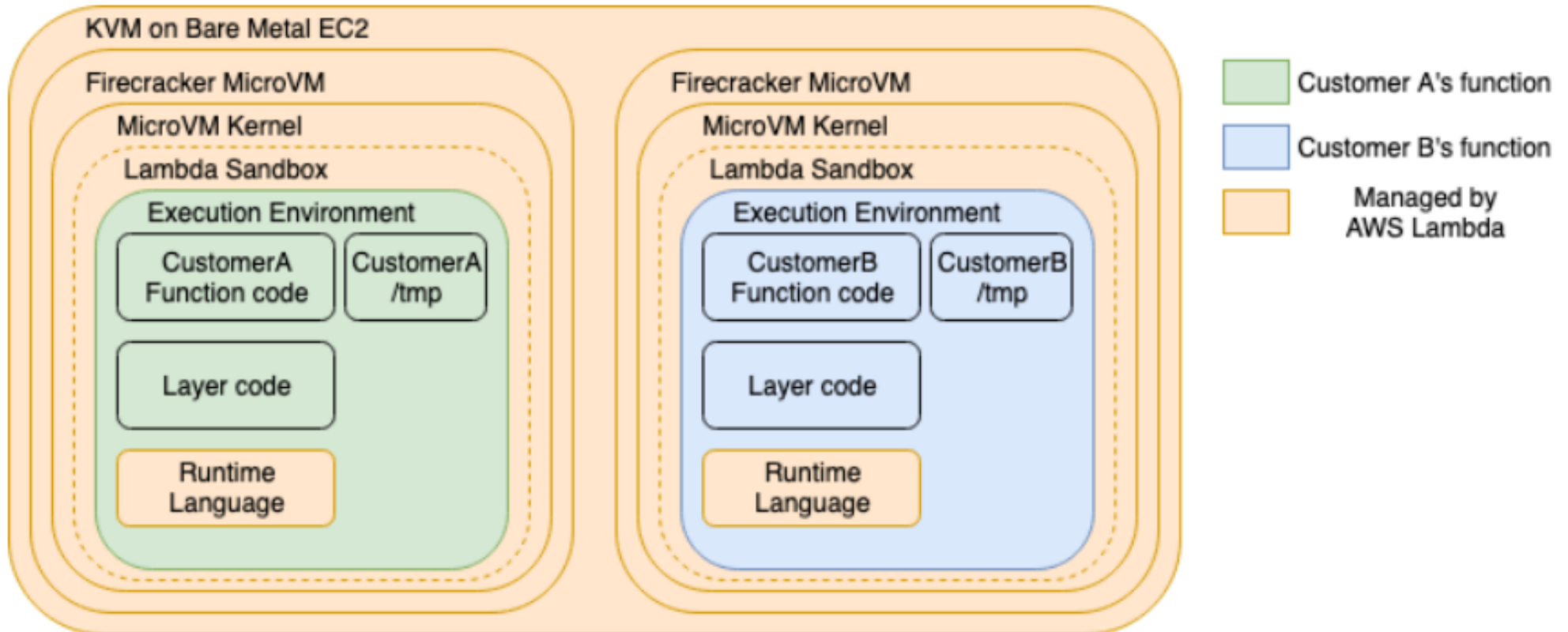
Serverless Computing in AWS

- Amazon's definition describes a managed service, while others in the industry have a narrower definition relating to 'Function as a Service' and event-based applications
- AWS: 'Serverless is the **native architecture** of the cloud that enables you to shift more of your operational responsibilities to AWS, increasing your agility and innovation'
 - Application Repository: <https://aws.amazon.com/serverless/serverlessrepo/>
- 'AWS Serverless Platform'
 - **Lambda**
 - Lambda@Edge (running at CloudFront locations)
 - AWS Fargate (Serverless Container Engine)
 - Storage Services (S3 and Elastic File System)
 - Database Services (DynamoDB and Aurora Serverless)
 - API Gateway
 - Integration Services (SNS, SQS, AppSync, EventBridge)
 - Multiple other services
- **Resources:** serverlessland.com

AWS Lambda

- <https://aws.amazon.com/lambda/>
- ‘Run code without thinking about servers. Pay only for the compute time you consume’
- You do not rent and manage virtual servers, your function is **triggered by an event** (including by other services)
 - Triggers from S3, DynamoDB, Kinesis, API Gateway ...
 - About 40 different event generators
- AWS Lambda takes care of scaling, availability etc.
 - Up to thousands of events per second
 - AWS handles trillions of events/month
 - Half of new apps on AWS use Lambda (as of 2020)
- Ideal for unpredictable workloads of short duration
 - Currently limit of 15 minutes for execution
- <https://console.aws.amazon.com/lambda/home?region=us-east-1#/discover>

Lambda Execution Environment



Events, Functions, Services

Event source



Changes in
data state



Requests to
endpoints



Changes in
resource state



Function



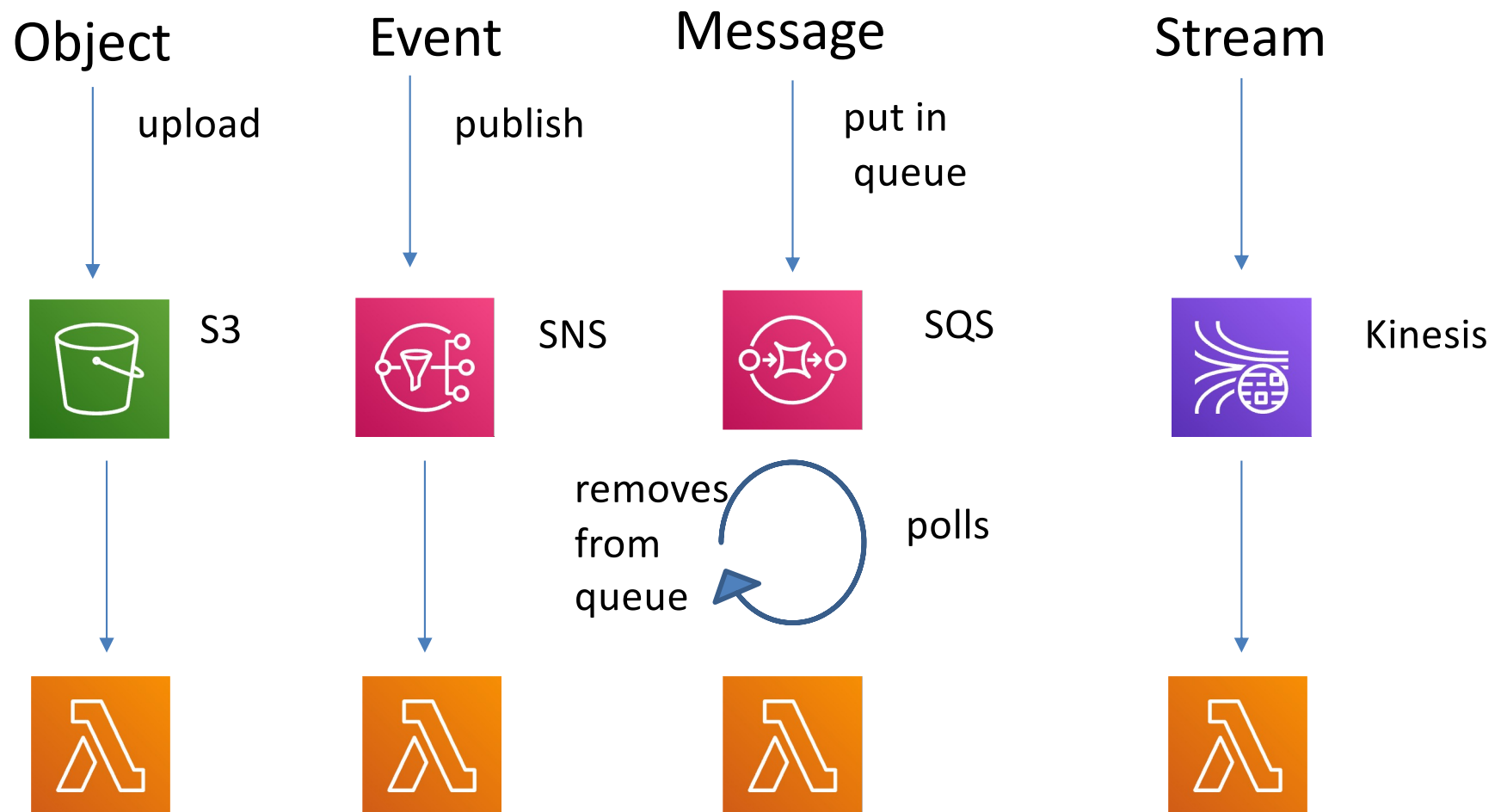
Node.js
Python
Java
C#
Go
Ruby
BYOR (Bring your own runtime)

Services (anything)



<https://aws.amazon.com/getting-started/deep-dive-serverless/?e=gs2020&p=gsrc>

Lambda Execution

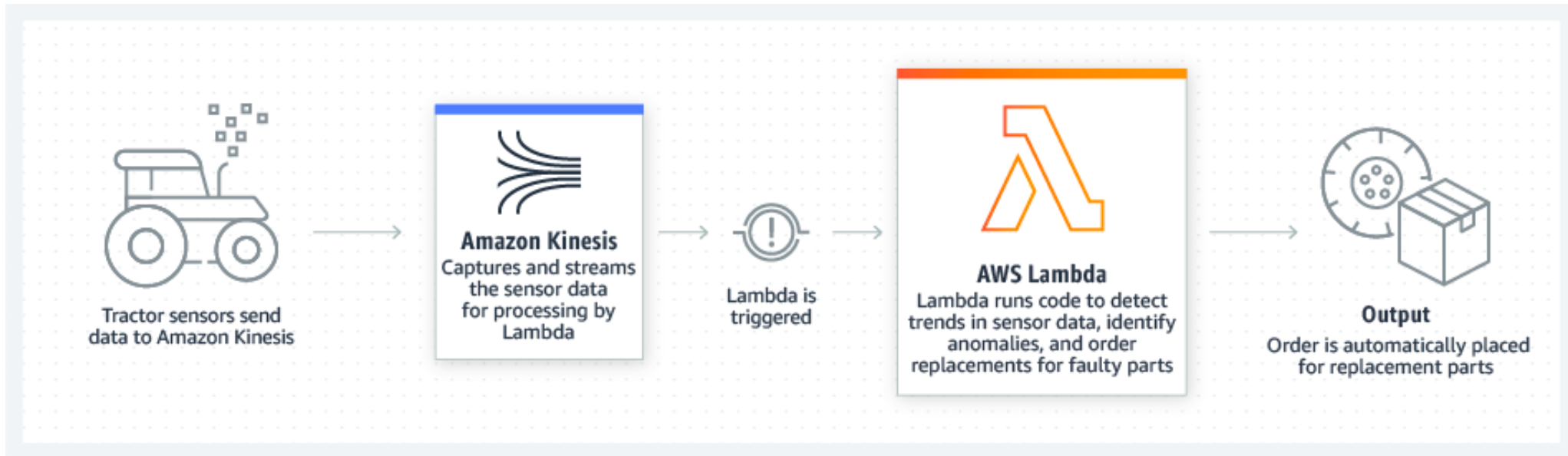


Typical Lambda Workloads

<https://docs.aws.amazon.com/lambda/latest/operatorguide>

- **Web applications:** serve the front-end code via Amazon S3 and Amazon CloudFront, or automating the entire deployment and hosting with AWS Amplify Console.
- **Web and mobile backends:** the front-ends interact with the backend via API Gateway. Integrated authorization and authentication are provided by Amazon Cognito or APN Partners like Auth0.
- **Data processing:** event-based processing tasks triggered by data changes in data stores, or streaming data ETL tasks with Amazon Kinesis and Lambda.
- **Parallelized computing tasks:** splitting highly complex, long-lived computations to individual tasks across many Lambda function instances to process data more quickly in parallel.
- **Internet of Things (IoT) workloads:** processing data generated by physical IoT devices.

Example: IoT Backend



This is a typical example for preventive maintenance that holds for many Industries.

Example: Real-Time File Processing



CreateThumbnail

AWS Lambda Concepts

- Lambda runs **functions** to process **events**.
- An **event** is a JSON formatted document that contains data for a function to process. The Lambda **runtime** converts the event to an object and passes it to your function code. Invoking a function determines the structure and contents of the event.
- All runtimes share a common programming model. You tell the runtime which method to run by defining a **handler** in the function configuration, and the runtime runs that method. The runtime passes objects to the handler that contain the invocation event and the **context**, such as the function name and request ID.
- Functions can be **synchronous** (e.g. for API Gateways), **asynchronous** (e.g. SNS, S3), or **poll-based** (DynamoDB, Kinesis)
- **Concurrency** is the number of requests that your function is serving at any given time.
- A **trigger** is a resource or service that invokes a function, it can be a program, an AWS service or an event source mapping (like reading from a stream).

Creating a Serverless Function

- From the dashboard, select 'create function'
- Three options:
 1. Create from Scratch
 2. Use a blueprint
 3. Deploy a container image
- If creating a function from scratch, provide
 - Name, runtime info, permissions
 - **Note: You have to choose LabRole as 'Custom Execution Role' under 'Additional Settings'**
 - .Net, Go, Java, Python, Node.js, Ruby
- Create Function
- This brings up the console for your application
- Select the test tab to test it out
 - You need to configure a test event

Sample Function (lambda_function.py)

```
import json
```

```
def lambda_handler(event, context):
```

```
    # TODO implement
```

```
    return {
```

```
        'statusCode': 200,
```

```
        'body': json.dumps('Hello from Lambda!')
```

```
    }
```

Sample Function (index.mjs)

```
export const handler = async (event) => {  
  // TODO implement  
  const response = {  
    statusCode: 200,  
    body: JSON.stringify('Hello from Lambda!'),  
  };  
  return response;  
};
```

Note: The handler structure changed with Node.js 24.x

Showing event content (Python)

A greeting by name

```
# import the JSON utility package since we will be working with a JSON object
import json
# define the handler function that the Lambda service will use as an entry point
def lambda_handler(event, context):
# extract values from the event object we got from the Lambda service
    name = event['firstName'] + ' ' + event['lastName']
# return a properly formatted JSON object
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda, ' + name)
    }
```

The event has the structure:

```
{
  "firstName": firstname,
  "lastName": lastname
}
```


Accessing a database from Lambda

```
# import the json utility package since we will be working with a JSON object
import json

# import the AWS SDK (for Python the package name is boto3)
import boto3

# import two packages to help us with dates and date formatting
from time import gmtime, strftime

# create a DynamoDB object using the AWS SDK
dynamodb = boto3.resource('dynamodb')

# use the DynamoDB object to select our table
table = dynamodb.Table('HelloWorldDatabase')

# store the current time in a human readable format in a variable
now = strftime("%a, %d %b %Y %H:%M:%S +0000", gmtime())

# define the handler function that the Lambda service will use as an entry point
def lambda_handler(event, context):

# extract values from the event object we got from the Lambda service and store in a variable
    name = event['firstName'] + ' ' + event['lastName']

# write name and time to the DynamoDB table using the object we instantiated and save response in a variable
    response = table.put_item(
        Item={
            'ID': name,
            'LatestGreetingTime':now
        })

# return a properly formatted JSON object
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda, ' + name)
    }
```

Example: SNS Event

```
{
  "Records": [
    {
      "EventVersion": "1.0",
      "EventSubscriptionArn": "arn:aws:sns:us-east-2:123456789012:sns-lambda:21be56ed-a058-49f5-8c98-aedd2564c486",
      "EventSource": "aws:sns",
      "Sns": {
        "SignatureVersion": "1",
        "Timestamp": "2019-01-02T12:45:07.000Z",
        "Signature": "tcc6faL2yUC6dgZdmrwh1Y4cGa/ebXEkAi6RibDsvpi+tE/1+82j...65r==",
        "SigningCertUrl": "https://sns.us-east-2.amazonaws.com/SimpleNotificationService-ac565b8b1a6c5d002d285f9598aa1d9b.pem",
        "MessageId": "95df01b4-ee98-5cb9-9903-4c221d41eb5e",
        "Message": "Hello from SNS!",
        "MessageAttributes": {
          "Test": {
            "Type": "String",
            "Value": "TestString"
          },
          "TestBinary": {
            "Type": "Binary",
            "Value": "TestBinary"
          }
        },
        "Type": "Notification",
        "UnsubscribeUrl": "https://sns.us-east-2.amazonaws.com/?Action=Unsubscribe&SubscriptionArn=arn:aws:sns:us-east-2:123456789012:test-lambda:21be56ed-a058-49f5-8c98-aedd2564c486",
        "TopicArn": "arn:aws:sns:us-east-2:123456789012:sns-lambda",
        "Subject": "TestInvoke"
      }
    }
  ]
}
```



Lambda II

Lambda Examples

- “Hello World”, accessing DynamoDB
 - Environment variable
- SNS trigger
- SQS trigger
- SNS-SQS-Lambda
- S3 - thumbnail function (a little more detail)
- Lambda-based micro-service with API Gateway

Environment Variables

- Used to store configuration information outside the code – follows 12-factor principles!
- Common use cases
 - Database connection details
 - API keys
 - Logging levels
 - Feature flags
- Example in Python for modified HelloWorldFunction

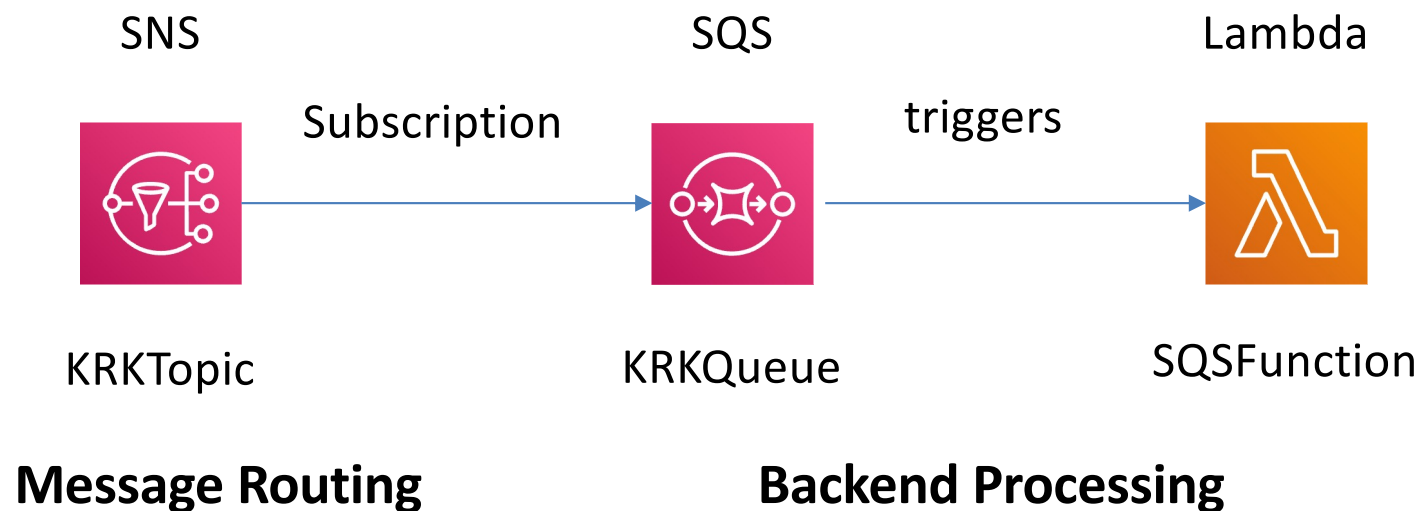
```
import os
table_name = os.environ['TABLE_NAME']
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table(table_name)
# where TABLE_NAME is specified in an environment variable
```

Blueprint Examples

- SNS-Message (Node.js or Python)
 - Name the function, choose or create role
 - Configure (and enable) an SNS Trigger (topic)
 - Create function
 - Configure a test event and run test
 - Publish a message to your topic, look at log
- SQS-Message (Node)
 - Name the function, choose or create role
 - Configure and enable an SQS queue as a trigger
 - Send a message to the queue, look at log
- S3 Metadata (Python) – underpins the Thumbnail example
 - Name the function, choose or create role
 - Configure and enable an S3 bucket as trigger
 - Configure a test event (insert bucket name and name of a public object)
 - Upload an object, look at log

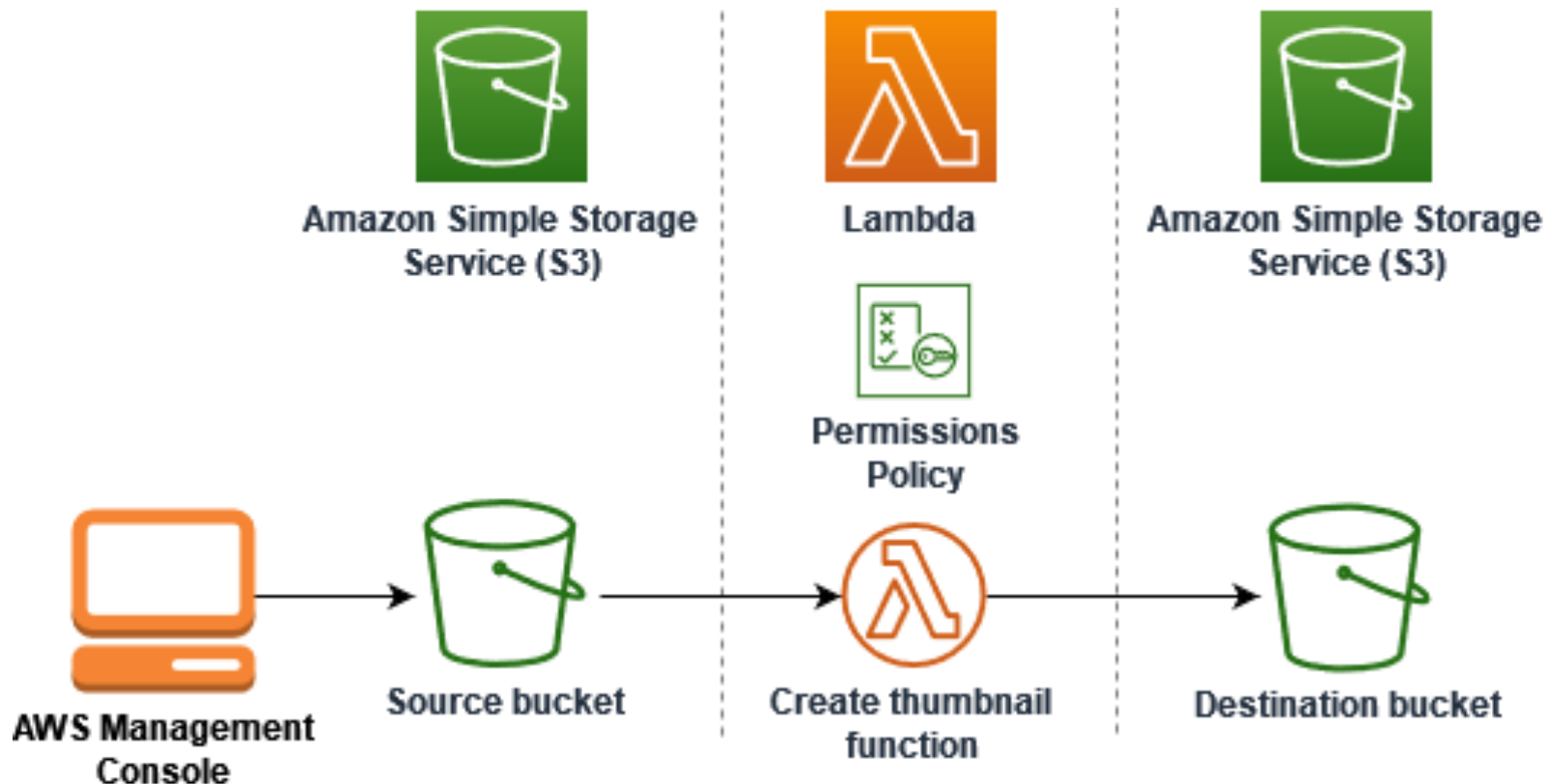
SNS to SQS to Lambda

- Use the Blueprint for SQS Message Handling
- Create a topic, e.g. KRKTopic
- Create a queue, e.g. KRKQueue and subscribe it to the Topic
- Create a Lambda Function from the Blueprint with KRKQueue as the trigger
- Publish a message to the topic and look at the Lambda function logs



Exercise: Using Lambda with S3

- <https://docs.aws.amazon.com/lambda/latest/dg/with-s3-tutorial.html>
- Creates a thumbnail in the target bucket for each image uploaded to source



Steps for Thumbnail

- Create IAM Role (**Use Lab Role for Learner Lab**)
 - Trusted entity – AWS Lambda
 - Permissions – AWSLambdaExecute
 - Role name – lambda-s3-role
- **Create a source and a target bucket**
 - Target bucket name is *source* followed by -resized, eg. krk-bucket-resized
- **Create the function code** (see tutorial, there are versions in node.js, Python and Java)
 - The function receives an Amazon S3 (upload) event input and resizes the image that is uploaded to the source bucket and saves the output to the target bucket
- Create a deployment package (see the tutorial) to include image processing libraries (or use 'deployment_package.zip' in ILIAS)
- Create the function with AWS CLI and Lambda create—function command or **upload function.zip using the console**
- **Configure your source-bucket as a trigger for the Lambda function**
- Test the Lambda function (see tutorial)
- You will have to increase memory, and potentially timeout and peripheral storage for the function – experiment to find optimal sizes

Invocation Examples (CLI)

- Synchronous – Lambda waits for a response

```
aws lambda invoke --region us-east-1 --function-name myFunction \  
  --cli-binary-format raw-in-base64-out \  
  --payload '{"key": "value"}' \  
  response.json  
{  
  "ExecutedVersion": "$LATEST",  
  "StatusCode": 200  
}
```

- Asynchronous – event handed off to Lambda

```
$ aws lambda invoke --function-name my-function --invocation-type  
Event --payload '{ "key": "value" }' response.json  
{  
  "StatusCode": 202  
}
```

Careful about the quotes!!

Serverless Microservices with API Gateway and Lambda

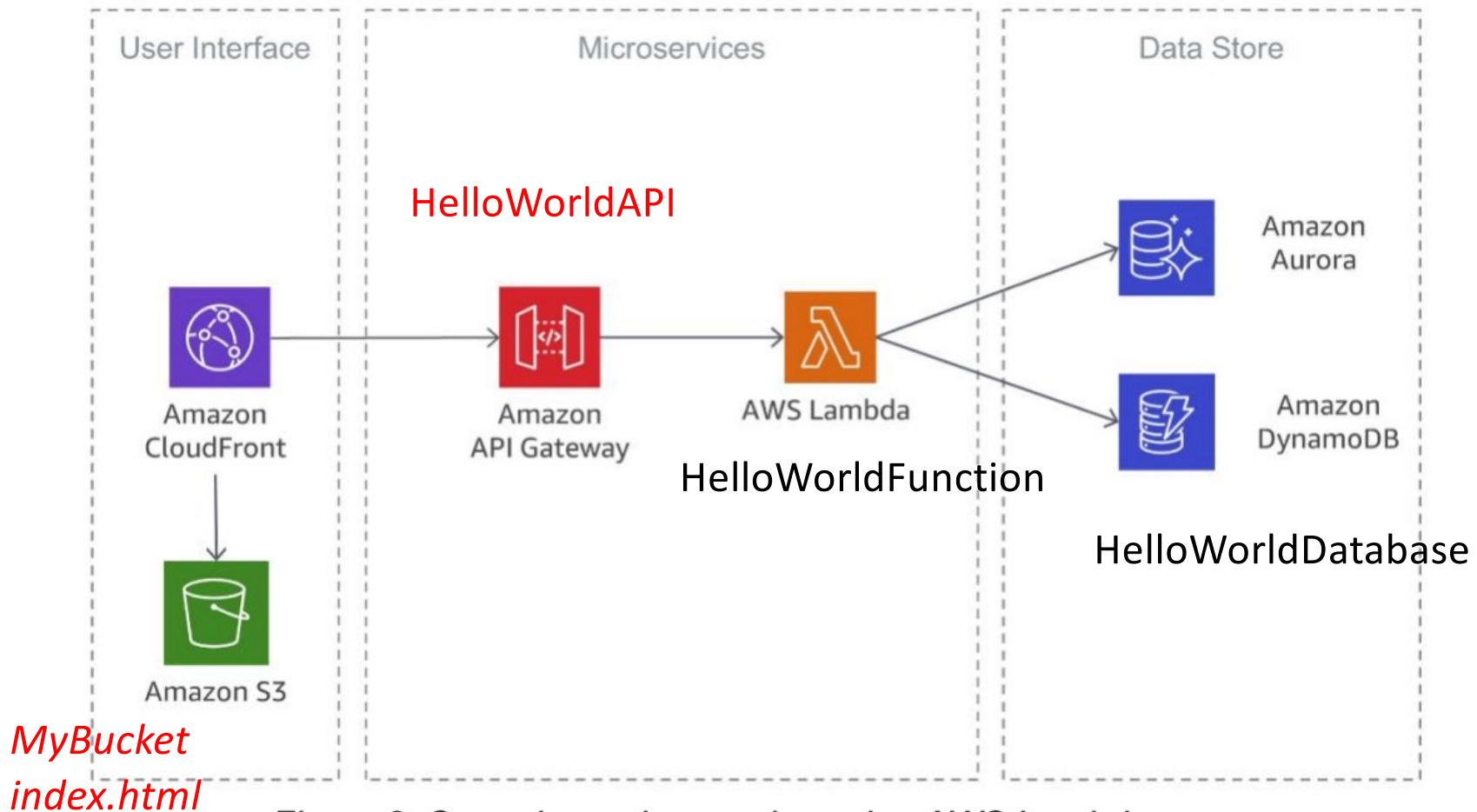


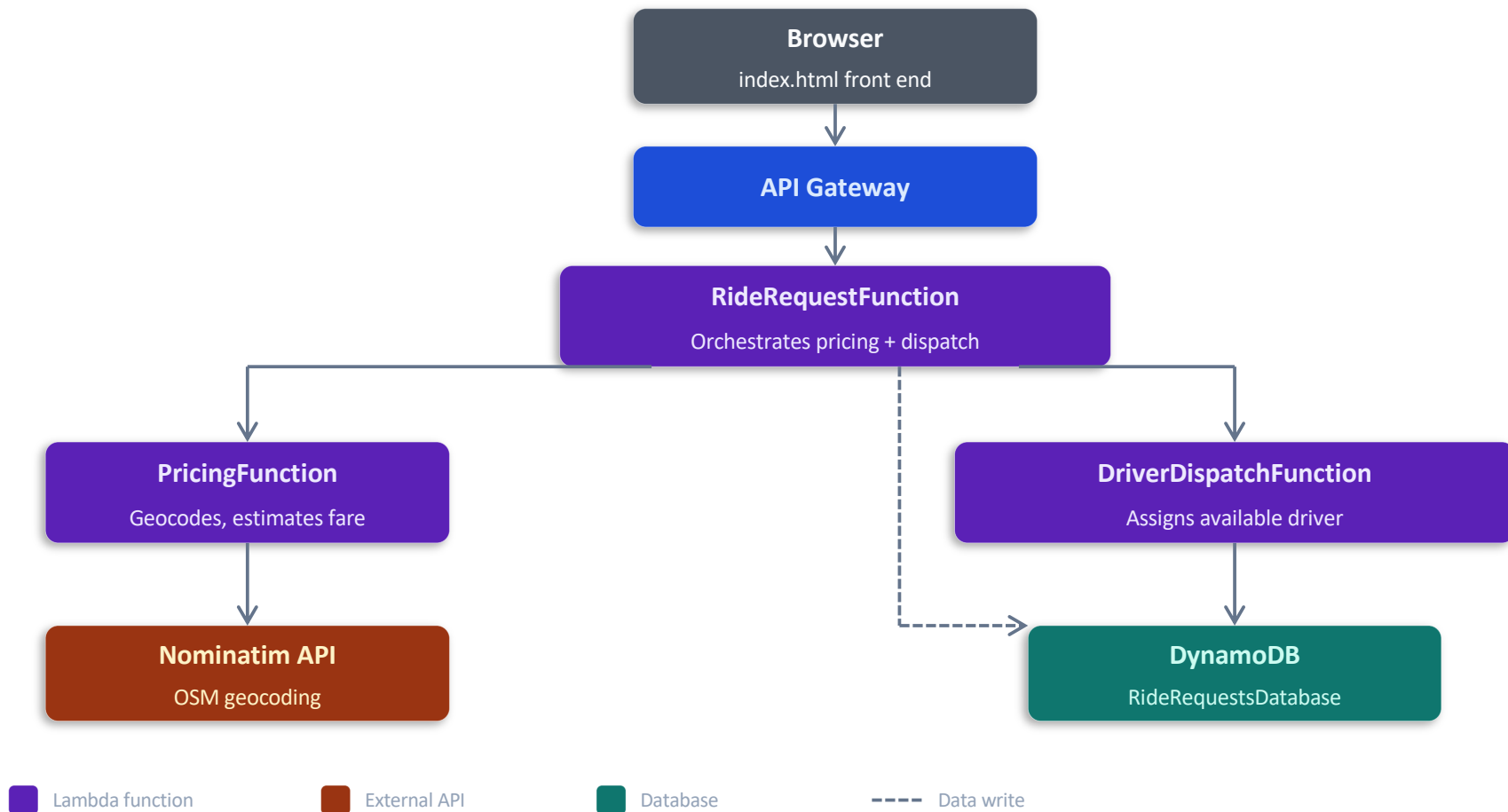
Figure 3: Serverless microservice using AWS Lambda

Source: Implementing Microservices with AWS

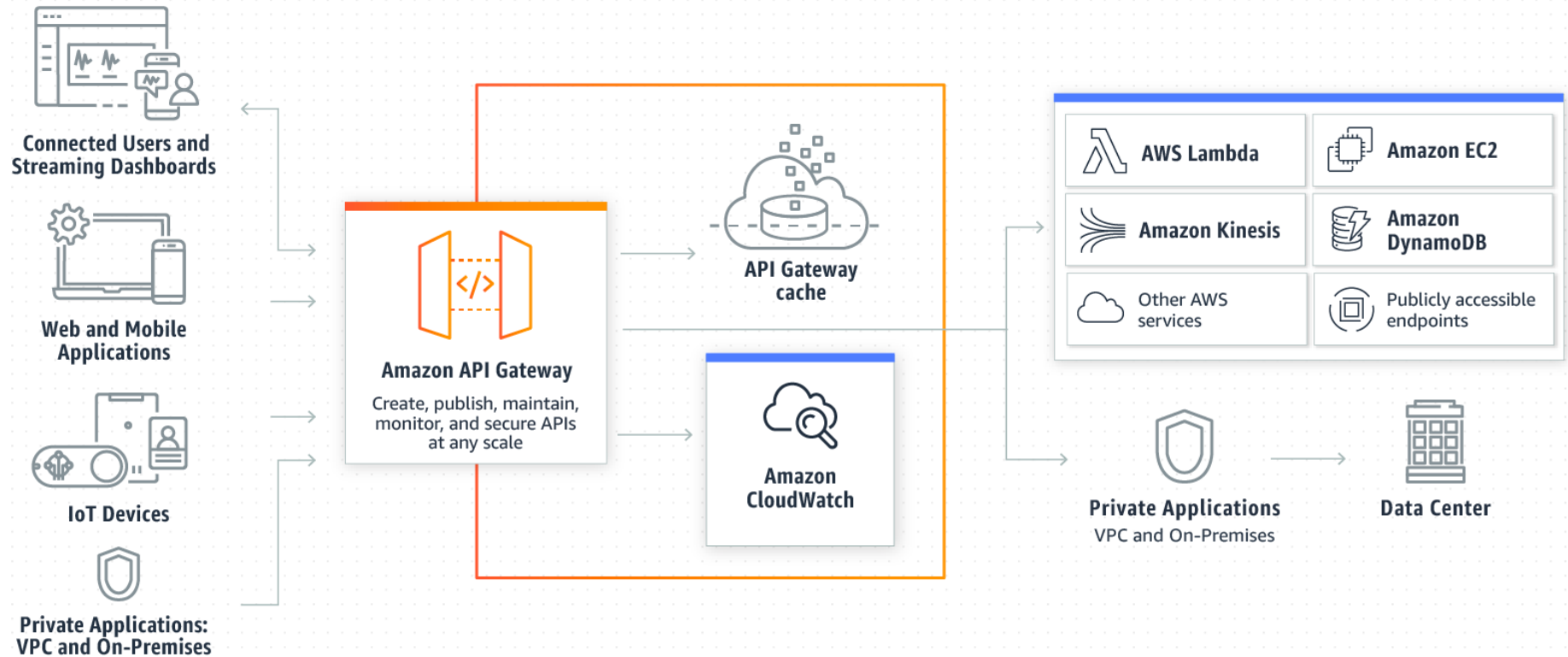
Development Steps

- Build a **Lambda Function** (HelloWorldFunction) that takes names in an event
- Build a **DynamoDB Table** to store names (HelloWorldDatabase) with sort key **ID**
- Add permission to access HelloWorldDatabase to the IAM role for HelloWorldFunction – **not needed when using LabRole for Lambda**
 - Inline policy with specific HelloWorldDatabase ARN
- Extend HelloWorldFunction to write the names passed in the event to HelloWorldDatabase
- Build a **REST API** (HelloWorldAPI) to connect the Lambda Function with the Web Application
 - A PUT (or POST) method with Lambda integration type (HelloWorldFunction), **no** Lambda Proxy
 - Enable CORS (for resource /). CORS – Cross-Origin Resource Sharing
- Build a **static web site** using an S3 bucket – index.html
 - Index.html will invoke the API Gateway
 - You can also create an Amplify application from S3

Ride-Hailing Lambda Dispatch — Architecture



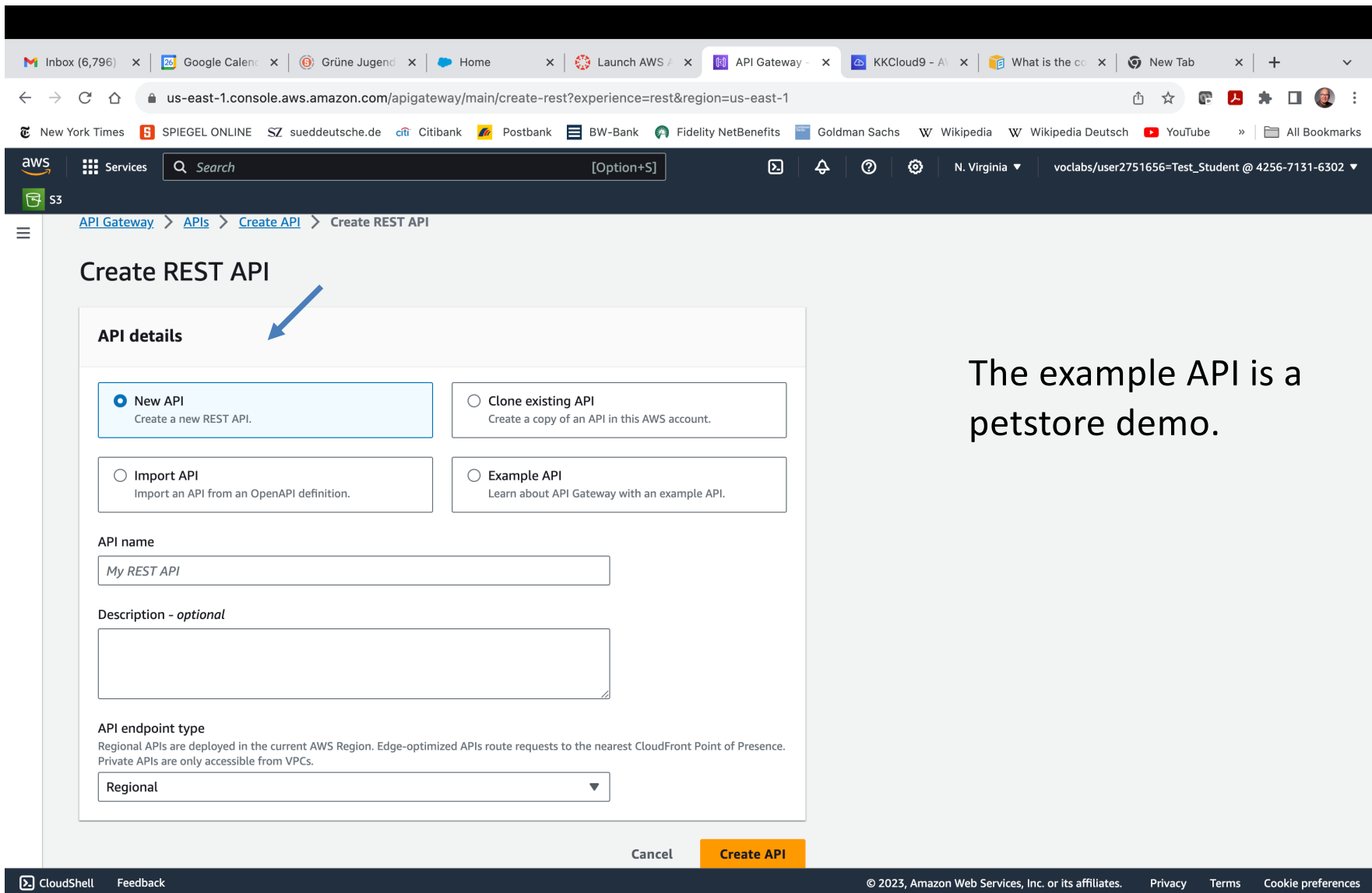
Amazon API Gateway



API Gateway

- API Gateways enable developers to create, publish and manage APIs
- Gateways serve as a **control point** for applications to access backend services
- They provide authorization, traffic management, version control etc.
- Amazon API Gateway options:
 - HTTP API for RESTful APIs that only require proxy functions
 - REST API for full API management
 - Websocket API for persistent two-way connection between clients and backend, e.g. for chat apps
- <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>
- API Gateway supports the OpenAPI standard (with Swagger)
- You pay only when your API gets invoked

Creating APIs



The screenshot shows the AWS Management Console interface for creating a REST API. The browser address bar indicates the URL: `us-east-1.console.aws.amazon.com/apigateway/main/create-rest?experience=rest®ion=us-east-1`. The console header shows the user is logged in as `voclabs/user2751656=Test_Student @ 4256-7131-6302` in the `N. Virginia` region.

The main content area displays the **Create REST API** wizard. The **API details** section is highlighted with a blue arrow. It contains four radio button options:

- ☒ **New API**
Create a new REST API.
- ☐ **Clone existing API**
Create a copy of an API in this AWS account.
- ☐ **Import API**
Import an API from an OpenAPI definition.
- ☐ **Example API**
Learn about API Gateway with an example API.

Below the options, the **API name** field is populated with `My REST API`. The **Description - optional** field is empty. The **API endpoint type** dropdown is set to `Regional`. A note explains that Regional APIs are deployed in the current AWS Region and are accessible from VPCs.

At the bottom of the wizard, there are **Cancel** and **Create API** buttons.

The example API is a petstore demo.

Resources and Methods

The screenshot shows the AWS API Gateway console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and various service icons. Below the navigation bar, there's a blue banner with a message about the new API Gateway console experience. A green banner below that indicates the successful creation of a REST API named 'PetStore (udyzh2omo2)'. The main content area is titled 'Resources' and shows the breadcrumb 'API Gateway > APIs > Resources - PetStore (udyzh2omo2)'. On the left, there's a sidebar with a 'Create resource' button and a tree view showing the resource hierarchy: '/' (selected), '/pets' (with GET method), and '/{petId}' (with GET and OPTIONS methods). The main area is divided into two sections: 'Resource details' and 'Methods (1)'. The 'Resource details' section shows the path '/' and the resource ID '0wwrh4t910'. The 'Methods (1)' section shows a table with one method: GET, Mock integration type, None authorization, and Not required API key. Two blue arrows point to the 'Resource details' and 'Methods' sections. At the bottom, there's a footer with 'CloudShell', 'Feedback', and copyright information.

us-east-1.console.aws.amazon.com/apigateway/main/apis/udyzh2omo2/resources?api=udyzh2omo2&experience=rest®ion=us-east-1

Introducing the new API Gateway console experience
We've redesigned the API Gateway console for REST APIs and WebSocket APIs. Continue to use the new console and [let us know what you think](#). Or you can [use the old console](#).

Successfully created REST API 'PetStore (udyzh2omo2)'

API Gateway > APIs > Resources - PetStore (udyzh2omo2)

Resources

Create resource

/

GET

/pets

GET

OPTIONS

POST

{petId}

GET

OPTIONS

Resource details

Update documentation

Enable CORS

Path: /

Resource ID: 0wwrh4t910

Methods (1)

Delete

Create method

	Method type	Integration type	Authorization	API key
☐	GET	Mock	None	Not required

CloudShell Feedback

© 2023, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Create a HelloWorld API with a POST Method

- Create a REST API
 - New API
- The resource page comes up, select the "/" resource .
- Choose 'Create Method' in the lower box
- On the next page, Select POST (or PUT) for Method Type
- Select Lambda Function for the Integration type.
- Do not select Lambda Proxy!
- Select the Lambda Region you used when making the function (or else you will see a warning box reading "You do not have any Lambda Functions in...").
- Enter *HelloWorldFunction* in the Lambda Function field.
- Create the method (orange button)
- Enable CORS if you want to access the API through a web page
- Deploy the API (select orange deploy button), choose stage name 'dev'
- From the deployment details, copy the Invoke URL for further use (in the web page)

Validate the API

- In the left navigation pane, select Resources.
- The methods for our API will now be listed on the right. Choose POST.
- In the lower box on the right, choose the 'Test' tab
- Paste the following into the Request Body field:

```
{  
  "firstName":"Grace",  
  "lastName":"Hopper"  
}
```
- Push the orange Test button
- You should see a response with Code 200.

API Design Considerations

- Basics
 - Platform independence
 - Design for evolution (versioning)
- URLs around data models and resources, not verbs
 - */orders* not */create-orders*
- Avoid chattiness
 - e.g. fetch multiple pieces of information in one call
- Filtering and Pagination
 - No GET requests over a large number of resources
- Versioning
 - Clear deprecation policy

<https://techcommunity.microsoft.com/t5/apps-on-azure-blog/step-by-step-practical-guide-to-architecting-cloud-native/ba-p/4057960>

Serverless Microservices with API Gateway and Lambda

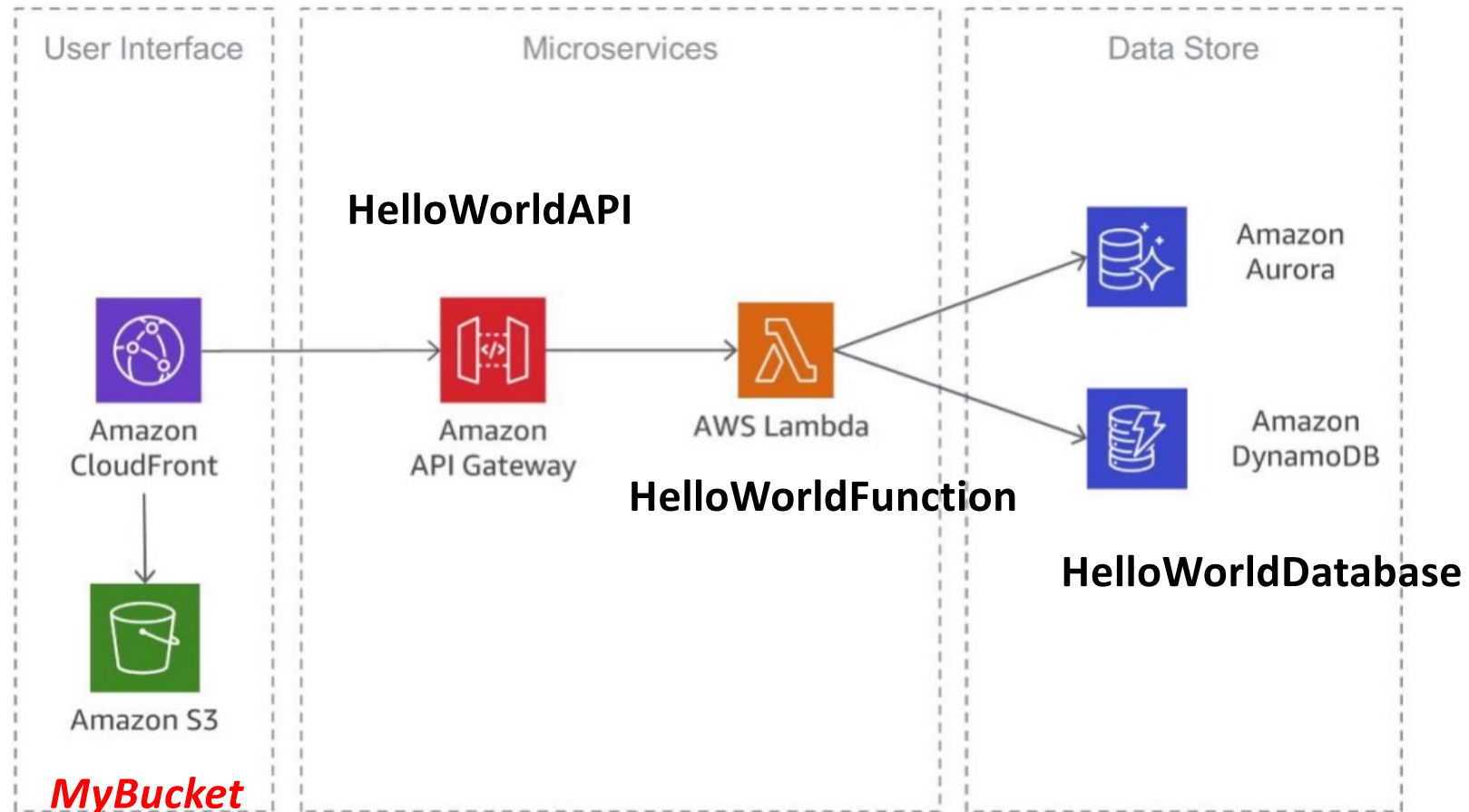


Figure 3: Serverless microservice using AWS Lambda

Source: Implementing Microservices with AWS

Accessing the API from the Web

- Create an S3 bucket whose objects can be public
- Enable static webhosting
- Invoke the API through a web page
 - **Use the API Invoke URL**
- Alternatively, use Amplify

Lambda Resources

- <https://aws.amazon.com/lambda/getting-started/>
- Tutorials:
 - CRUD API with Lambda and DynamoDB
<https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-dynamodb.html>
 - Basic Web Application
https://aws.amazon.com/getting-started/hands-on/build-web-app-s3-lambda-api-gateway-dynamodb/?trk=gs_card
 - Serverless workshops
<https://github.com/aws-samples/aws-serverless-workshops>
- Lambda Practice
 - <https://medium.com/@csjcode/avoid-these-aws-lambda-mistakes-checklist-symptoms-solutions-e32124cb2118>
- Serverless resources from AWS: serverlessland.com
- github.com/serverless/serverless: Github repo with a serverless framework

serverlessland.com

- Building event driven architectures
 - SQS, SNS, Lambda, Step Functions
- 405 serverless patterns
- 49 workflows
- 51 code snippets
- Learning paths
 - Innovator Island
 - <https://serverlessland.com/learn/innovator-island>

Serverless Best Practices

<https://www.digitalocean.com/blog/best-practices-for-serverless-computing>

- One event, one task
 - Faster startup, easier test, small attach surface
- Keep functions short-lived
- Remember that functions are stateless
 - Use databases for persistent data
- Test and configure
 - Optimize duration, memory needs
- Secure your function
- Future-proof your setup

Well-Architected Best Practices for Lambda

<https://www.datadoghq.com/blog/well-architected-serverless-applications-best-practices/>

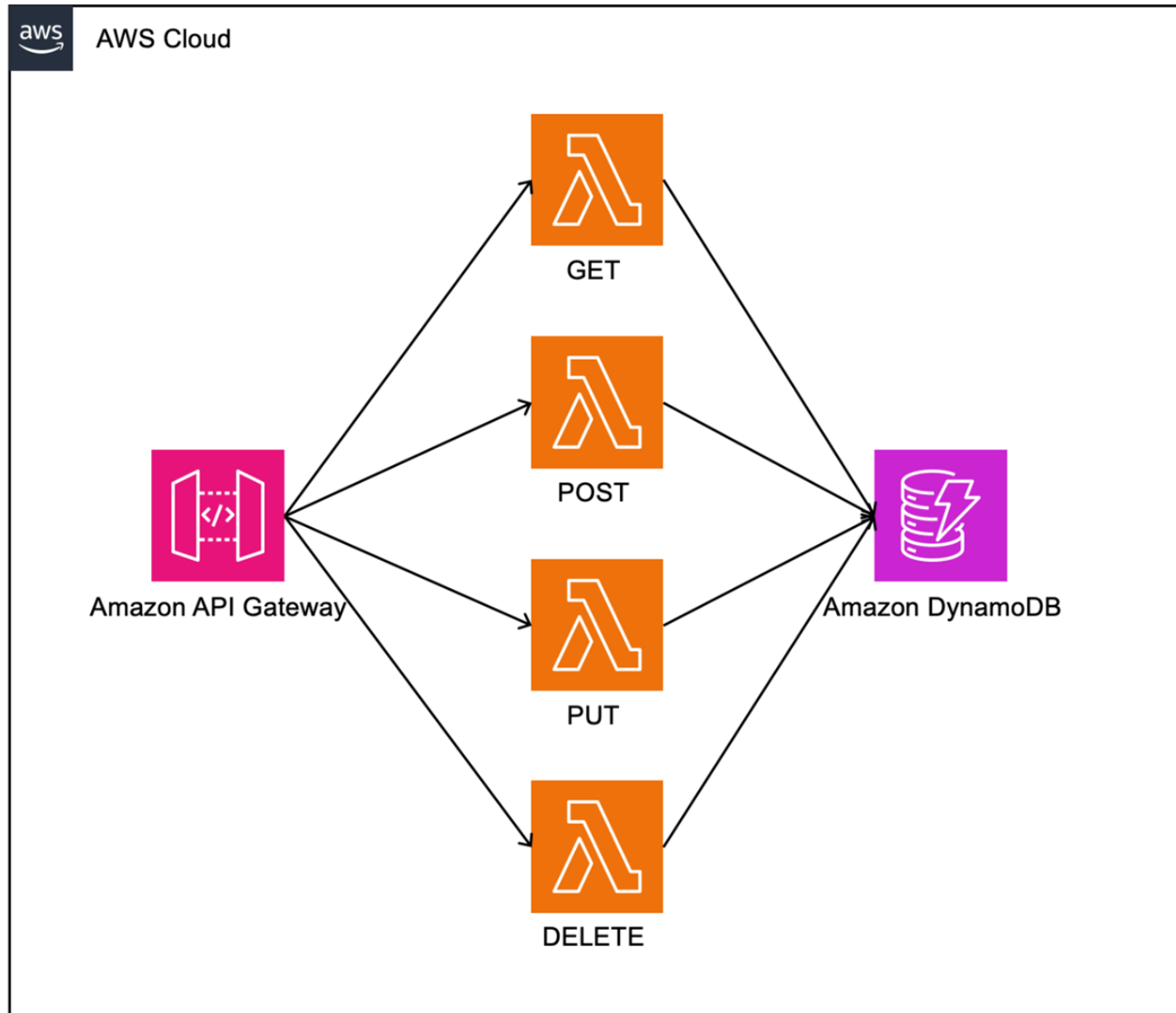
- Limit Lambda privileges (attach policies to roles)
- Limit access to application through VPC (with at least 2 subnets)
- Manage failures (retry behavior)
- Reduce cold starts, use caching, keep packages small (reduced initialization)
- Optimize memory size (test)
- Don't overspend on provisioned concurrency
 - Consider auto-scaling
- Monitor resource usage (a plug for Datadog, but good practice)
- Standardize logs (ditto)

Design Approaches for Serverless Microservices

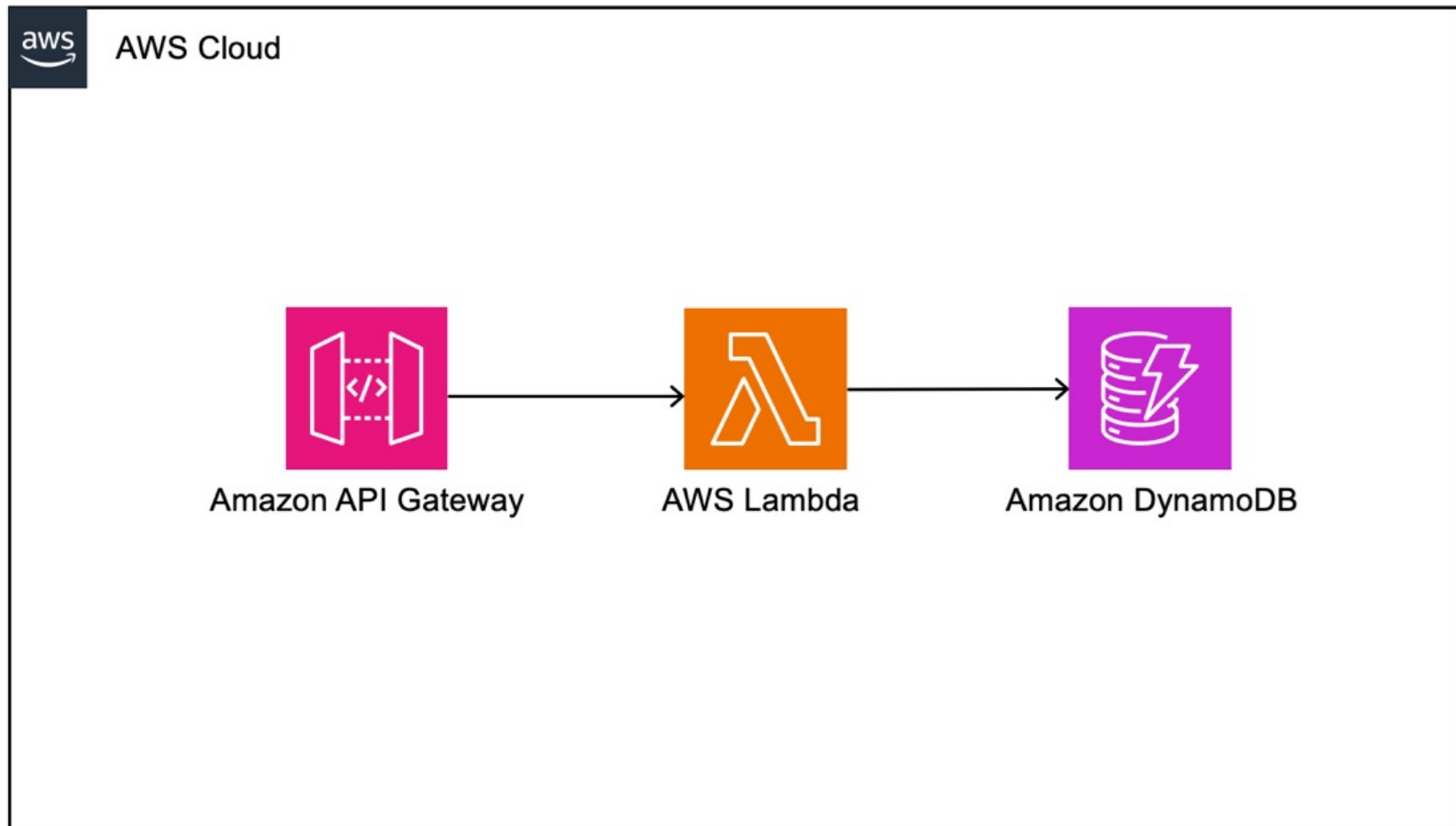
<https://aws.amazon.com/blogs/compute/comparing-design-approaches-for-building-serverless-microservices>

- Single responsibility Lambda function
 - (+) Separation of concerns, highly granular, fast execution
 - (-) Code duplication, maintenance, more cold starts
- Lambda-lith
 - (+) Code cohesion, simpler maintenance, fewer cold starts
 - (-) Longer cold start, coarse-grained config
- Separation of read and write functions (CQRS)
 - (+) Performance optimization, code cohesion where needed
 - (-) Introduces eventual consistency, pot. 2 data models

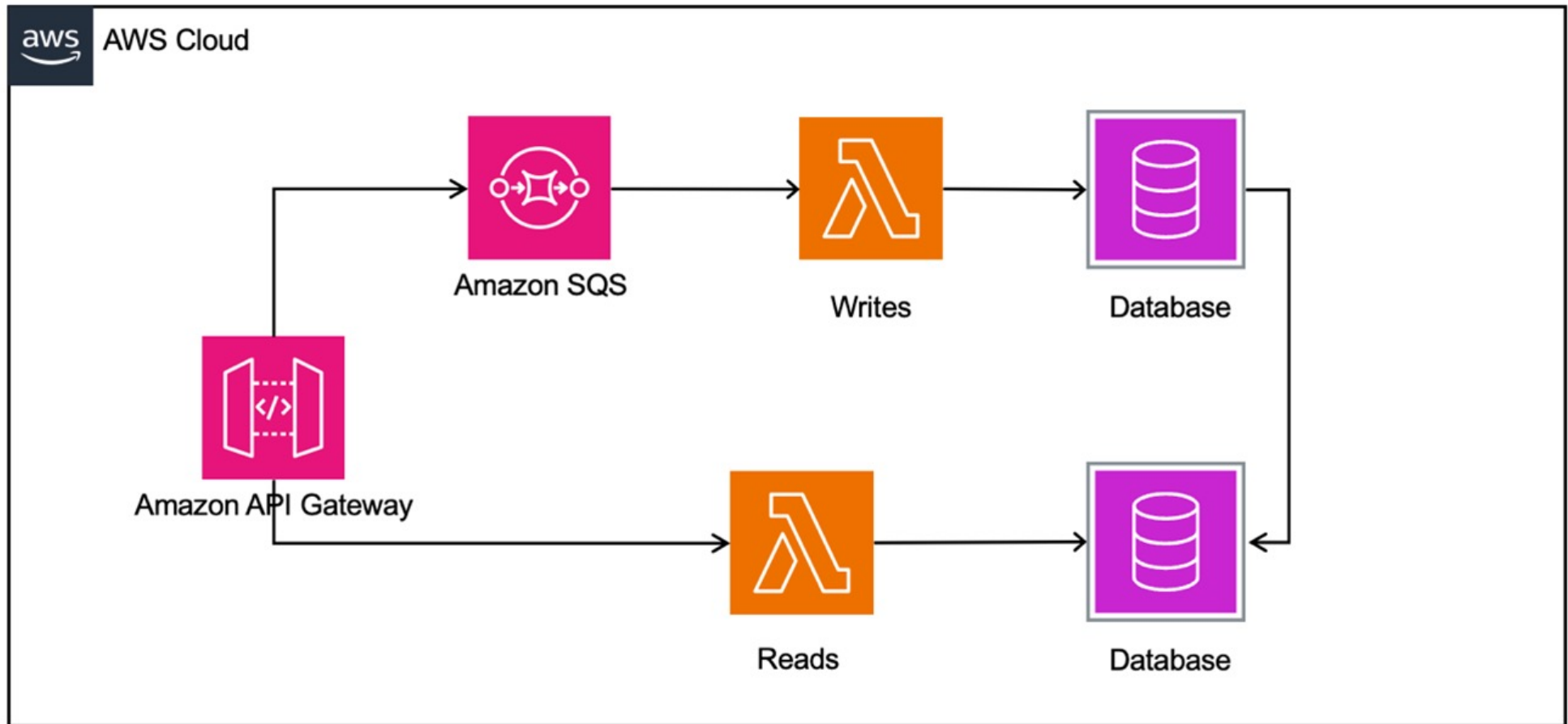
Single Responsibility

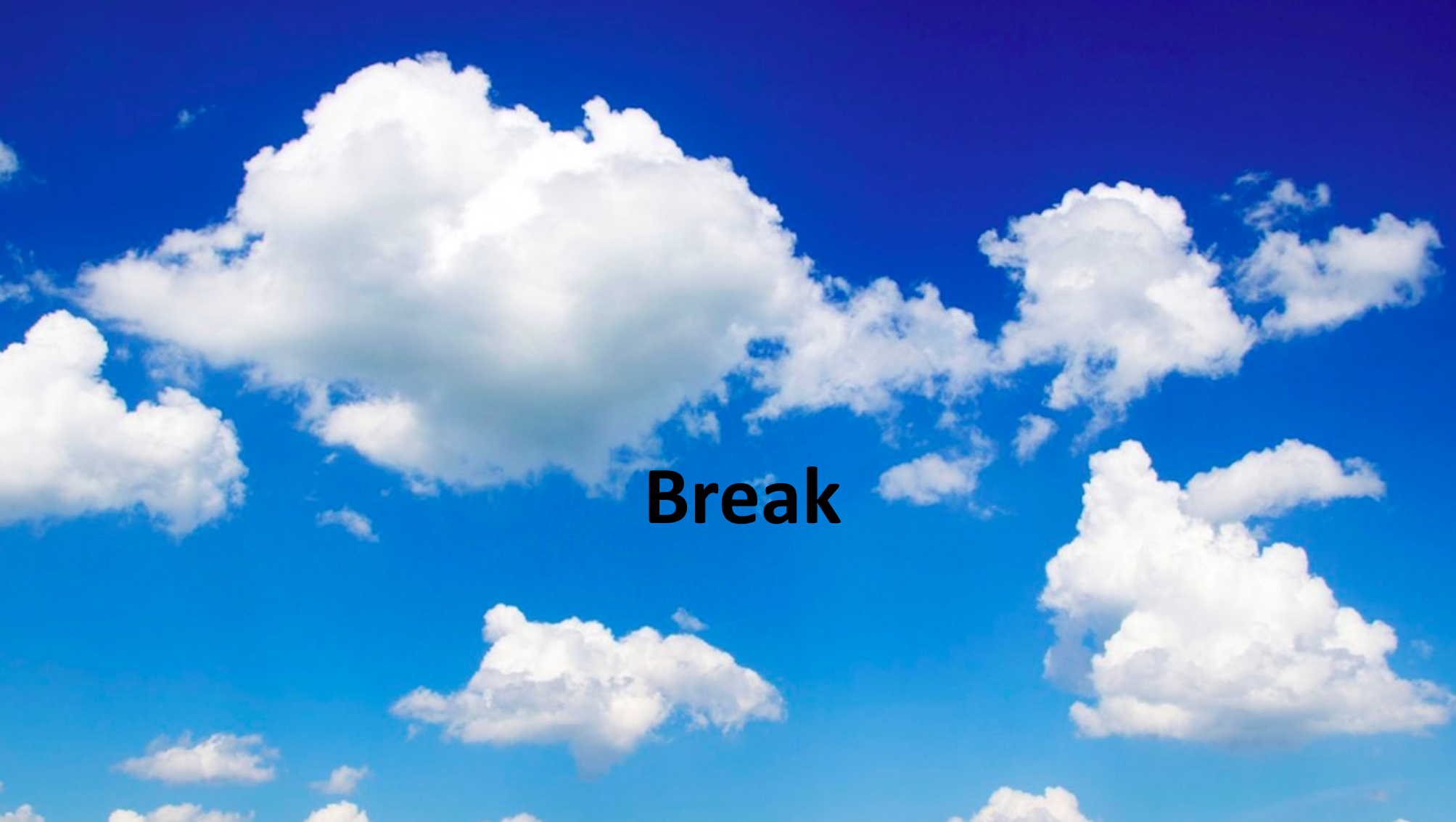


Lambda-ith



Read-write





Break

Backup Considerations

- Use **CloudFormation Stacks** (IaC) for your application setup
- Tag everything (name, project, environment)
 - **MyApplications** can be used for discovery, but doesn't substitute for a tagging strategy
- Use versioning throughout, also on S3
- Store all code artifacts, including templates, app configurations, backup plans **in GitHub** for lifecycle management
 - Separate repositories for infrastructure, code, operations
- Develop automation (**CLI**)
- Use native tools for data backup (RDS, EC2, EBS etc.)
- Consider using AWS Backup

Modules 8 and 9

Cloud Platforms: Containers



Containers

- Benefits and origins
 - Packaging concept for application runtimes
 - Portability & staging of environments
 - Separation of concerns for DevOps
 - Efficiency
- Docker
- AWS: Elastic Container Services & Registry
- Kubernetes and EKS

Levinson, Marc: “The Box: How the Shipping Container Made the World Smaller and the World Economy Bigger”; Princeton University Press, 2008

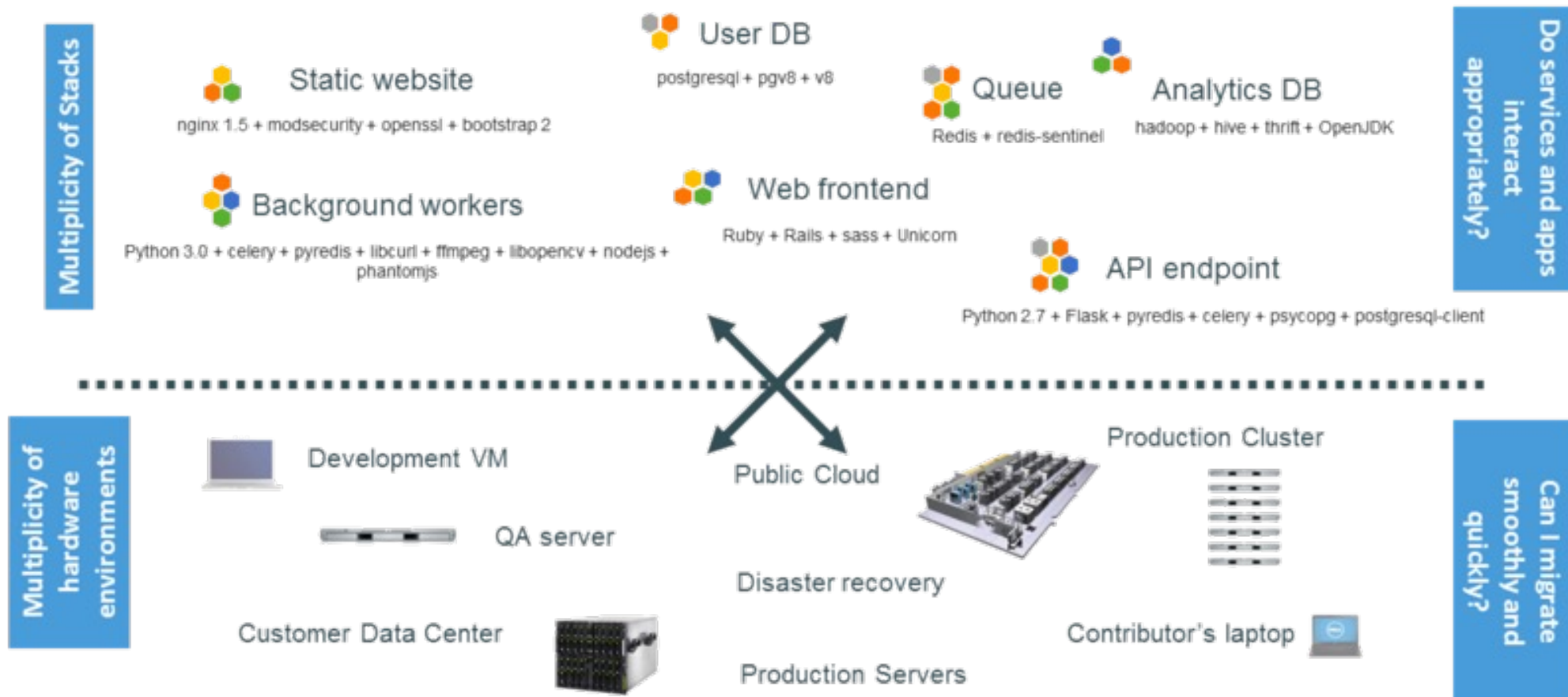
An Analogy: Transportation



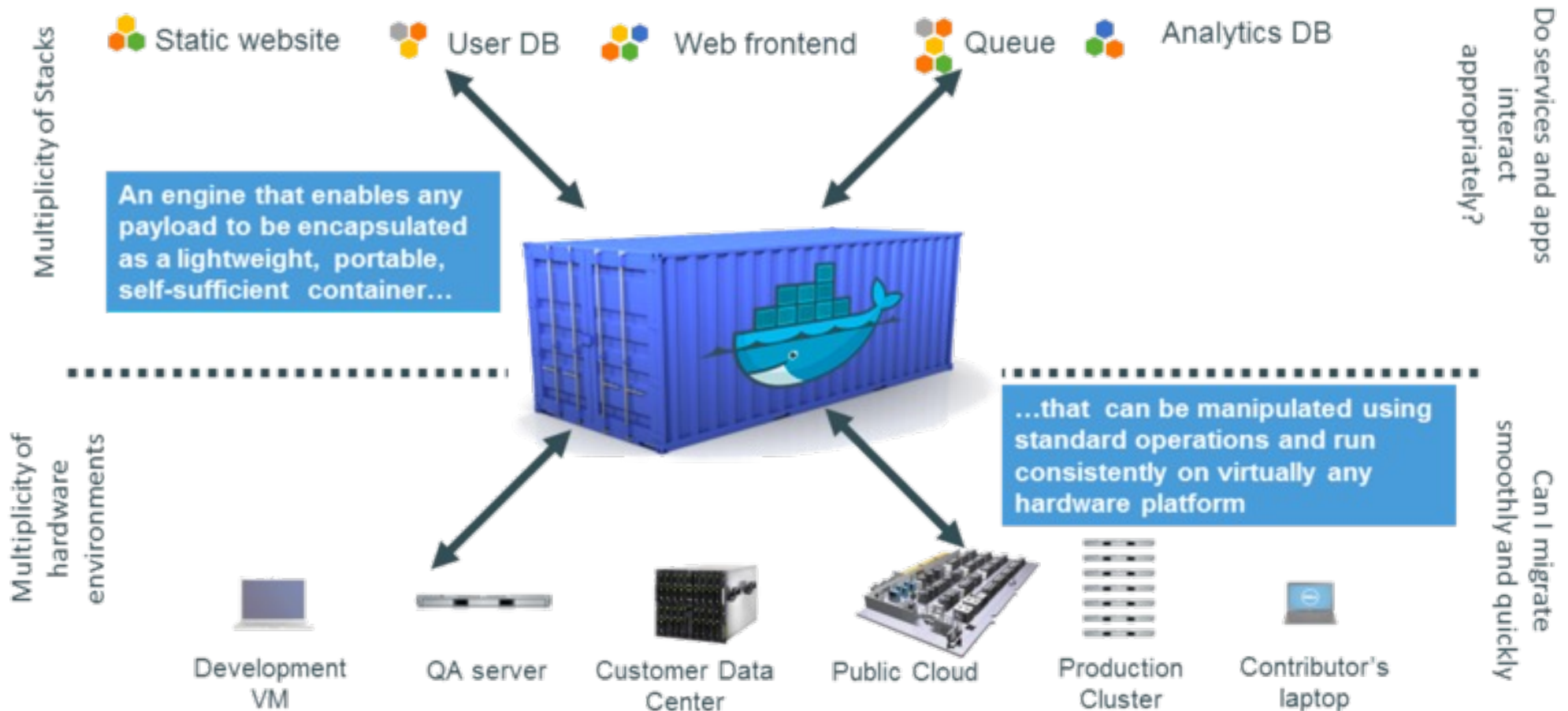
Containers for Multimodal Transportation



Staging an IT Service before Containers

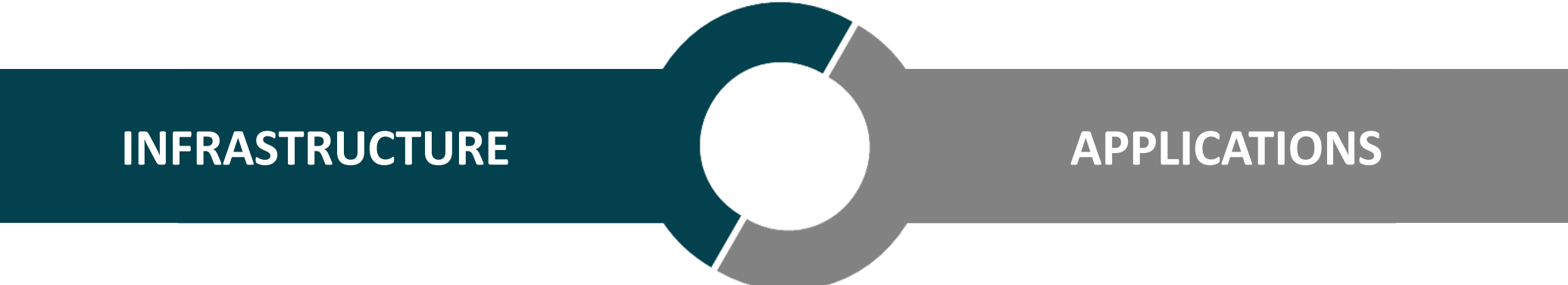


Containers in IT



Container Perspectives

Ops and Dev



INFRASTRUCTURE

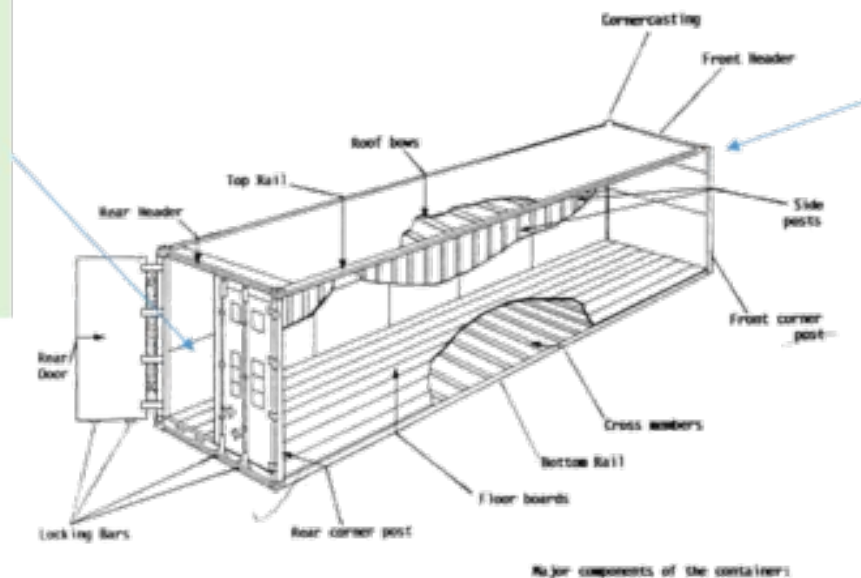
APPLICATIONS

- Application processes on a shared kernel
 - Simpler, lighter, and denser than VMs
 - Portable across different environments
- Package apps with all dependencies
 - Deploy to any environment in seconds
 - Easily accessed and shared

Separation of Duties between Dev and Ops

- **Dan the Developer**

- Worries about what's "inside" the container
 - His code
 - His Libraries
 - His Package Manager
 - His Apps
 - His Data
- All Linux servers look the same



- **Oscar the Ops Guy**

- Worries about what's "outside" the container
 - Logging
 - Remote access
 - Monitoring
 - Network config
- All containers start, stop, copy, attach, migrate, etc. the same way

Four major use cases for containers

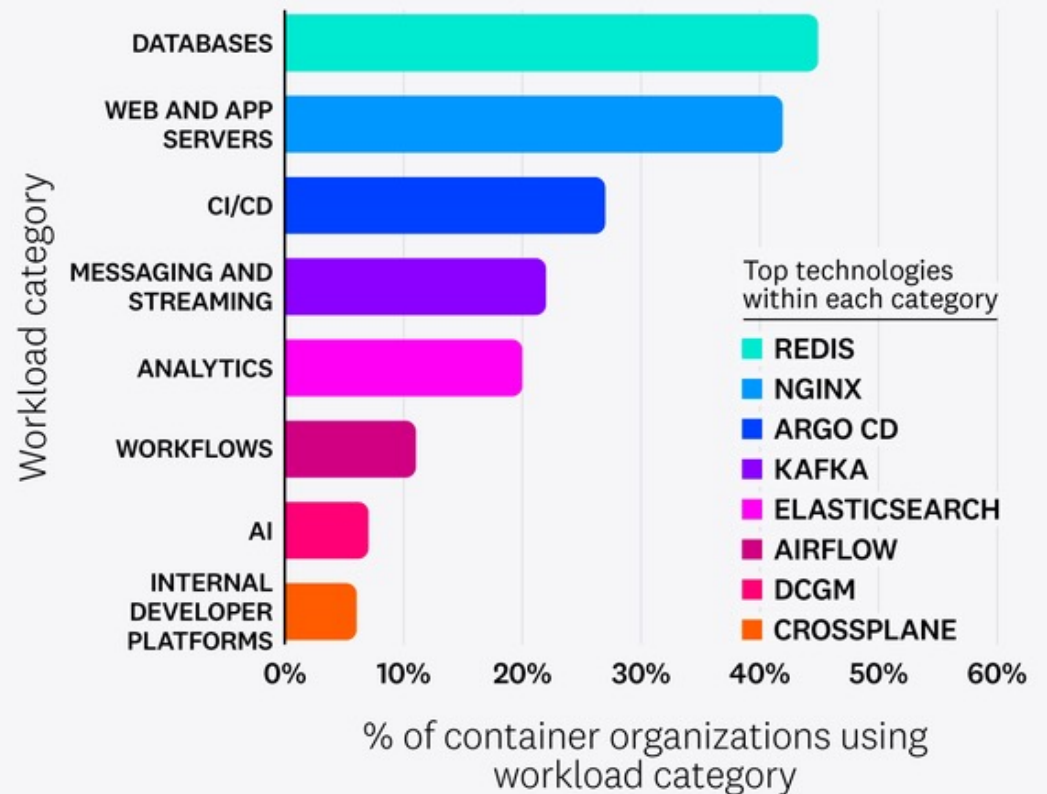
- Continuous Integration/Continuous Delivery:
 - Go from developer's laptop, through automated test, to production, and through scaling without modification
- Alternative (light-weight) form of virtualization for multi-tenant services (SaaS, Databases, Web Applications)
- Scale-out/Big Data/AI:
 - Rapidly scale same application across hundreds or thousands of servers...and scale down as rapidly
- Cross Cloud Deployment
 - Move the same application across multiple clouds (public, private, or hybrid) without modification or noticeable delay
 - Also deploy at the Edge

Container Workloads in 2025

**AI JOINS
THE LEADING
WORKLOAD
CATEGORIES FOR
CONTAINERS**

Source: Datadog

Usage of containerized workloads by category



Containers: Some History

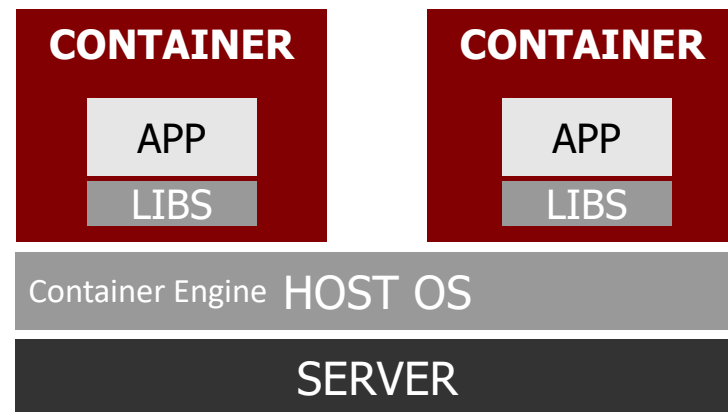
- Linux Kernel: Process Isolation Namespaces (2006)
- cgroups (2007) – control groups that isolate resource usage for collections of processes
- LXC (2008) – Linux containers as interface to low level kernel containment features
- CloudFoundry Warden (2011) – container implementation
- Docker (2013) made containers ‘safe for the masses’

Containers package applications with dependencies and isolate the runtime

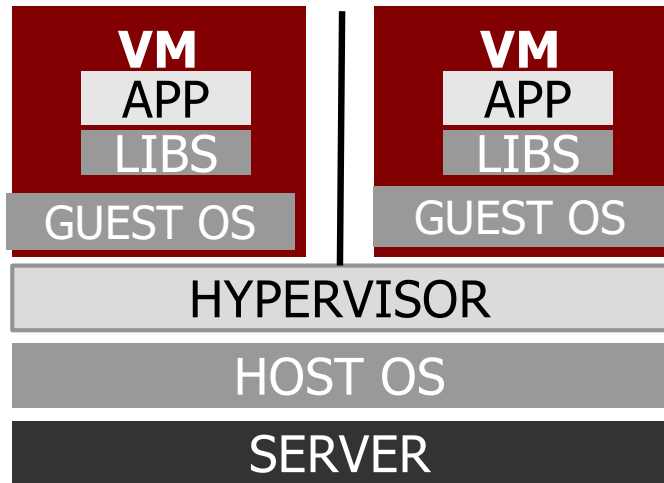
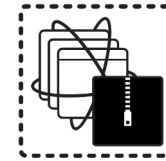
- Easy to deploy and portable across host systems
- Created from immutable, layered images
- Isolated from a host operating system.

In Linux (e.g. RHEL), this is done through:

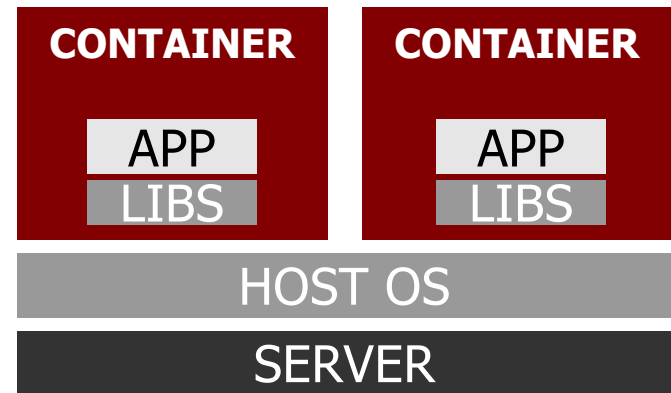
- Control Groups (cgroups)
- kernel namespaces
- SELinux (Security Enhanced Linux), sVirt, iptables
- Docker



Containers provide high density and efficiency at the expense of isolation



10 per
Server



100 per
Server

PHYSICAL
SERVER

VIRTUAL
MACHINE

CONTAINER
INSTANCE



27 HRS



12 MINS

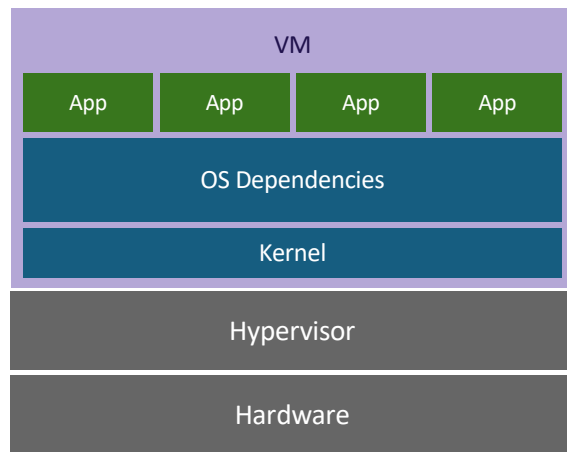


10 SECS

Time to set up and start

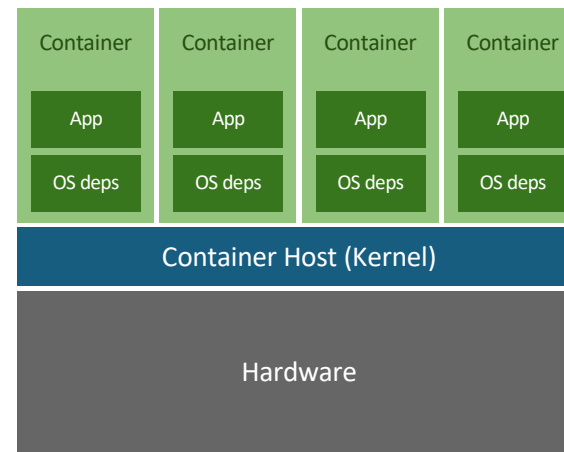
VIRTUAL MACHINES AND CONTAINERS

VIRTUAL MACHINES



VM virtualizes the hardware

CONTAINERS

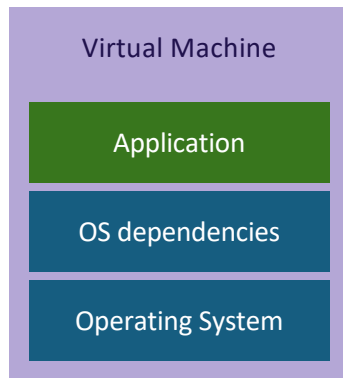


Container isolates the process

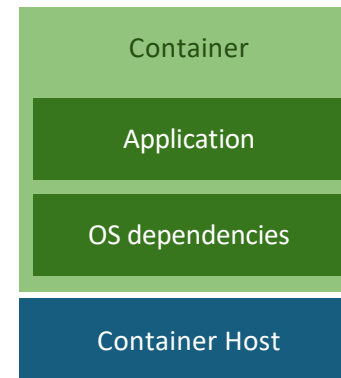
Note:

It is recommended to have one process per container

VIRTUAL MACHINES AND CONTAINERS

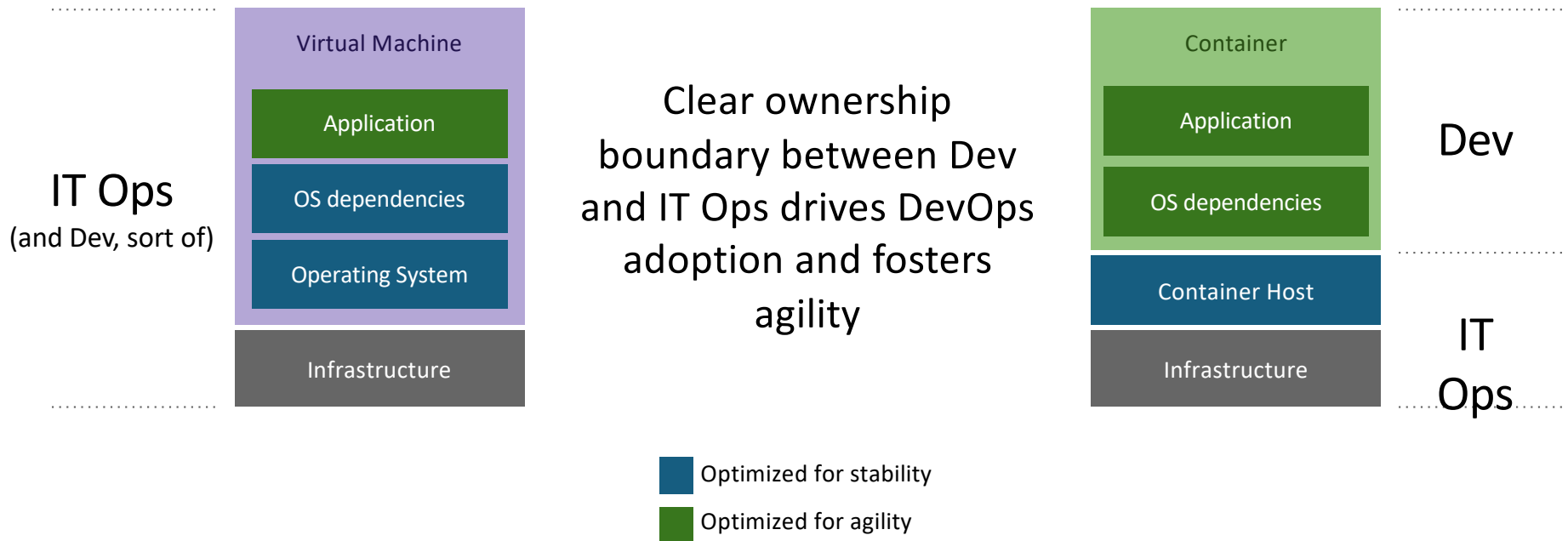


- + VM Isolation
- Complete OS
- Static Compute
- Static Memory
- High Resource Usage



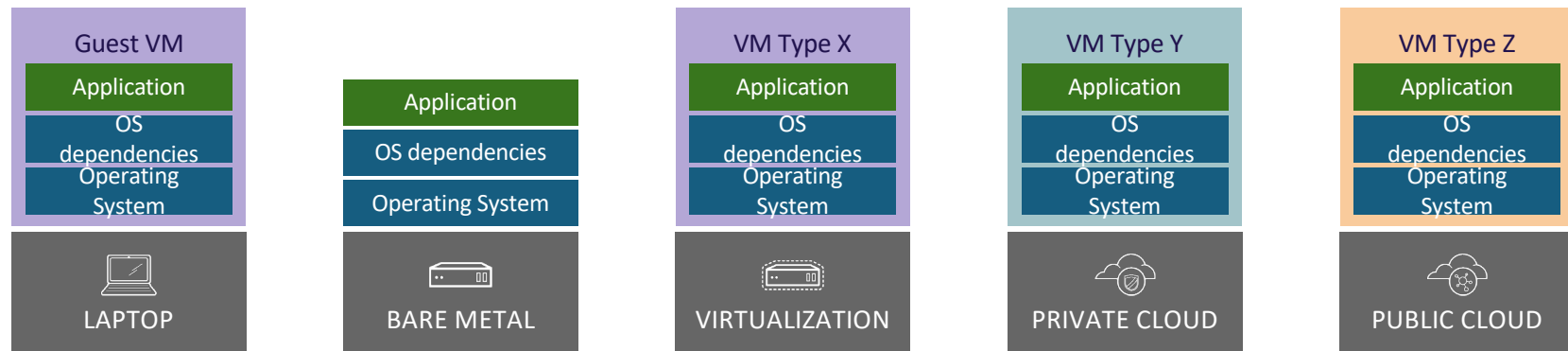
- + Container Isolation
- + Shared Kernel
- + Burstable Compute
- + Burstable Memory
- + Low Resource Usage

VIRTUAL MACHINES AND CONTAINERS



APPLICATION PORTABILITY WITH VM

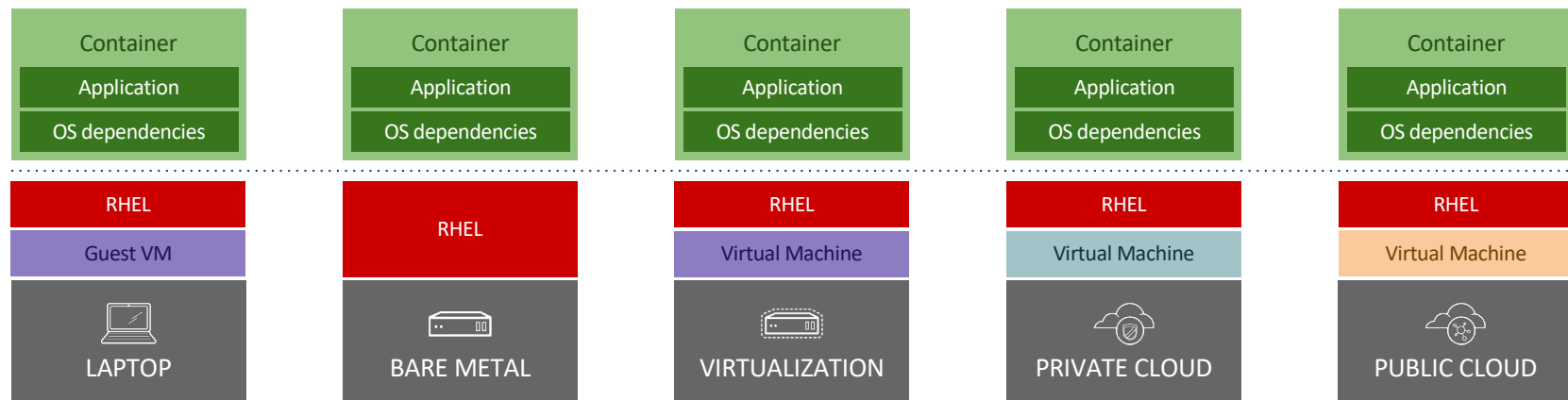
Virtual machines are **NOT** portable across hypervisor and do **NOT** provide portable packaging for applications



APPLICATION PORTABILITY WITH CONTAINERS

Standard Format: Open Container Initiative (OCI)

OCI Containers + OCI Host = Guaranteed Portability
Across Any Infrastructure





A lightweight, OCI-compliant container runtime

OCI = Open Container Initiative

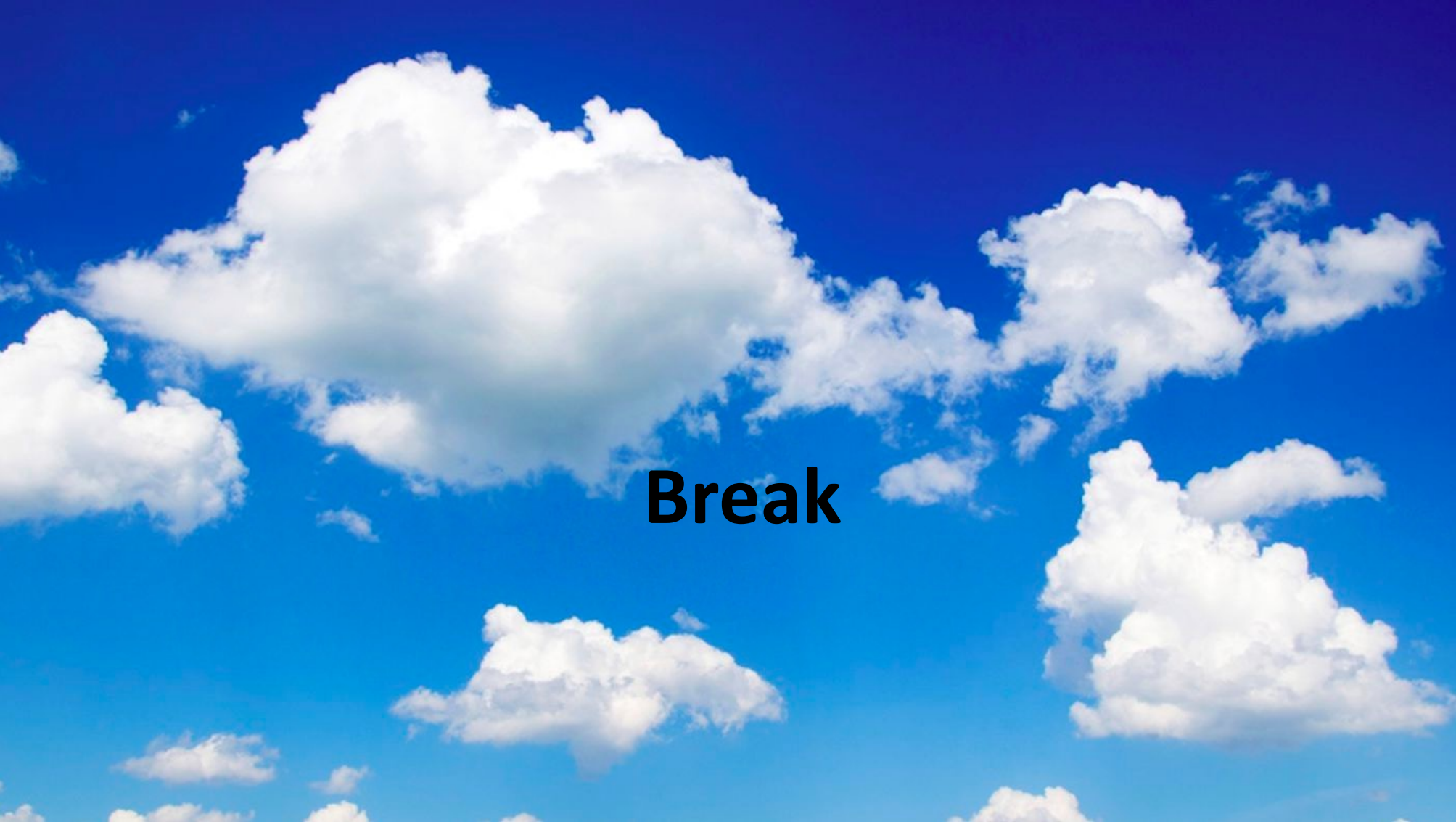
Optimized for
Kubernetes

Any OCI-compliant
container from any OCI
registry
(including docker)

Improve Security and
Performance at scale

Best Practices

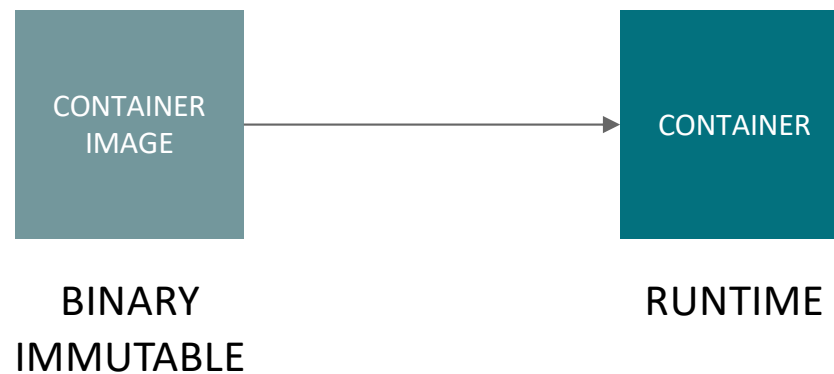
- <https://developers.redhat.com/blog/2016/02/24/10-things-to-avoid-in-docker-containers/>
- <https://cloud.google.com/solutions/best-practices-for-building-containers>
- https://docs.docker.com/develop/develop-images/dockerfile_best-practices/



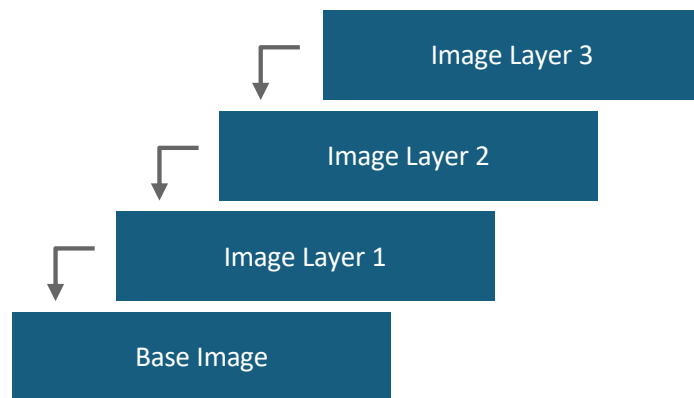
Break

Containers and Images

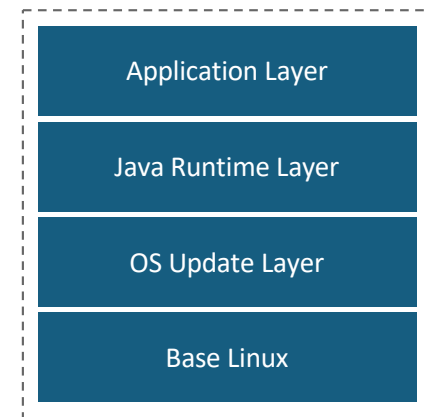
Containers are created from container images



Container Images can be layered

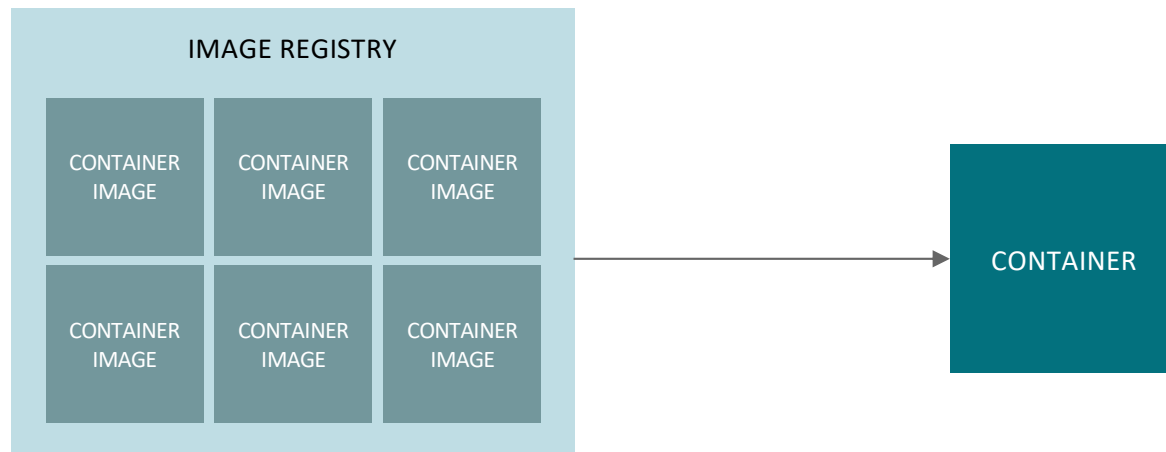


Container Image
Layers

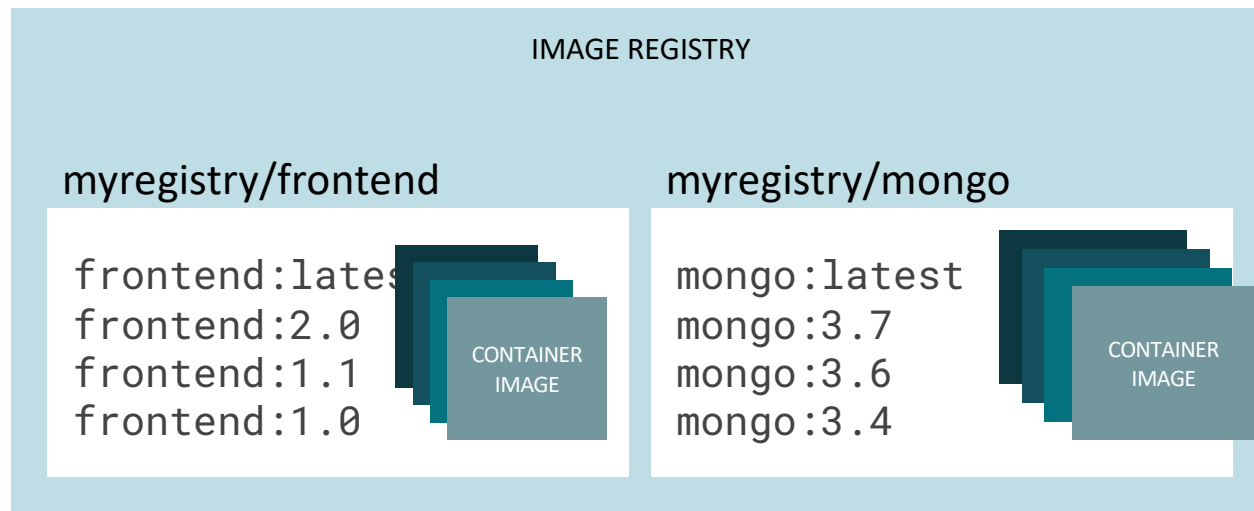


Example Container Image

Container images are stored in an image registry



An image repository contains all versions of an image in the image registry

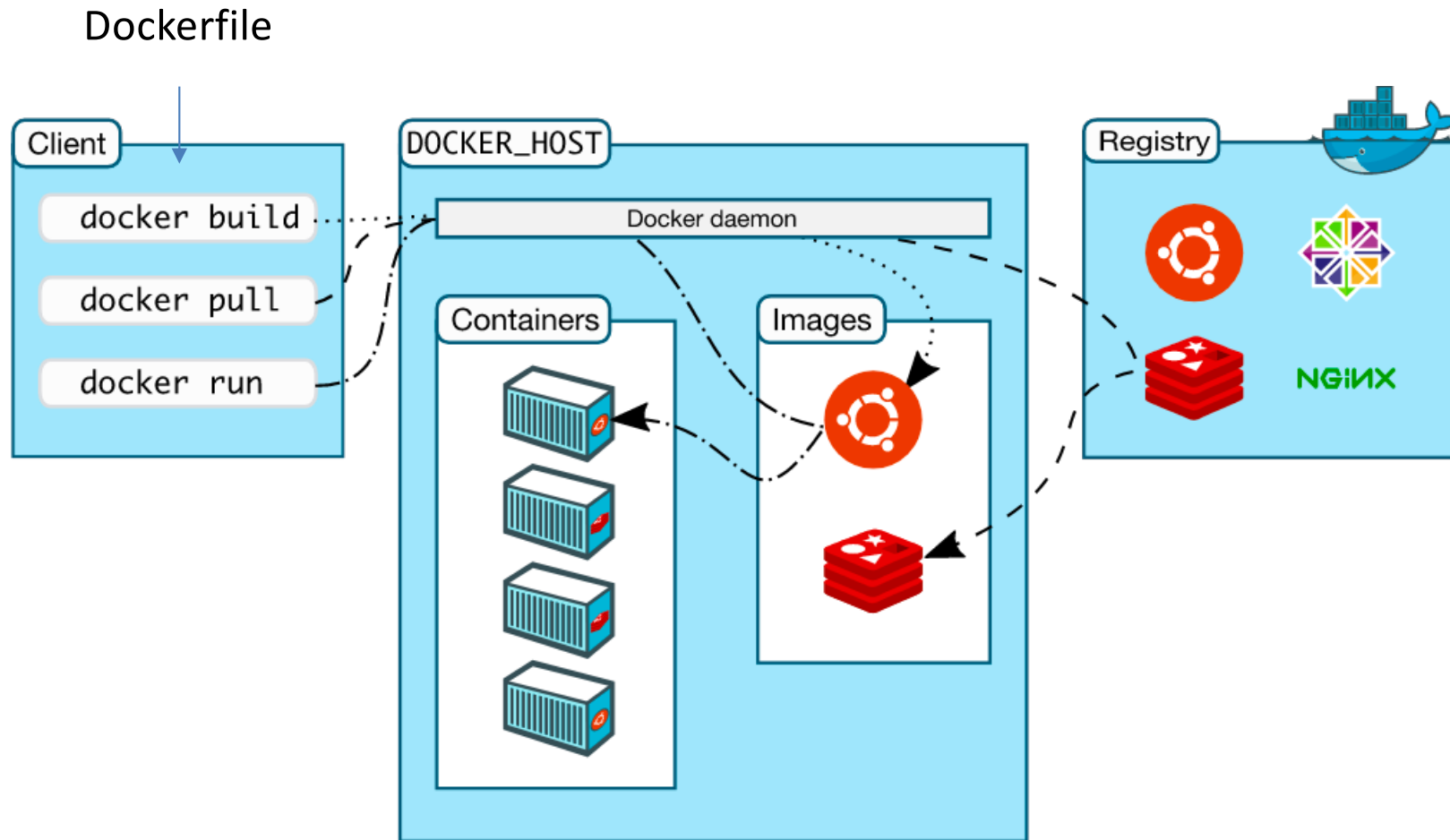


A number of vendors provide container registries.

Docker



Docker Architecture

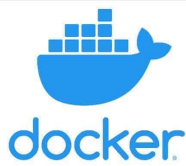


1. 'docker build' uses the Dockerfile to build an image (OCI compliant)
2. The 'docker run' command fetches the local image or else goes to the Registry and fetches the latest version.

Common Docker Commands

- **build** *uses a Dockerfile to build an image*
`docker build -t myapp:v1 .`
- **run** *runs a docker container*
`docker run myapp:v1`
- **tag** *assigns name and repository*
`docker tag myapp:v1 yourapp:v1 (new repo, same image)`
- **images** *lists images with tags and repositories*
- **push/pull** *stores/retrieves from tagged repo*
`docker push repositoryname/registryname:tag`
`docker pull docker.io./ubuntu:18.04`

The Docker CLI is compatible with podman.



Commands Cheat Sheet

Container Lifecycle

docker create [IMAGE]	create a container without starting it
docker rename [CONTAINER_NAME] [NEW_CONTAINER_NAME]	rename a container
docker run [IMAGE]	create and start a container
docker run --rm [IMAGE]	remove a container after it stops
docker run -td [IMAGE]	start a container and keep it running
docker run -it [IMAGE]	create, start the container, and run a command in it
docker run -it-rm [IMAGE]	create, start the container, and run a command in it; after executing, the container is removed
docker rm [CONTAINER]	delete a container if it isn't running
docker update [CONTAINER]	update the configuration of a container

Networking

docker network ls	list networks
docker network rm [NETWORK]	remove one or more networks
docker network inspect [NETWORK]	show information on one or more networks
docker network connect [NETWORK] [CONTAINER]	connect a container to a network
docker network disconnect [NETWORK] [CONTAINER]	disconnect a container from a network

Image Lifecycle

docker build [URL]	create an image from a Dockerfile
docker build -t [URL]	build an image from a Dockerfile and tags it
docker pull [IMAGE]	pull an image from a registry
docker push [IMAGE]	push an image to a registry
docker import [URL/FILE]	create an image from a tarball
docker commit [CONTAINER] [NEW_IMAGE_NAME]	create an image from a container
docker rmi [IMAGE]	remove an image
docker load [TAR_FILE/STDIN_FILE]	load an image from a tar archive as stdin
docker save [IMAGE] > [TAR_FILE]	save an image to a tar archive stream to stdout with all parent layers, tags, and versions

Start & Stop

docker start [CONTAINER]	start a container
docker stop [CONTAINER]	stop a running container
docker restart [CONTAINER]	stop a running container and start it up again
docker pause [CONTAINER]	pause processes in a running container
docker unpause [CONTAINER]	unpause processes in a container
docker wait [CONTAINER]	block a container until other containers stop
docker kill [CONTAINER]	kill a container by sending SIGKILL to a running container
docker attach [CONTAINER]	attach local standard input, output, and error streams to a running container

Information

docker ps	list running containers
docker ps -a	list running and stopped containers
docker logs [CONTAINER]	list the logs from a running container
docker inspect [OBJECT_NAME/ID]	list low-level information on an object
docker events [CONTAINER]	list real time events from a container
docker port [CONTAINER]	show port (or specific) mapping from a container
docker top [CONTAINER]	show running processes in a container
docker stats [CONTAINER]	show live resource usage statistics of containers
docker diff [CONTAINER]	show changes to files (or directories) on a filesystem
docker images ls	show all locally stored images
docker history [IMAGE]	show history of an image

See also docs.docker.com

Writing a Dockerfile

https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

- Docker builds images automatically by reading the instructions from a Dockerfile -- a text file that contains all commands, in order, needed to build a given image. You can find the Dockerfile instructions at [Dockerfile reference](#).
- A Docker image consists of **read-only layers**, each of which represents a **Dockerfile instruction**. The layers are stacked and each one is a delta of the changes from the previous layer. Example:

```
FROM ubuntu:22.04
COPY . /app
RUN make /app
CMD python /app/app.py
```
- Each instruction creates one layer:
 - FROM creates a layer from the ubuntu:18.04 Docker image.
 - COPY adds files from your Docker client's current directory.
 - RUN builds your application with make.
 - CMD specifies what command to run within the container.
- When you run an image and generate a container, you add a new *writable layer* (the “container layer”) on top of the underlying layers.
- All changes made to the running container, such as writing new files, modifying existing files, and deleting files, are written to this thin writable container layer.

Docker and AWS

Install Docker on an EC2 Instance

- Amazon Documentation
 - <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/docker-basics.html>
- Install docker in an AMI (or in Linux on your laptop). **It is already installed in CloudShell.**
 - `$ sudo apt install docker.io` (for Ubuntu)
 - `$ sudo amazon-linux-extras install docker` (for Amazon Linux)
- Note: You could also install on a Linux environment on your desktop
- Start docker service and add ubuntu (or ec2-user) to the docker group (**otherwise you may have to use sudo throughout**)
 - `$ sudo service docker start` (or `sudo systemctl start docker`)
 - `$ sudo usermod -aG docker ubuntu` (or `ec2-user`)
 - Log out and in again to reevaluate group membership
 - `$ docker info`

Docker and AWS

Create a Docker Image

- Create a **Dockerfile**

- A Dockerfile is a **manifest** that describes the base image and what you want to install and run
- Example Dockerfile for a simple web server using a Ubuntu image

```
FROM ubuntu:24.04
```

```
# Install dependencies
```

```
RUN apt-get update && \  
    apt-get -y install apache2
```

```
# Install apache and write hello world message
```

```
RUN echo 'Hello World!' > /var/www/html/index.html
```

```
# Configure apache
```

```
RUN echo './etc/apache2/envvars' > /root/run_apache.sh && \  
    echo 'mkdir -p /var/run/apache2' >> /root/run_apache.sh && \  
    echo 'mkdir -p /var/lock/apache2' >> /root/run_apache.sh && \  
    echo '/usr/sbin/apache2 -D FOREGROUND' >> /root/run_apache.sh && \  
    chmod 755 /root/run_apache.sh
```

```
EXPOSE 80
```

```
CMD /root/run_apache.sh
```

Docker and AWS

Create a Docker Image

- **Build** an image with **tag** 'hello-world' based on the Dockerfile
`$ docker build -t hello-world .` (the dot is important)
- Check that the image was created
`$ docker images --filter reference=hello-world`
- **Run** the image, mapping the host's port 80 to the container's port 80 (-p hostport:containerport)
`$ docker run -t -i -p 80:80 hello-world`
Note: Stop other services listening on port 80! Enable port 80.
- Point your browser to the IP address of your EC2 image
- Optional: Tag your image and push to a registry
 - Docker Hub is the registry that 'comes with' Docker
 - Sign up to create repositories
 - `registry.hub.docker.com/[username]/[repositoryname]`

Docker and AWS

Push your image to the Amazon Elastic Container Registry

- Create an ECR repository (returns a json document)

```
$ aws ecr create-repository --repository-name hello-repository --region region
```

- **Tag** the image with the repositoryUri from the previous step

```
$ docker tag hello-world
```

```
aws_account_id.dkr.ecr.region.amazonaws.com/hello-repository
```

- Authenticate to ECR:

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS  
--password-stdin aws_account_id.dkr.ecr.us-east-1.amazonaws.com
```

Note: You may have to refresh your credentials for each lab session.

Docker and AWS

Push your image to Amazon Elastic Container Registry

- **Push** the image to ECR

```
$ docker push aws_account_id.dkr.ecr.region.amazonaws.com/hello-repository
```

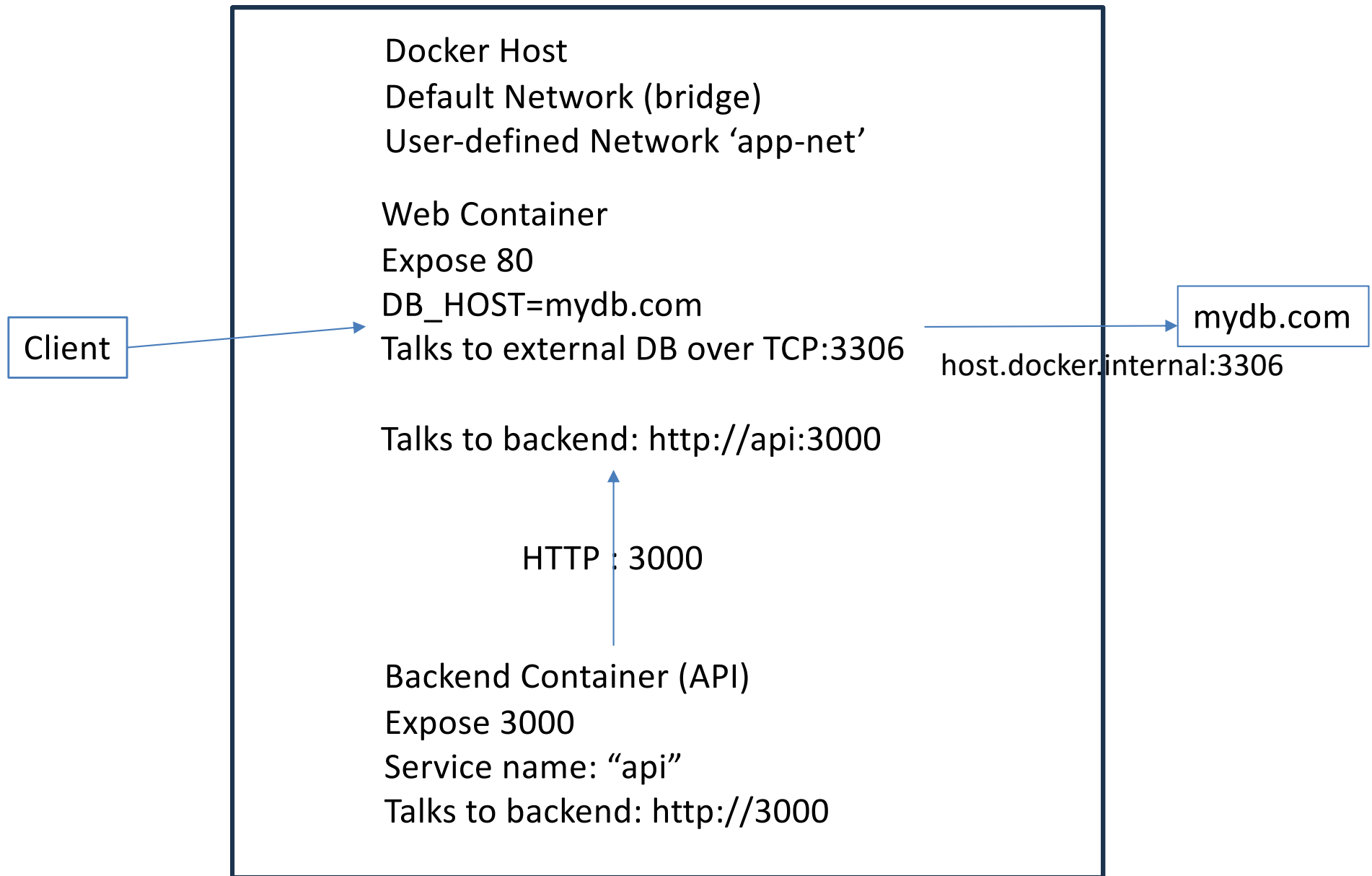
- **Verify** the push worked

```
$ aws ecr describe-images --repository-name hello-repository
```

- Look at the repository from the ECR console

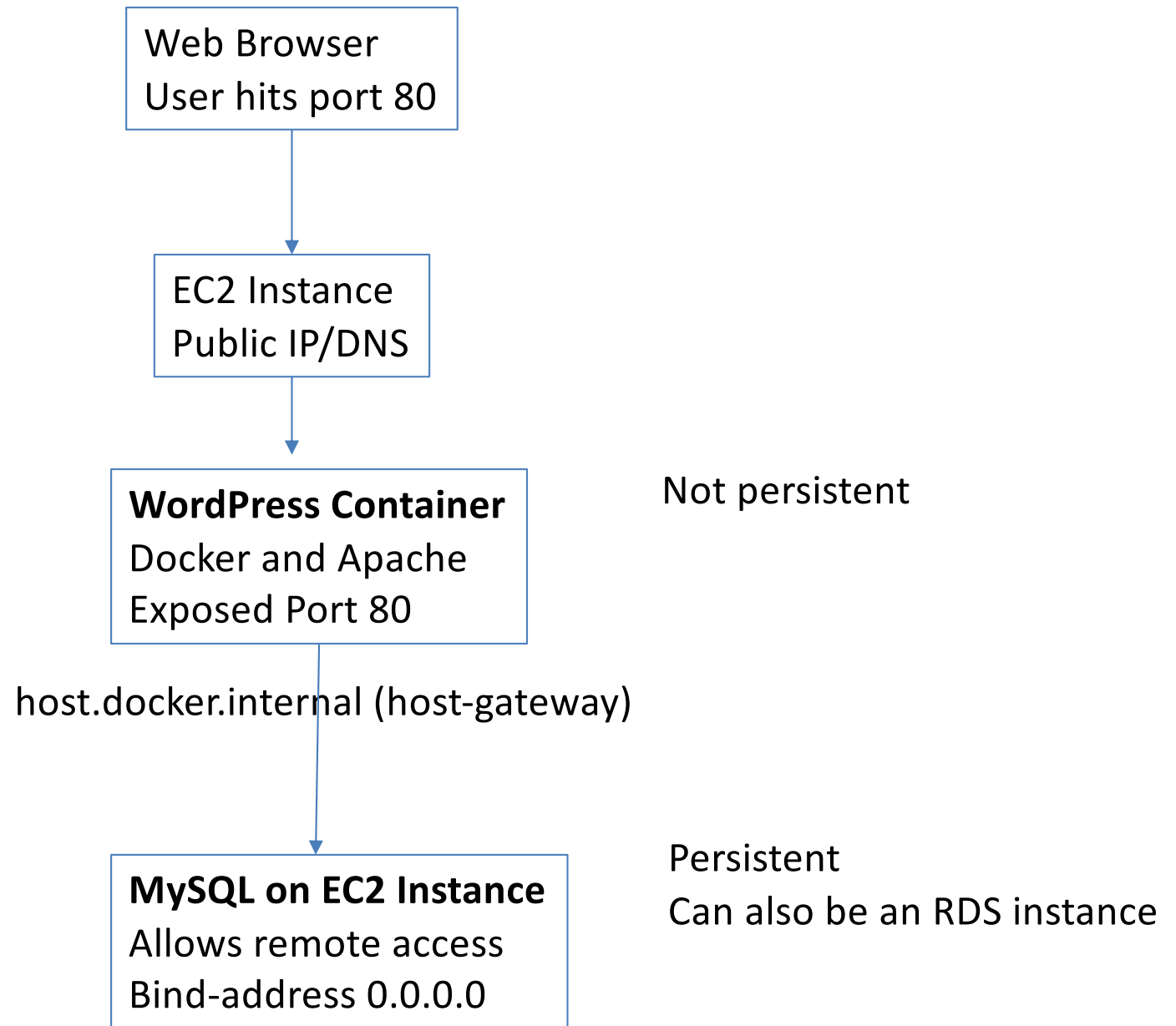
Docker Networking

- The docker host has a default network (bridge)
 - NATed through host
 - Allows communication with outside world
 - Containers only communicate via IP-addresses, cannot resolve names
 - Can communicate with an outside service
- You can create user-defined networks for communication between containers
docker network create app-net
- A user-defined network provides:
 - **container name → IP resolution**
 - automatic service discovery
 - internal routing
 - isolation from other containers
- You can run a container in user-defined network, e.g. a database container
`docker run --name db --network app-net -e MYSQL_ROOT_PASSWORD=rootpass \`
`-e MYSQL_DATABASE=wordpress -d mysql:8.0`
- Other containers in the same network can connect to the container using its name



Docker and WordPress

- https://hub.docker.com/_/wordpress
- Various tags available, e.g. [6.8.3-php8.3-apache](#)
- Get an image, e.g.
`docker pull wordpress:6.8.3-php8.3-apache`
- Tag the image
- Push to ECR
- Add '**wp-user**'@'%' identified by 'AWStest2025!' to your mysql database and grant privileges to wordpressdb. This allows access from outside 'localhost'
- Set bind-address = 0.0.0.0 in /etc/mysql/mysql.conf.d/mysqld.cnf to **make mysql listen to more than just loopback, restart mysql**
- In the run command, point host.docker.internal to the EC2 instance and tell WP to connect to mysql on the host
- Run locally (after stopping process using port 80),
`docker run -it -p 80:80 \`
`--add-host=host.docker.internal:host-gateway \`
`-e WORDPRESS_DB_HOST=host.docker.internal:3306 \`
`-e WORDPRESS_DB_USER=wp-user \`
`-e WORDPRESS_DB_PASSWORD='AWStest2025!' \`
`-e WORDPRESS_DB_NAME=wordpressdb \`
`wordpress`



Deploying Python Lambda Functions with Container Images

- If you use an AWS base image, you only need to copy your function to the container and install dependencies
- **Sample app.py with simple handler (make sure it has permission 644)**

```
import sys
def handler(event, context):
    return 'Hello from AWS Lambda using Python' + sys.version + ' !'
```

- **Sample Dockerfile**

```
FROM public.ecr.aws/lambda/python:3.11

COPY app.py ./
RUN chmod 644 /var/task/app.py
CMD ["app.handler"]
```

Lambda MicroVMs

The screenshot shows the AWS Lambda MicroVM Images console. The left sidebar contains navigation links for Lambda, MicroVM Images, Functions, Function resources, MicroVMs, Network connectors, Documentation, Serverless workshops, and Related AWS resources. The main content area is titled 'MicroVM Images' and includes a 'How it works' section with three steps: 1. Create MicroVM Image, 2. Run MicroVM, and 3. Use MicroVM. Below this is a table for 'MicroVM Images (0)' with columns for Name, State, Active, Failed, Created, and Image ARN. The table is currently empty, displaying 'No data to display.' The bottom of the page features a footer with links to CloudShell, Agent Toolkit for AWS, Feedback, Console Mobile App, and copyright information for Amazon Web Services.

MicroVM Images Info

▼ **How it works** [Get MicroVM agent instructions](#)

Lambda MicroVMs are compute environments designed for rapid, sandboxed code execution. Use MicroVMs to build AI coding assistants, interactive data analytics platforms, and secure code execution environments. MicroVMs are designed for use cases requiring a sandbox per end-user. [Learn more](#)

Step 1. Create MicroVM Image

A MicroVM image is a snapshot of your running application code and configuration. Build once, then run MicroVMs from the snapshot point, no container or application bootstrapping necessary.

[Create new MicroVM image](#)

Step 2. Run MicroVM

To run your application, run a MicroVM from the image. Serve traffic within seconds.

[Run MicroVM](#)

Step 3. Use MicroVM

Your application is running. Serve traffic, suspend-resume execution, connect to VPC resources, or interact with your MicroVM from the console.

MicroVM Images (0) Info

Last fetched 30 seconds ago [Refresh](#) [Actions](#) [Create](#)

Name	State	Active ...	Failed ...	Created	Image ARN
No data to display.					

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

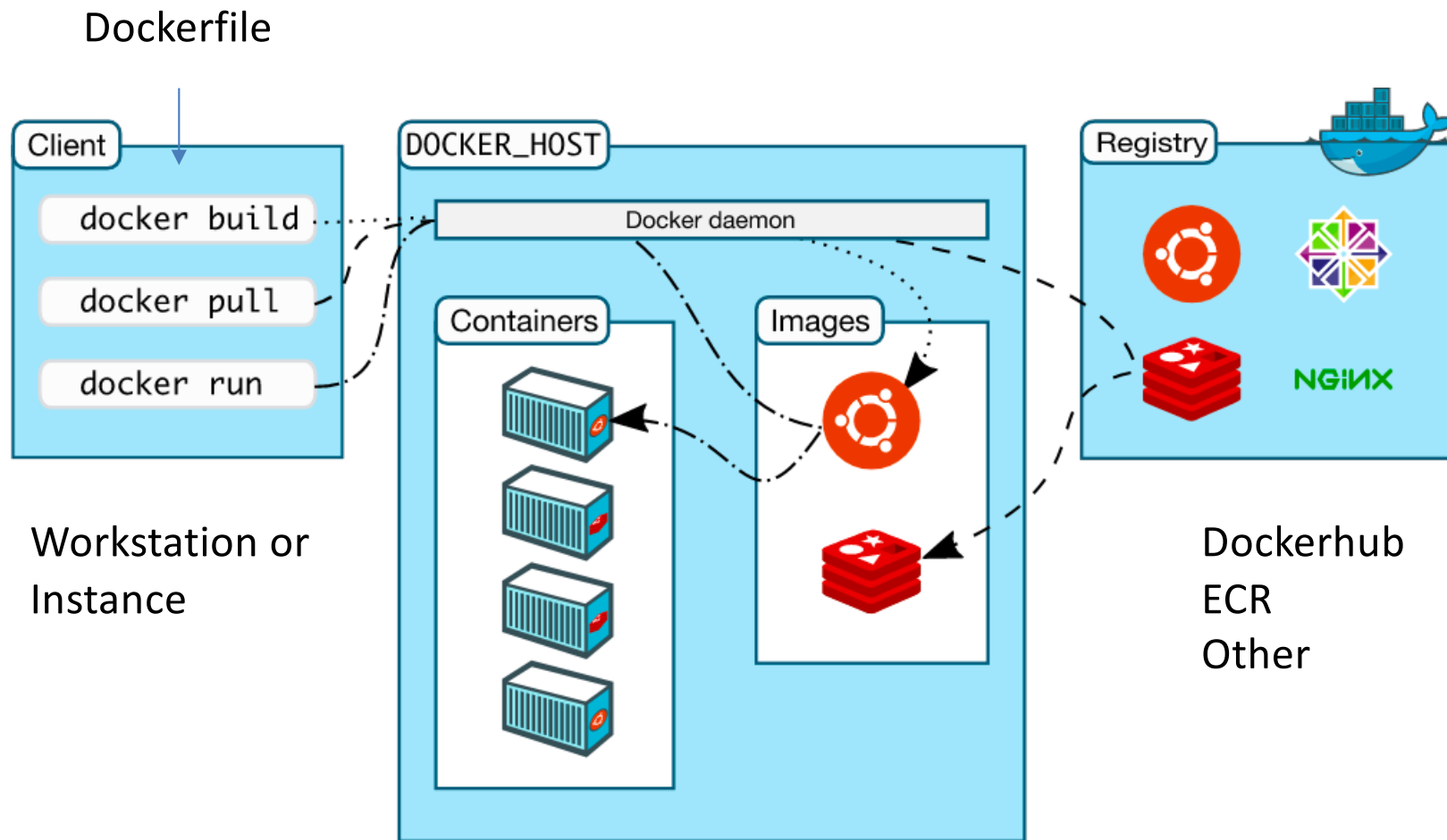
The code configuration needs to be a zip file including a Dockerfile in its root directory

Container Management

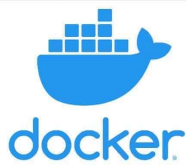
ECS and EKS



Recap: Docker



1. 'docker build' uses the Dockerfile to build an image
2. The 'docker run' command fetches the local image or else goes to the Registry and fetches the latest version.



Commands Cheat Sheet

Container Lifecycle

docker create [IMAGE]	create a container without starting it
docker rename [CONTAINER_NAME] [NEW_CONTAINER_NAME]	rename a container
docker run [IMAGE]	create and start a container
docker run --rm [IMAGE]	remove a container after it stops
docker run -td [IMAGE]	start a container and keep it running
docker run -it [IMAGE]	create, start the container, and run a command in it
docker run -it-rm [IMAGE]	create, start the container, and run a command in it; after executing, the container is removed
docker rm [CONTAINER]	delete a container if it isn't running
docker update [CONTAINER]	update the configuration of a container

Networking

docker network ls	list networks
docker network rm [NETWORK]	remove one or more networks
docker network inspect [NETWORK]	show information on one or more networks
docker network connect [NETWORK] [CONTAINER]	connect a container to a network
docker network disconnect [NETWORK] [CONTAINER]	disconnect a container from a network

Image Lifecycle

docker build [URL]	create an image from a Dockerfile
docker build -t [URL]	build an image from a Dockerfile and tags it
docker pull [IMAGE]	pull an image from a registry
docker push [IMAGE]	push an image to a registry
docker import [URL/FILE]	create an image from a tarball
docker commit [CONTAINER] [NEW_IMAGE_NAME]	create an image from a container
docker rmi [IMAGE]	remove an image
docker load [TAR_FILE/STDIN_FILE]	load an image from a tar archive as stdin
docker save [IMAGE] > [TAR_FILE]	save an image to a tar archive stream to stdout with all parent layers, tags, and versions

Start & Stop

docker start [CONTAINER]	start a container
docker stop [CONTAINER]	stop a running container
docker restart [CONTAINER]	stop a running container and start it up again
docker pause [CONTAINER]	pause processes in a running container
docker unpause [CONTAINER]	unpause processes in a container
docker wait [CONTAINER]	block a container until other containers stop
docker kill [CONTAINER]	kill a container by sending SIGKILL to a running container
docker attach [CONTAINER]	attach local standard input, output, and error streams to a running container

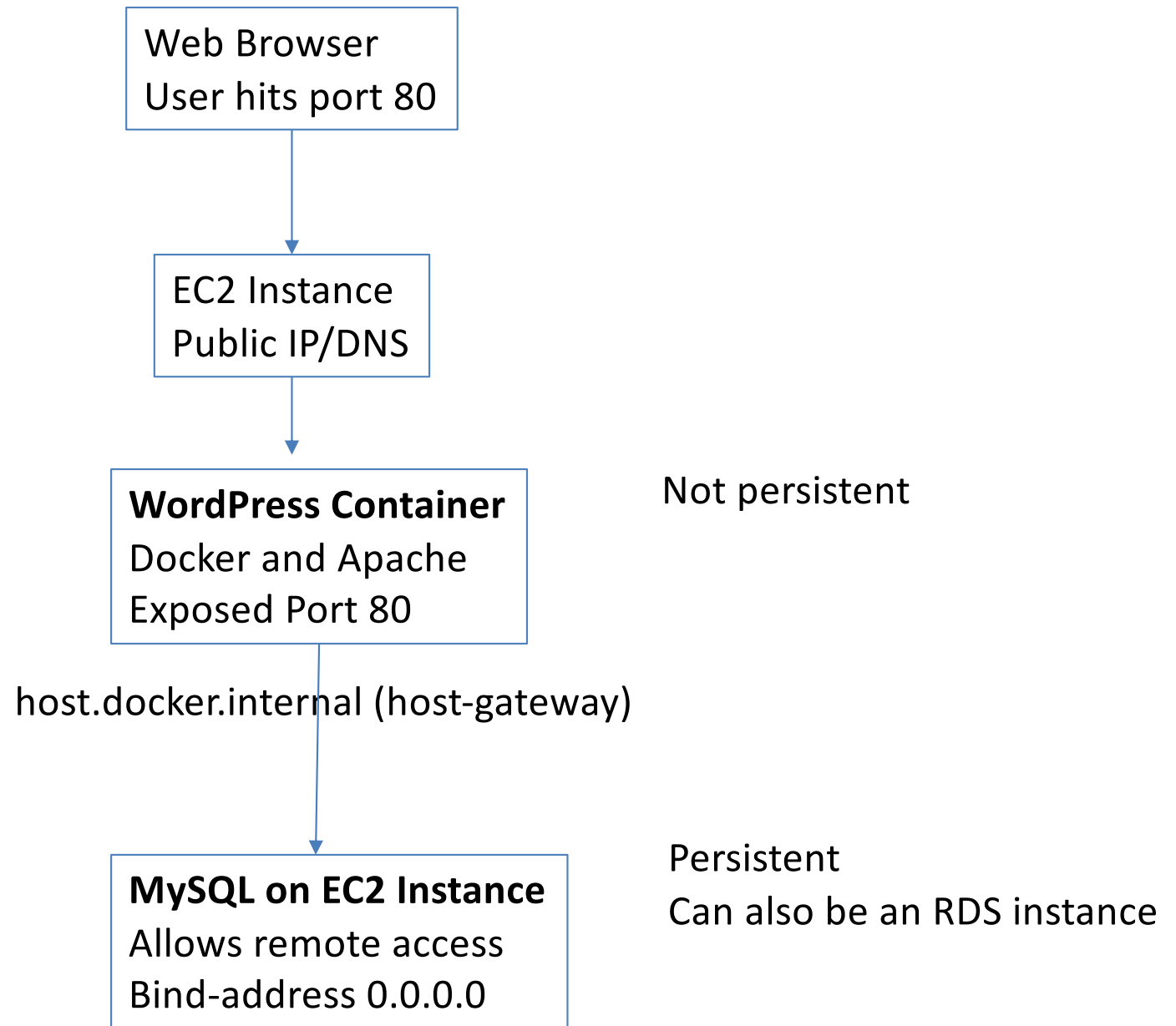
Information

docker ps	list running containers
docker ps -a	list running and stopped containers
docker logs [CONTAINER]	list the logs from a running container
docker inspect [OBJECT_NAME/ID]	list low-level information on an object
docker events [CONTAINER]	list real time events from a container
docker port [CONTAINER]	show port (or specific) mapping from a container
docker top [CONTAINER]	show running processes in a container
docker stats [CONTAINER]	show live resource usage statistics of containers
docker diff [CONTAINER]	show changes to files (or directories) on a filesystem
docker images ls	show all locally stored images
docker history [IMAGE]	show history of an image

See also docs.docker.com

Docker and Databases (WordPress)

- https://hub.docker.com/_/wordpress
- Various tags available, e.g. [6.8.3-php8.3-apache](#)
- Get an image, e.g.
`docker pull wordpress:6.8.3-php8.3-apache`
- Tag the image
- Push to ECR
- Add '**wp-user**'@'%' identified by 'AWStest2025!' to your mysql database and grant privileges to wordpressdb. This allows access from outside 'localhost'
- Set bind-address = 0.0.0.0 in /etc/mysql/mysql.conf.d/mysqld.cnf to **make mysql listen to more than just loopback, restart mysql**
- In the run command, point host.docker.internal to the EC2 instance and tell WP to connect to mysql on the host
- Run locally (after stopping process using port 80),
`docker run -it -p 80:80 \`
`--add-host=host.docker.internal:host-gateway \`
`-e WORDPRESS_DB_HOST=host.docker.internal:3306 \`
`-e WORDPRESS_DB_USER=wp-user \`
`-e WORDPRESS_DB_PASSWORD='AWStest2025!' \`
`-e WORDPRESS_DB_NAME=wordpressdb \`
`wordpress`



Elastic Container Registry (ECR)



Source: <https://console.aws.amazon.com/ecr/home?region=us-east-1>

Container Management

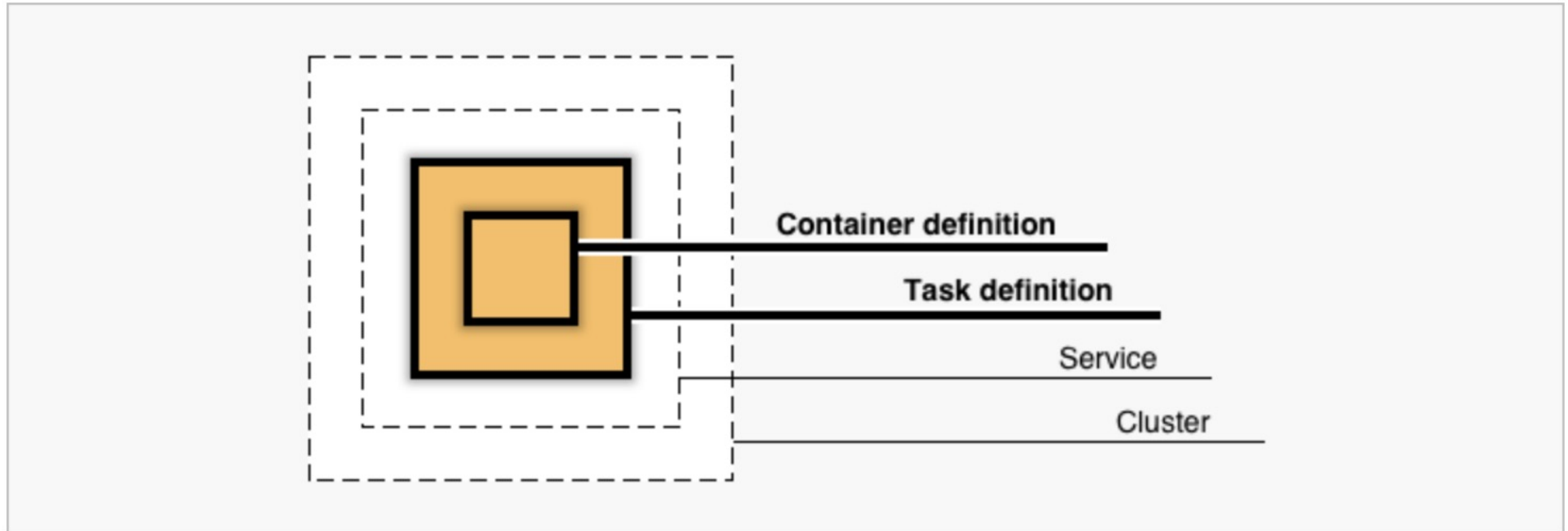
- Container scheduling & placement
- High Availability / Health Checks
- Load Balancing & Scaling
- Service Discovery
- Attaching various types (network, local) of storage to containers in a cluster.

Similar to VM management, but with potentially higher volume and speed

Container Management in AWS

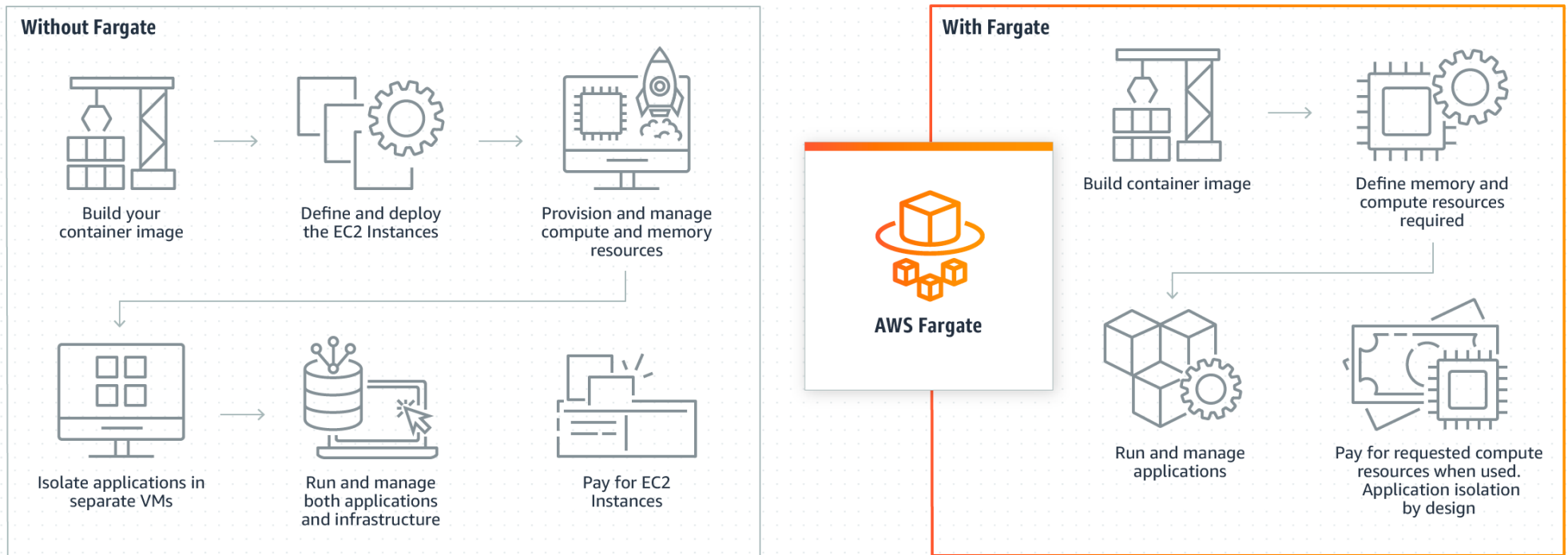
- Elastic Container Service (ECS) – AWS proprietary
 - Deploy, manage and run Docker containers
 - Integrated with Load Balancer, EBS, IAM,...
 - Launches into EC2 or **Fargate** (serverless containers, default)
 - <https://aws.amazon.com/blogs/containers/the-role-of-aws-fargate-in-the-container-world/>
 - For an introduction see: <https://ecsworkshop.com/>
- Elastic Container Registry (ERS)
 - Fully managed container registry
 - Integrates with ECS and Docker CLI
- Elastic Kubernetes Service (EKS) – based on open source
 - For an introduction see: <https://eksworkshop.com/>
- OpenShift on AWS – from Red Hat

Elastic Container Services Concepts



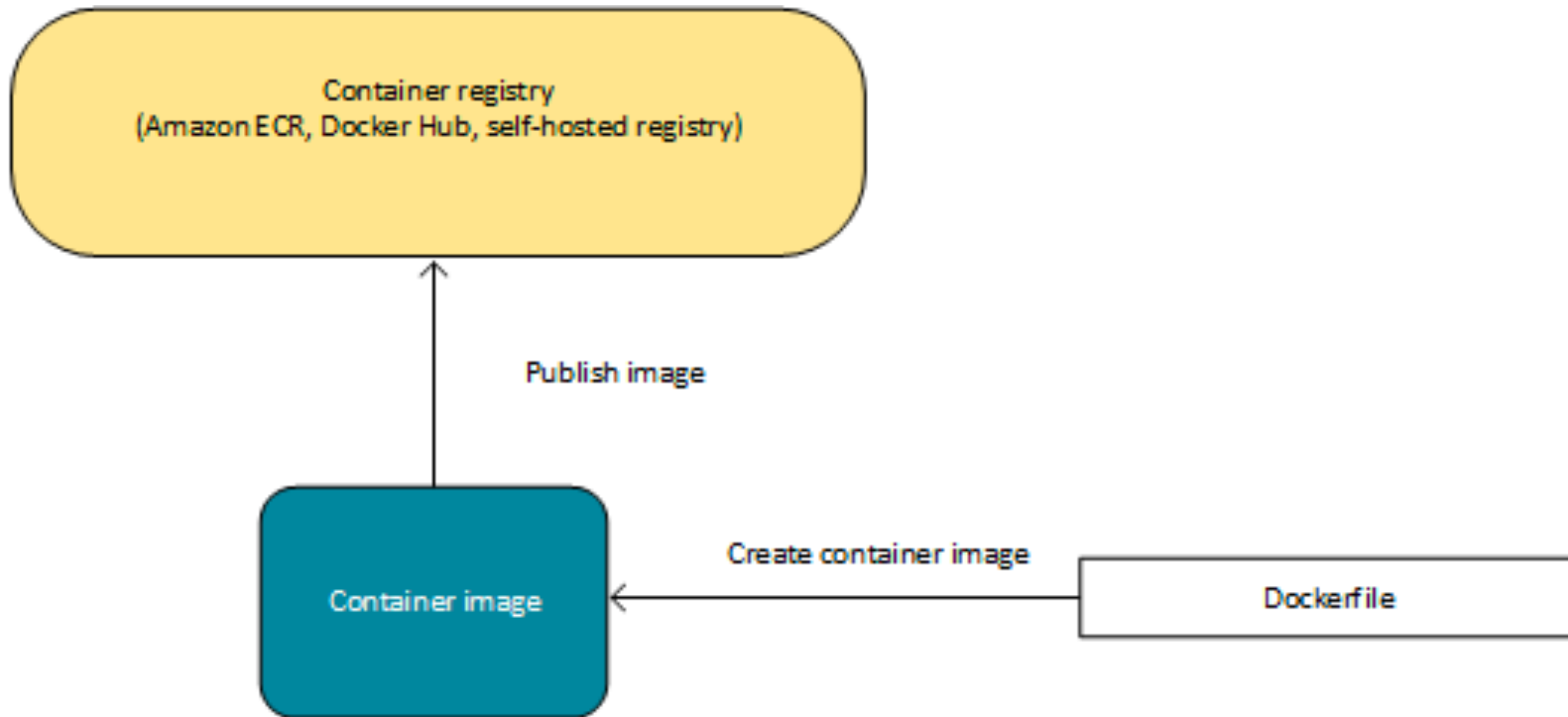
- The default setup is serverless (Fargate)
- A **task** is a collection of containers
- A **service** is an autoscaling group of tasks, can be load-balanced
- A **cluster** can run multiple services
 - It has infrastructure (capacity provider) options

Elastic Container Service with EC2 and Fargate (default)



Elastic Container Service

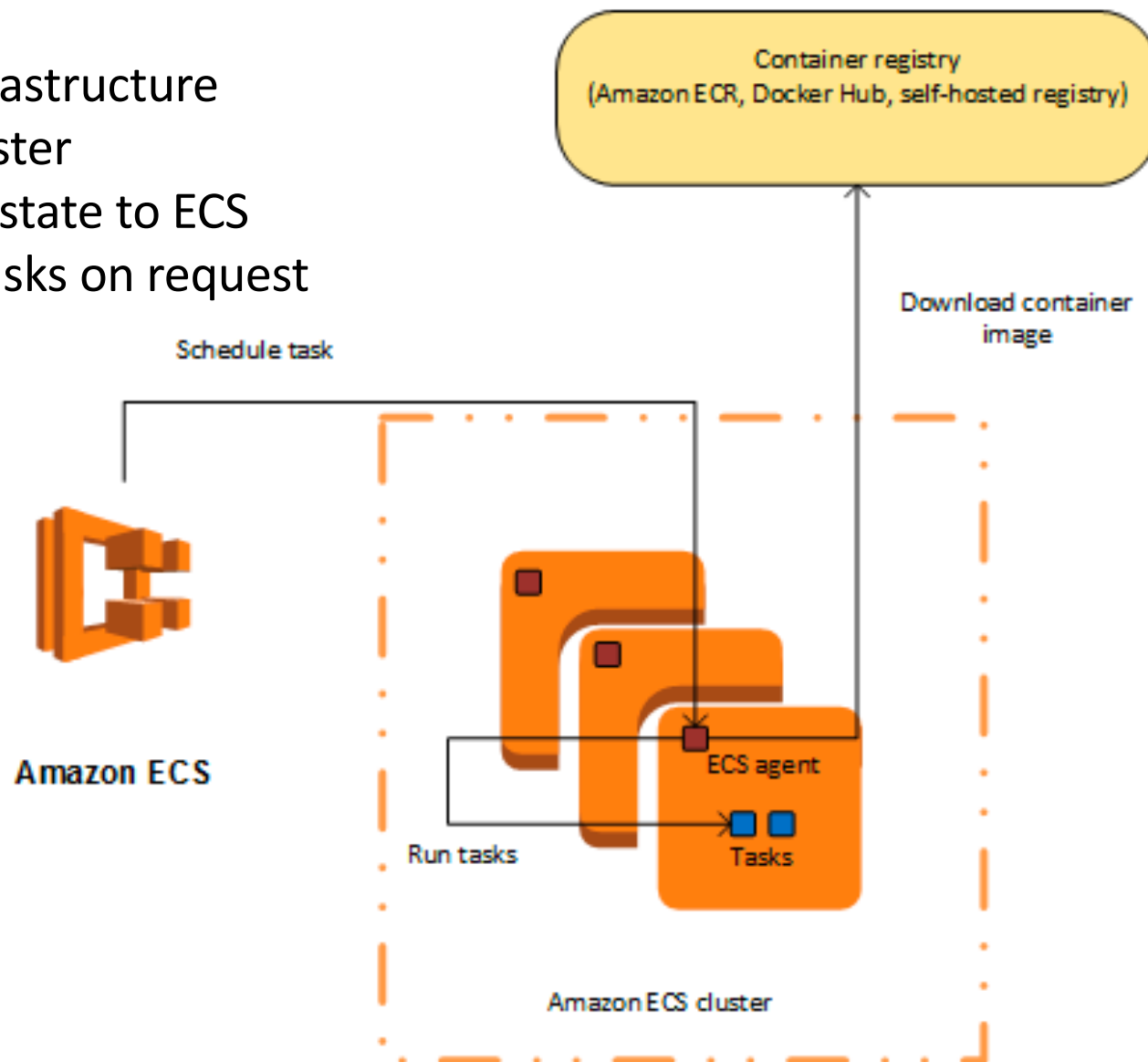
Containers and Images



Elastic Container Service

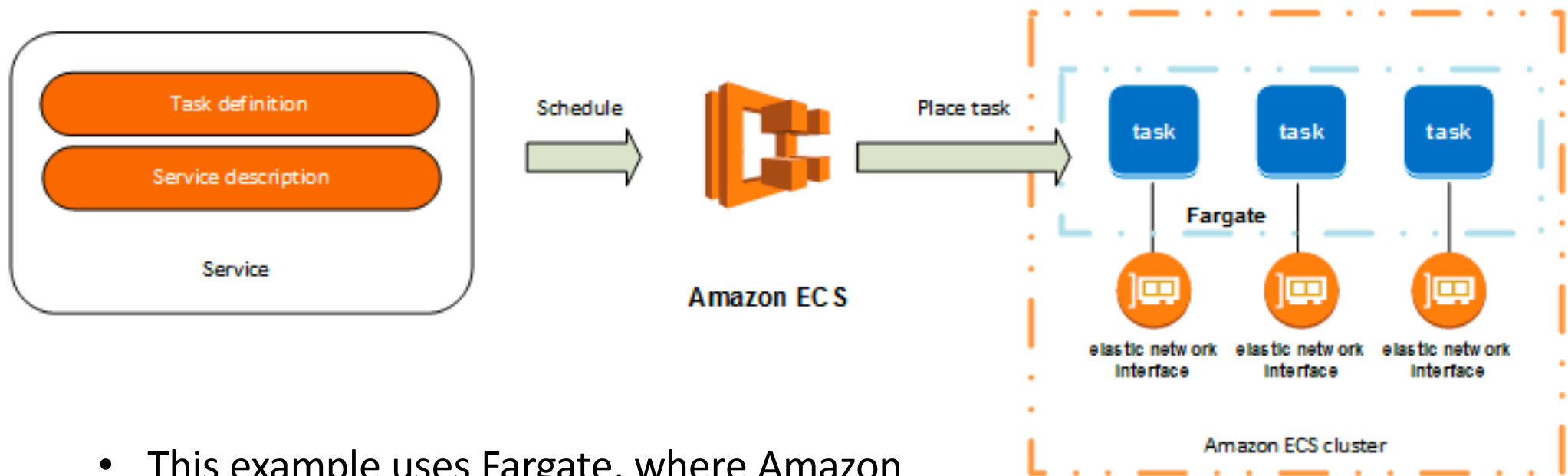
Container Agents

- Run on each infrastructure element in a cluster
- Send info about state to ECS
- Start and stop tasks on request



Elastic Container Service

Tasks and Scheduling



- This example uses Fargate, where Amazon manages resources
 - Tasks are groupings of multiple containers
 - Services are 'auto-scaling groups' for tasks
- For EC2, you have to manage your resources yourself
- A cluster is a logical grouping of resources

Elastic Container Service

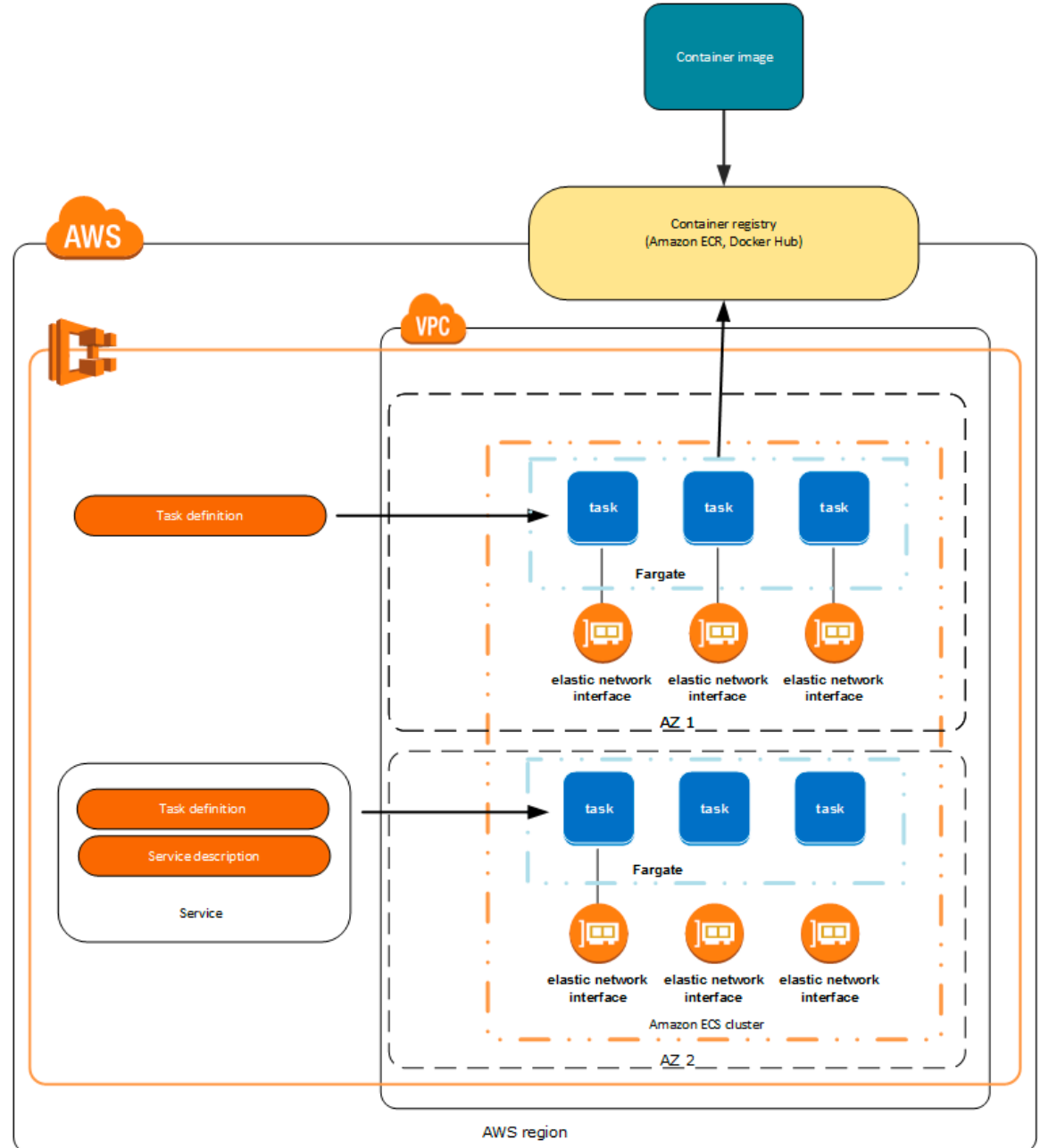
Task Definitions

- Define (in JSON) the containers that make up an application (i.e. a blueprint for the application)
- Example: NGINX web server with Fargate

```
{
  "family": "webserver",
  "containerDefinitions": [
    {
      "name": "web",
      "image": "nginx",
      "memory": "100",
      "cpu": "99"
    }
  ],
  "requiresCompatibilities": [
    "FARGATE"
  ],
  "networkMode": "awsvpc",
  "memory": "512",
  "cpu": "256",
}
```


ECS Fargate Launch Type

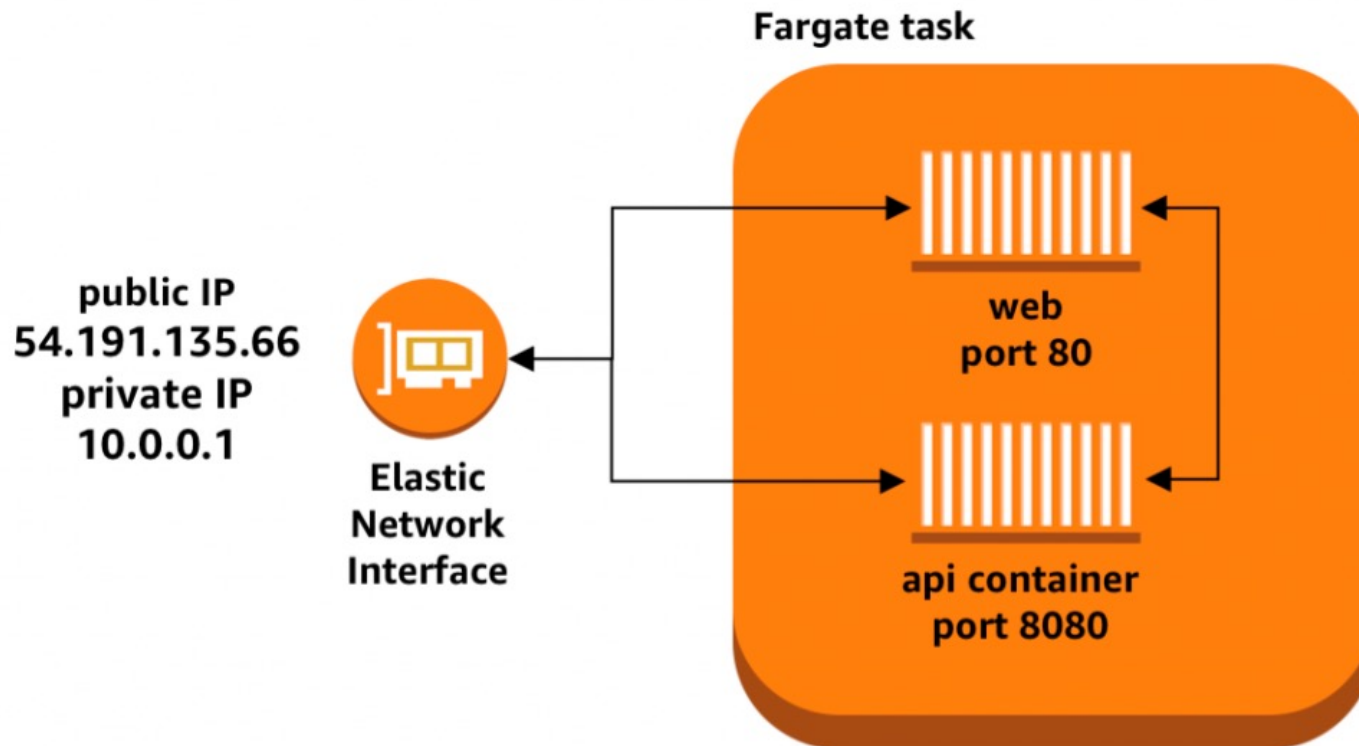
- Fully managed
- Emphasized by AWS
- Potential alternative to Lambda for workloads of short duration



Source:

<https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html>

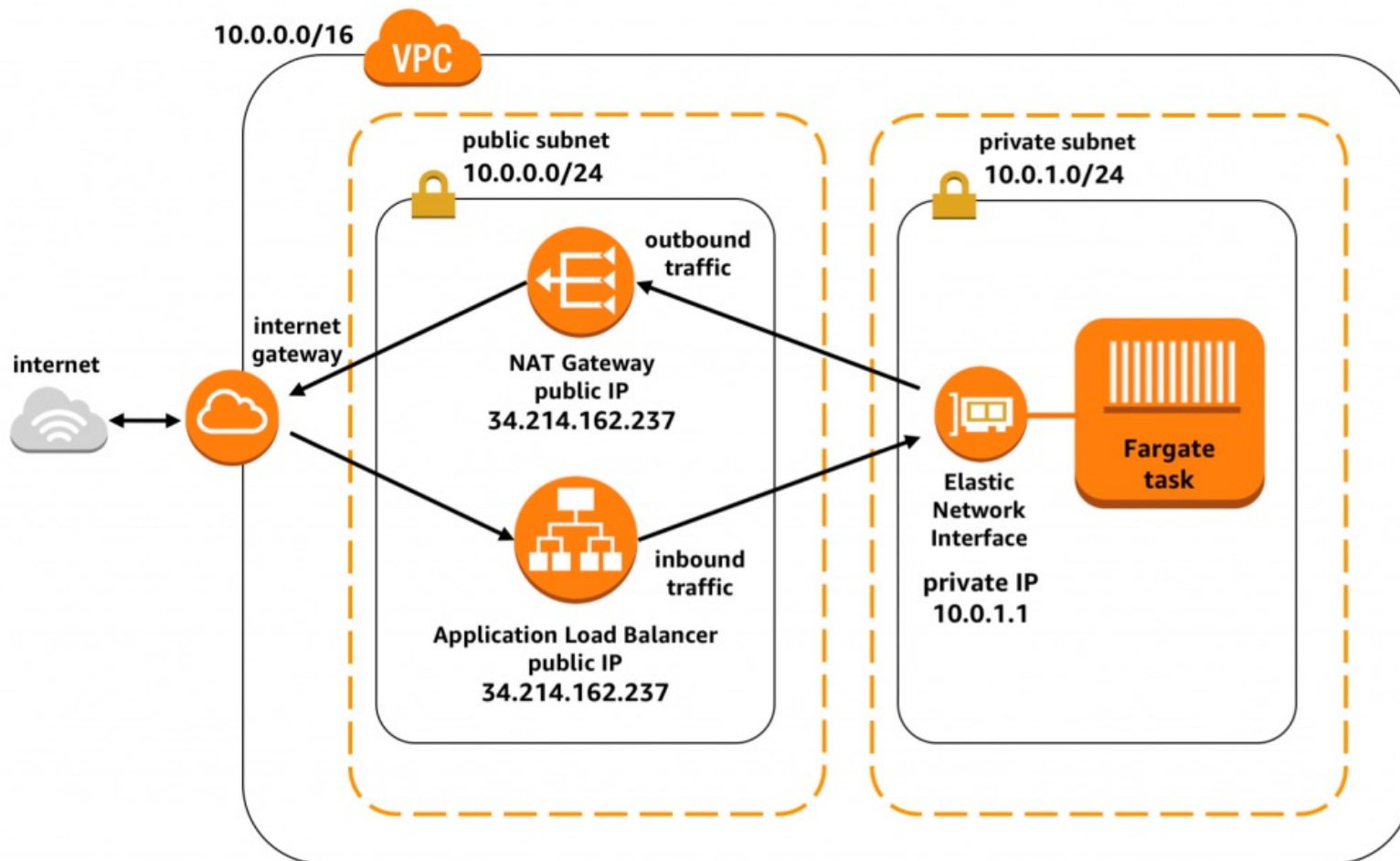
Container networking in Fargate



- With **awsvpc** all containers share an elastic network interface (ENI)
- Containers within a task can use a 'local loopback' with their static ports
- In public subnets, the containers can communicate with the Internet through the ENI
- <https://aws.amazon.com/blogs/compute/task-networking-in-aws-fargate/>

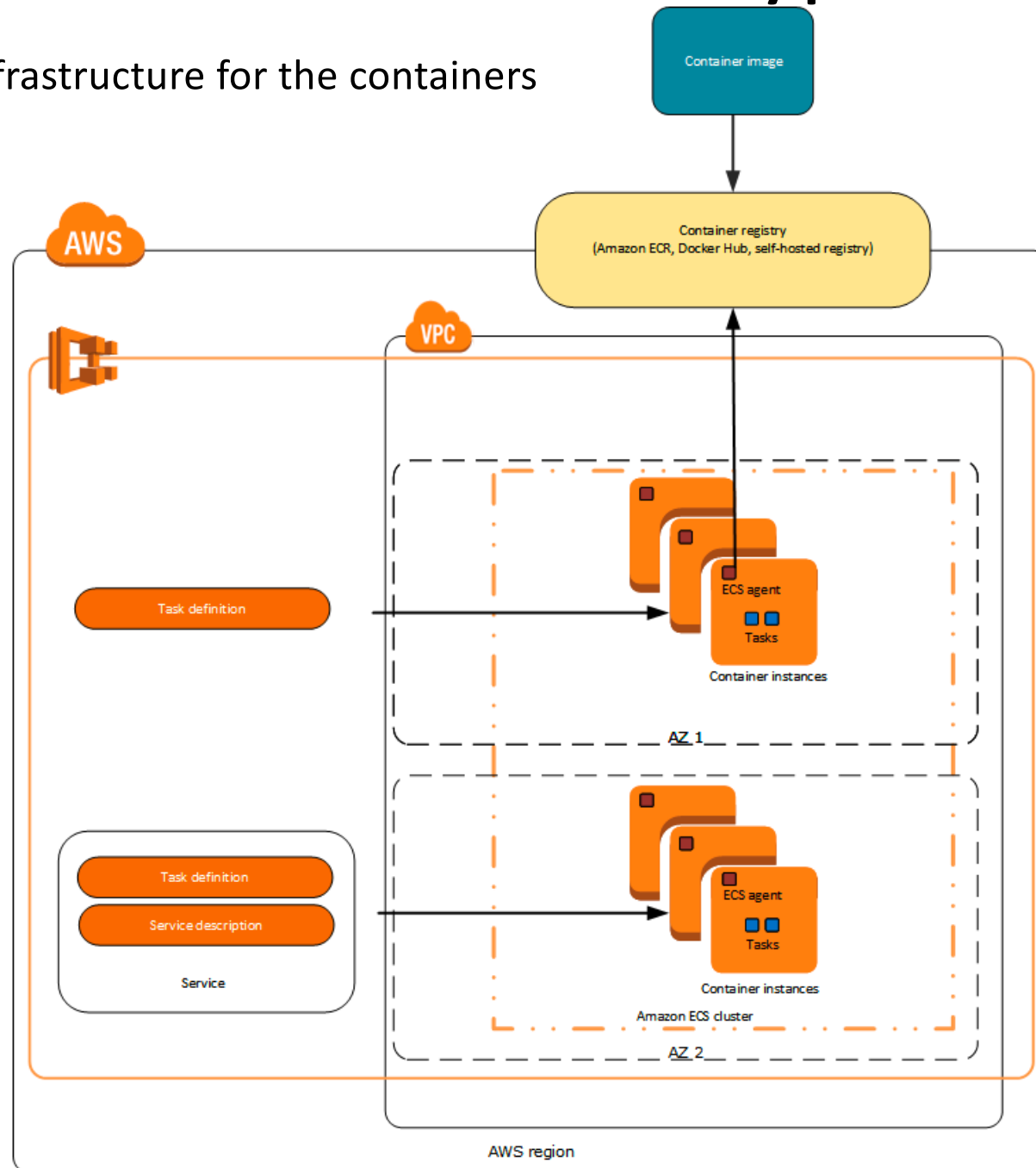
Fargate Tasks in Private Subnets

NAT Gateways and Load Balancers



ECS EC2 Launch Type

You control the infrastructure for the containers

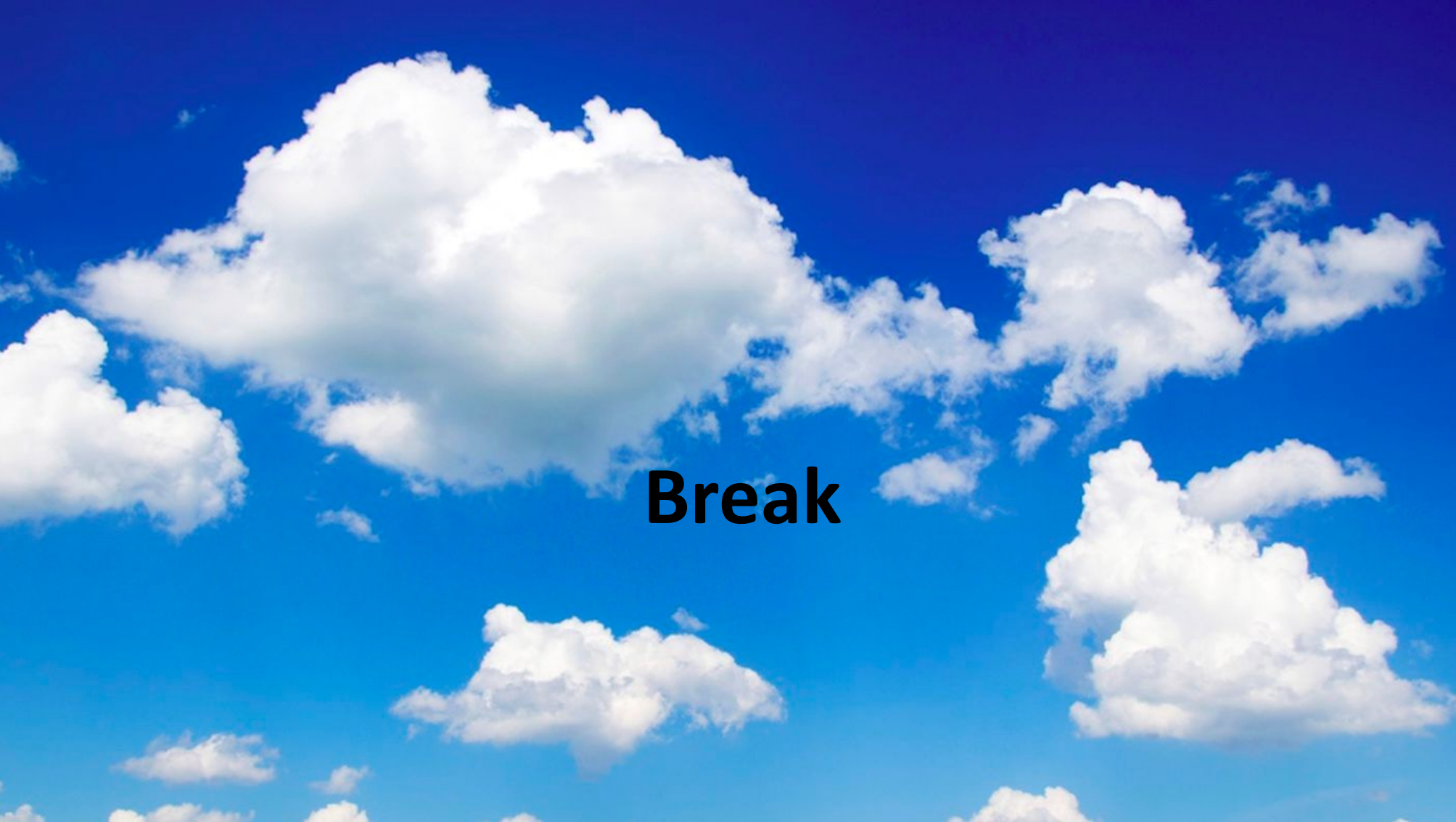


Source:

https://docs.aws.amazon.com/AmazonECS/latest/developerguide/launch_types.html

Connecting to a Database

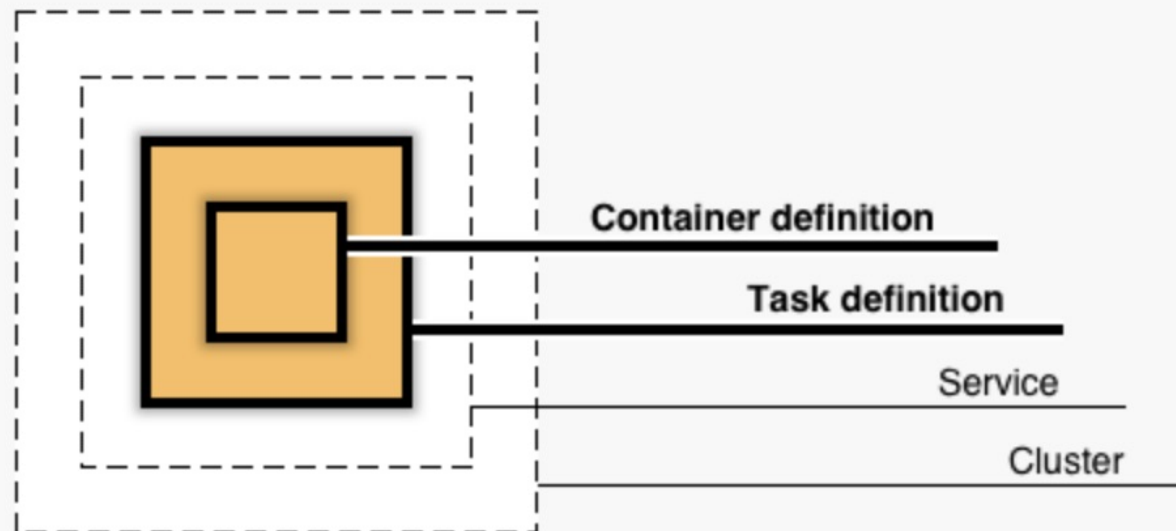
- <https://repost.aws/knowledge-center/ecs-fargate-task-database-connection>
- Containers are stateless, so you need to provide persistent state through a database, filesystem or EBS volume
- We will use the mechanism demonstrated for WordPress on an EC2 host also for ECS



Break

Setting up a cluster with ECS

- Go to ECS console
 - <https://console.aws.amazon.com/ecs/home?region=us-east-1#/getStarted>
- Set up a **cluster**
- Fargate is the default infrastructure (but instances are an option)
 - Fargate completely manages the environment



Configuring a Cluster

1. Define **task** (in the console or by submitting a JSON file)
 - Choose the 'launch type' (app environment) and network mode
 - Task definition requires roles, choose LabRole
 - Choose an 'essential container' image from a repository
 - Go with default for sizes, define port mapping
 - You can **deploy a task directly** into a cluster (choose launch type or capacity provider)
2. Define **service** (in the console or by submitting a JSON file)
 - Security group (add a group that allows access to port 80)
 - Number of tasks (default: 1)
 - Load balancer can be selected (default: None)
 - Review and create
 - AWS brings up status page for cluster
3. View (and modify) Service
 - Cluster and Task Definition
 - The task definition specifies a container image
 - In the tasks tab, select the task and under 'Network' find public IP. Navigate to it in your browser
 - Start and stop tasks
 - Delete service after reducing desired tasks to 0

Creating a Service within a Cluster

- Go to Service Tab, choose Deploy
- Either go with Service Provider or Launch Type
- Deployment:
 - Choose Service (as opposed to single task)
 - Initially select 1 desired task
 - Select task definition (family)
 - Name the service
 - Deploy
- Check out the running task, access the public ip
- Before deleting the cluster, delete the service & verify later that everything has been deleted
 - Reduce the number of desired tasks to 0 first

Load Balancers and ECS

- Putting an application load balancer in front of your containers allows directing traffic to specific containers based on listener rules
- You can choose an existing load balancer **when creating a service**
 - You may have to modify/delete the listener
- The load balancer needs to be in the same VPC as your cluster
- Choose a security group that gives access to port 80
- A target group is created for you, it has the target type 'IP' for Fargate and 'Instance' otherwise
- A listener is also created with a default rule to forward to the target group

us-east-1.console.aws.amazon.com/ec2/v2/home?region=us-east-1#TargetGroup:targetGroupArn=arn:aws:elasticloadbalancing:us-east-1:498790321813:targetgroup/EC2Co-Defau-2MVN1VB4PVCK/fabca5dfdf2e53b4

aws

Services

Search for services, features, marketplace products [Option+S]

vocstartsoft/user449003=kristof.kloeckner@iaas.uni-stuttgart....

N. Virginia

Support

EC2Co-Defau-2MVN1VB4PVCK

Delete

arn:aws:elasticloadbalancing:us-east-1:498790321813:targetgroup/EC2Co-Defau-2MVN1VB4PVCK/fabca5dfdf2e53b4

Basic configuration

Target type IP	Protocol : Port HTTP: 80 Protocol version HTTP1	VPC vpc-0725738173acbb135	Load balancer EC2Co-EcsEL-9VHF8QCHX46P
-------------------	--	------------------------------	---

Group details

Targets

Monitoring

Tags

Registered targets (2)

Filter resources by property or value

IP address

	IP address	Port	Zone	Status	Status details
☐	10.0.1.234	80	us-east-1b	healthy	
☐	10.0.0.225	80	us-east-1a	healthy	

Refresh

Deregister

Register targets

<

1

>

Settings

Building a WordPress Cluster

- Create a cluster wordpress-cluster (preferably with Fargate)
- Create a task definition (wordpress-task)
 - Use the official wordpress image (either from download or directly)
 - Provide mapping for port 80 (usually set as default)
 - You can (initially) use defaults
 - Use LabRole for task and execution roles
 - Add environment variables WORDPRESS_DB_HOST with RDS endpoint, WORDPRESS_DB_USER, WORDPRESS_DB_NAME, WORDPRESS_DB_PASSWORD
 - You can also store the value of e.g. the password in an AWS secret
- Create a security group sg-wp (or reuse existing ones)
 - outbound rule for port 3306 to access the RDS instance (e.g. ec2-rds-xx, to match rds-ec2-xx)
 - inbound rule to allow access to port 80/443
- Create a service (wordpress-service)
 - Use wordpress-task
 - Initially set desired task as 1 and don't assign a loadbalancer
 - Use sg-wp
- When directing the browser to the task's public IP, you will see a WordPress installation screen

Microservice Implementation with ECS

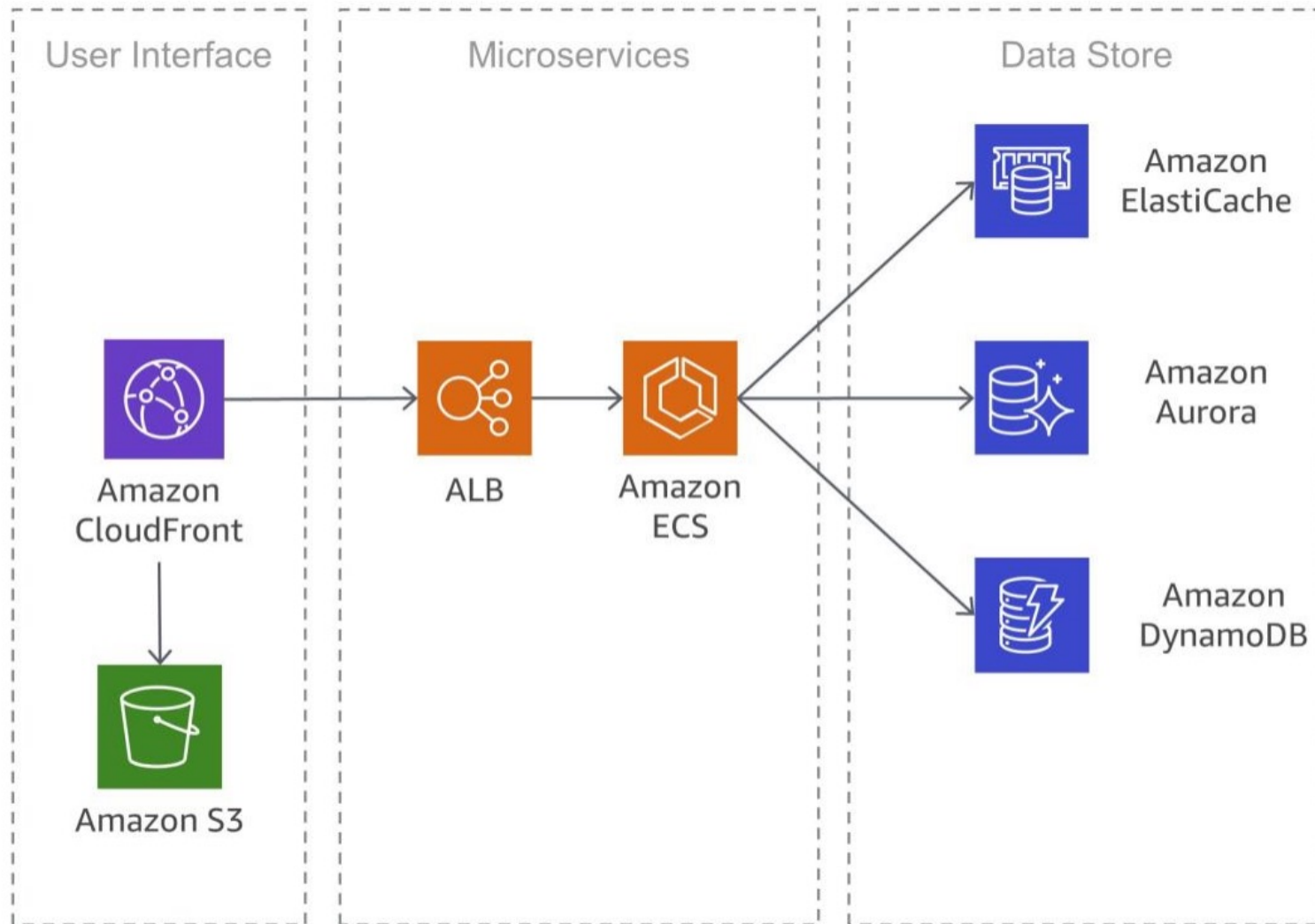


Figure 1: Typical microservices application on AWS

Serverless Microservices with Fargate

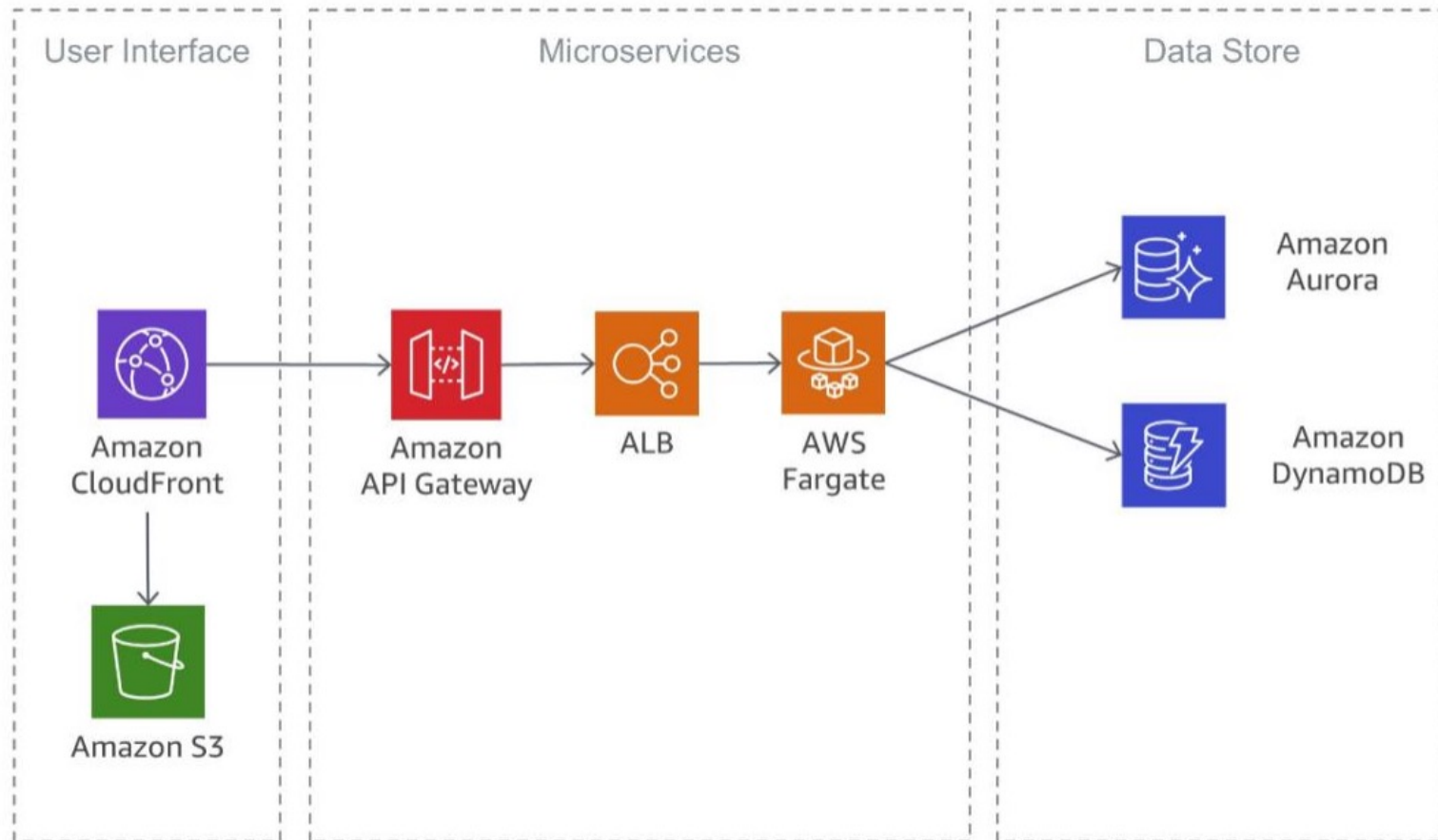
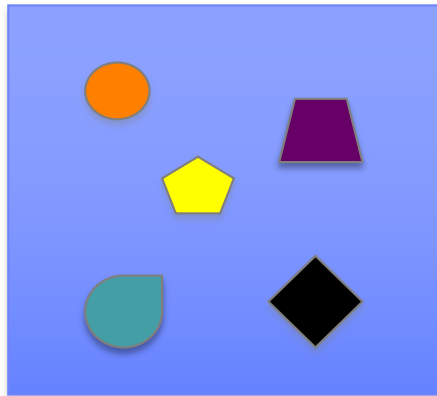


Figure 4: Serverless microservice using AWS Fargate

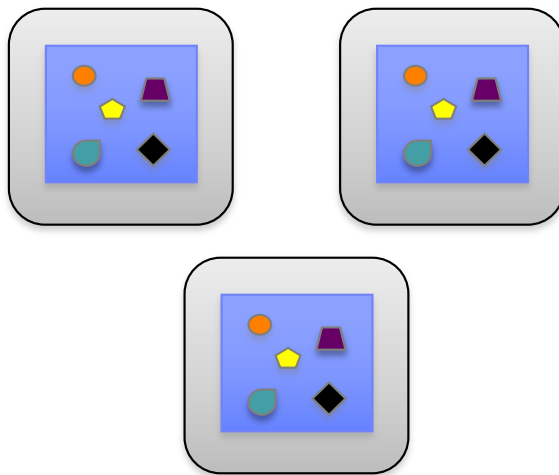
Source: Implementing Microservices with AWS

Moving from monolithic applications to **micro-services**

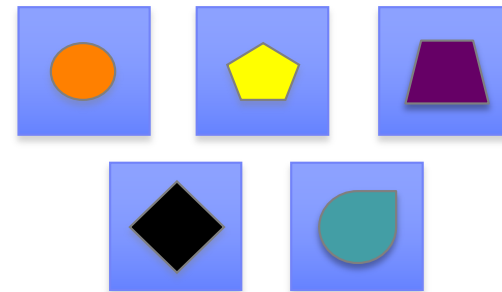
Monolithic app



Scaling

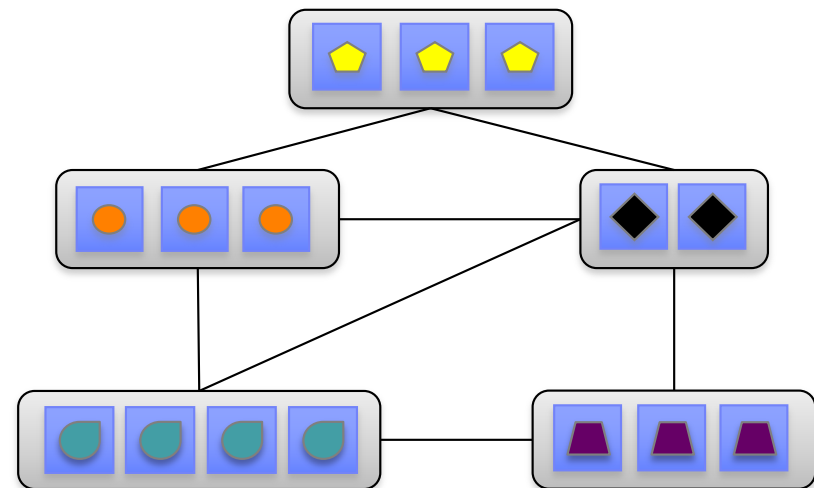


Micro services



Separation of
concerns

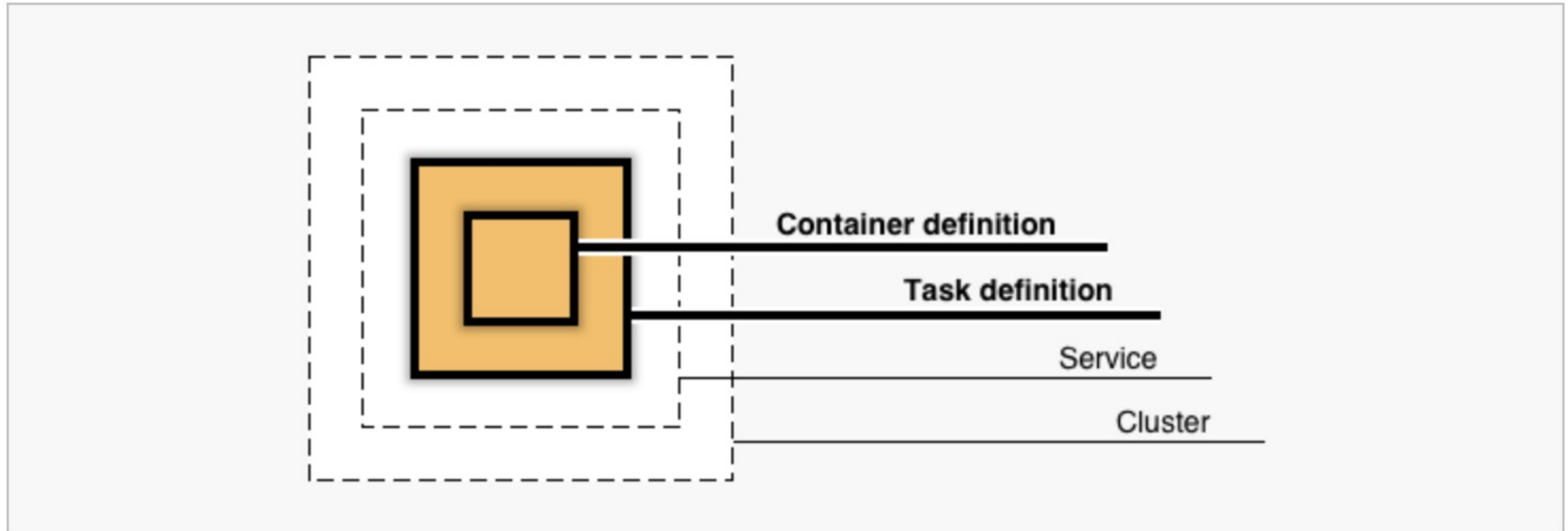
Scaling





Container Management, Part 2

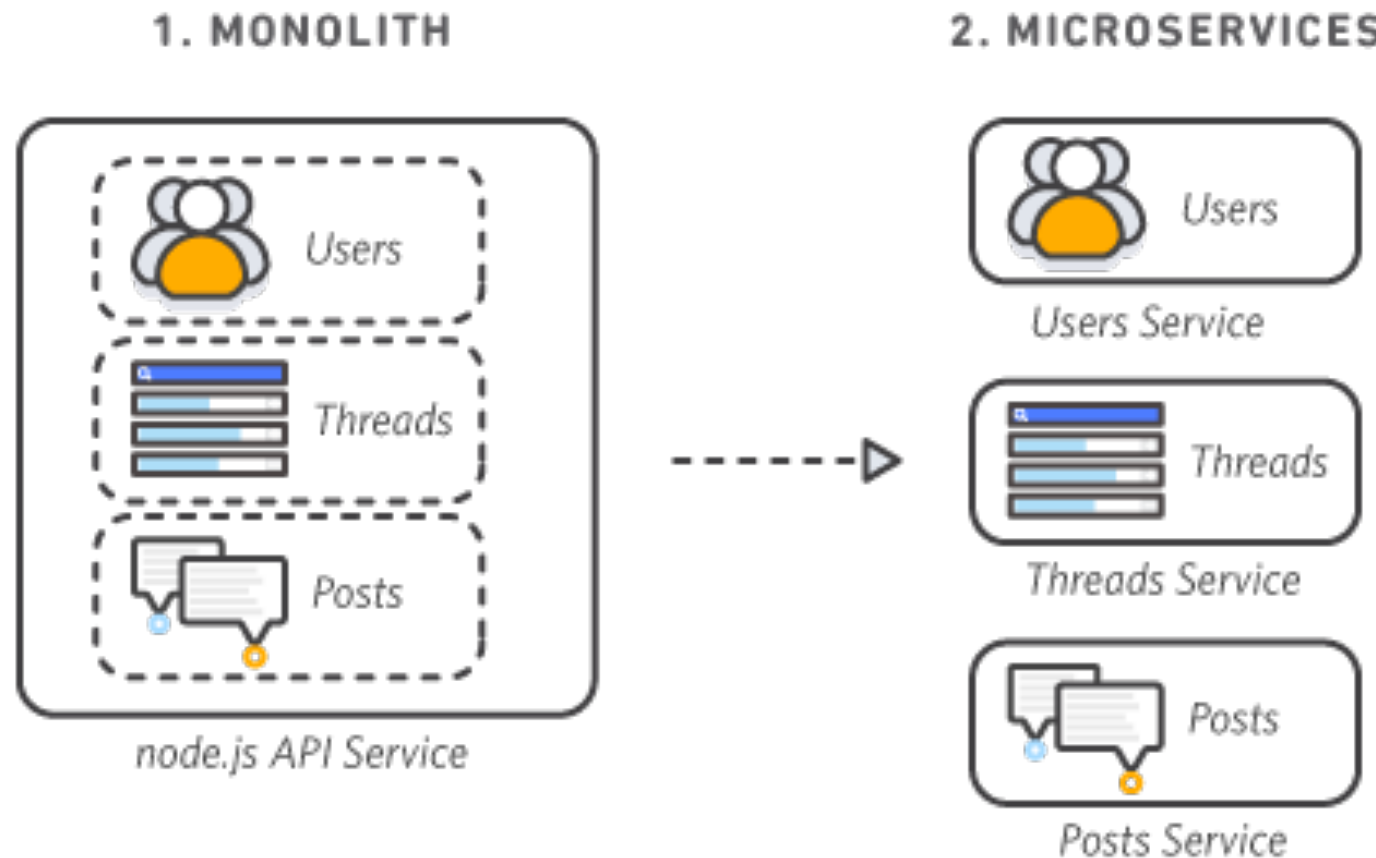
Elastic Container Services Concepts



- A **cluster** has infrastructure (capacity provider) options
 - The default setup is serverless (Fargate)
- A **task** is a collection of containers
- A **service** is an autoscaling group of tasks, it can be load-balanced
 - A **cluster** can run multiple services

Breaking up a Monolith into Microservices using Containers

<https://aws.amazon.com/getting-started/hands-on/break-monolith-app-microservices-ecs-docker-ec2/>



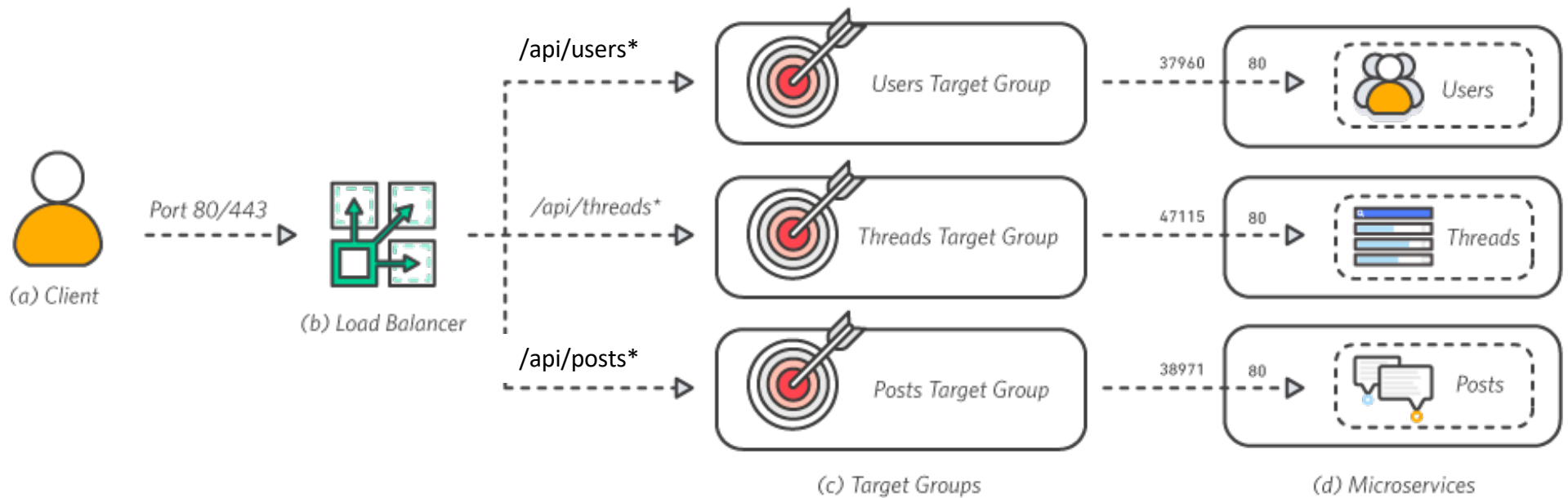
Steps to break up a Monolith into Microservices

- The original setup manually deploys containers onto instances, you can use the instructions uploaded to ILIAS
- The new setup uses co-pilot, which unfortunately tries to create roles
 - https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Copilot.html
- Clone code from Github
 - git clone <https://github.com/awslabs/amazon-ecs-nodejs-microservices>
- Use an EC2 instance or CloudShell for docker, CLI etc.
- Use the console to create ECS resources (or modify ecs.yml) to avoid creating new roles which is not permitted for AWS Academy
- The setup uses container instances, you can also use Fargate
- Steps
 1. Containerize the Monolith (a node.js application listening on port 3000)
 - Test the container with `docker run -l -t -p 80:3000 api`
 2. Deploy the Monolith (with load balancer)
 3. Break the Monolith (into microservices)
 4. Deploy the former Monolith as microservices, adding rules to the listener
- Delete the resources (especially load balancer) to avoid costs

Breaking up the Monolith with ECS Fargate

- Create an ECS cluster with Fargate
- Create a task definition (api) with 3000 as the container port
- Create a service with 1 task
 - Enable port 3000 for Custom TCP in your security group
 - Find public address in task tab, go to `public_address:3000/api`
- Else with 2 tasks and a new loadbalancer
 - Modify the load balancer security group to allow HTTP traffic on port 80
 - Test the service with `loadbalancer_dns/api`

Orchestrated Containerized Microservices

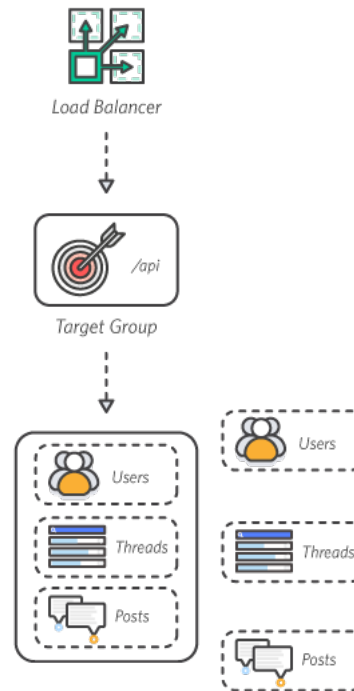


Note: In the listener, the paths need to be */api/users**, */api/threads**, */api/posts**

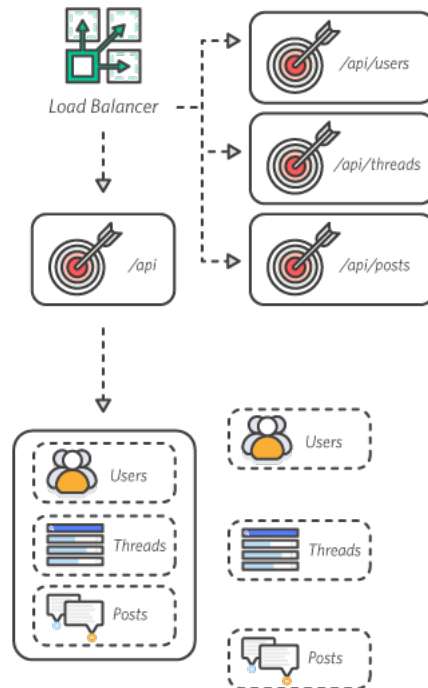
(1) Switching Traffic



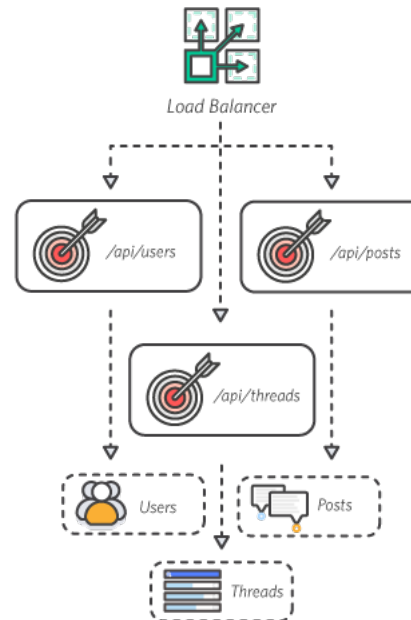
(2) Start Microservices



(3) Update Target Groups



(4) Shut Down Monolith



Creating a Cluster with EC2

- Setting up the cluster allows you to select the infrastructure hosting the containers
 - Select 'Fargate and self-managed instances' as infrastructure
 - Create an auto-scaling group (or choose an appropriate existing one) and set desired capacity
 - Choose an instance size big enough to host your images
 - Choose LabRole as instance role
 - Choose subnets and security group with access to the Internet, enable public IP
- Create or reuse a task definition with 'App Environment EC2'.
 - Default networking mode is '**Bridge**' for Linux, '**awsvpc**' for Fargate
 - You need to select an image and define run time parameters, including port mappings
 - Don't allocate too much memory for a simple container
- Create a service (or task) to run in the cluster using the task definition
 - Specify number of tasks (running containers) plus(existing) load balancer, if desired
 - When running multiple containers on an instance, beware of memory and port limitations
- If using an httpd public image, you can test it by invoking the public IP address of the EC2 image hosting your containers
- Before deleting the cluster, stop tasks, delete the service & verify later that everything has been deleted
- Note: There is a new option 'managed instances' which we cannot use with Learner Labs



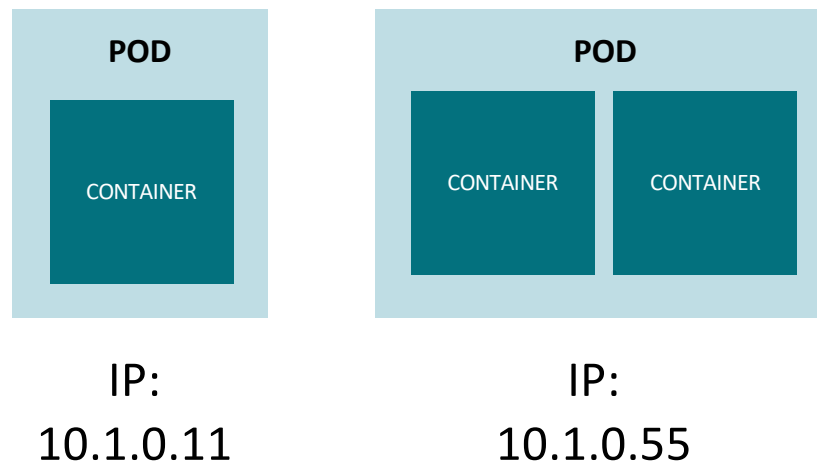
Kubernetes

Kubernetes Container Orchestration

- Kubernetes is Greek for 'helmsman'
- Kubernetes originated with Google in 2014
- Now sponsored by the Cloud Native Foundation as a 'graduated project'
- Supported by all major cloud providers
- Central to Google, Azure, IBM (Red Hat)
- Third largest number of committers on GitHub



Containers are wrapped in pods which are units of deployment and management

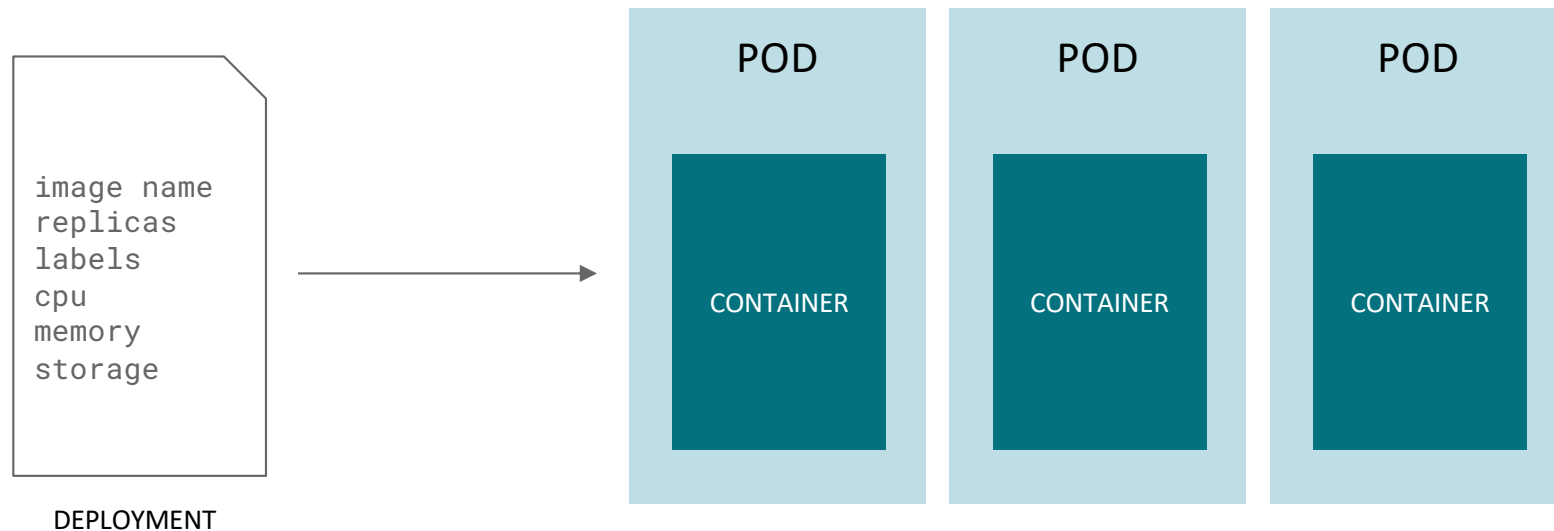


Pod: pod of whales

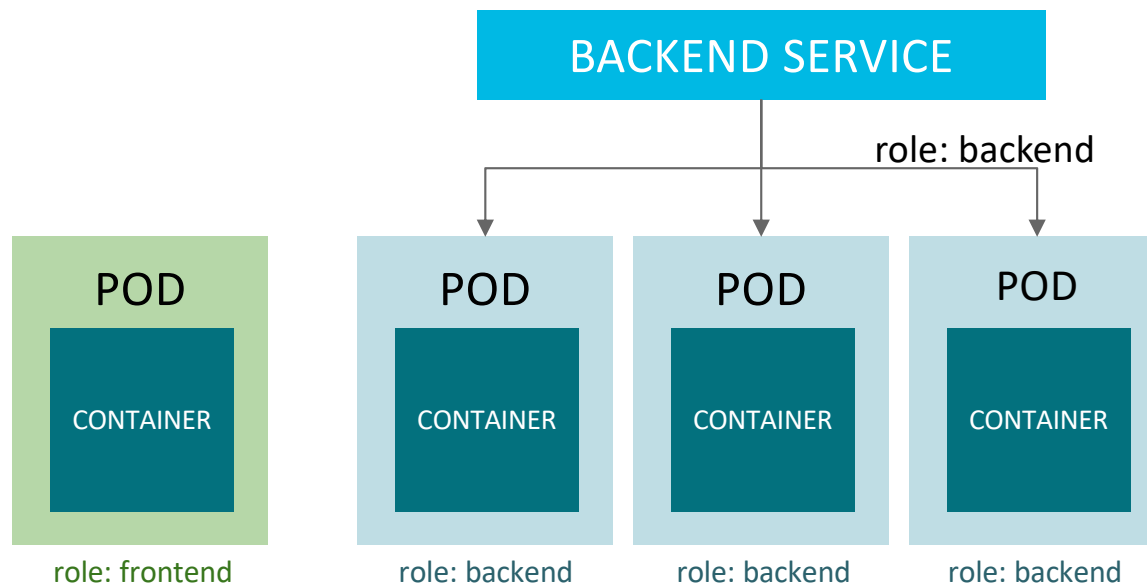
Kubernetes Pods

- Smallest deployment unit in k8s
- Collection of containers that run on a worker node
 - Roughly similar to ECS task
- Each has its own IP
- Pod shares a PID namespace, network, and hostname

Pods configuration is defined in a deployment

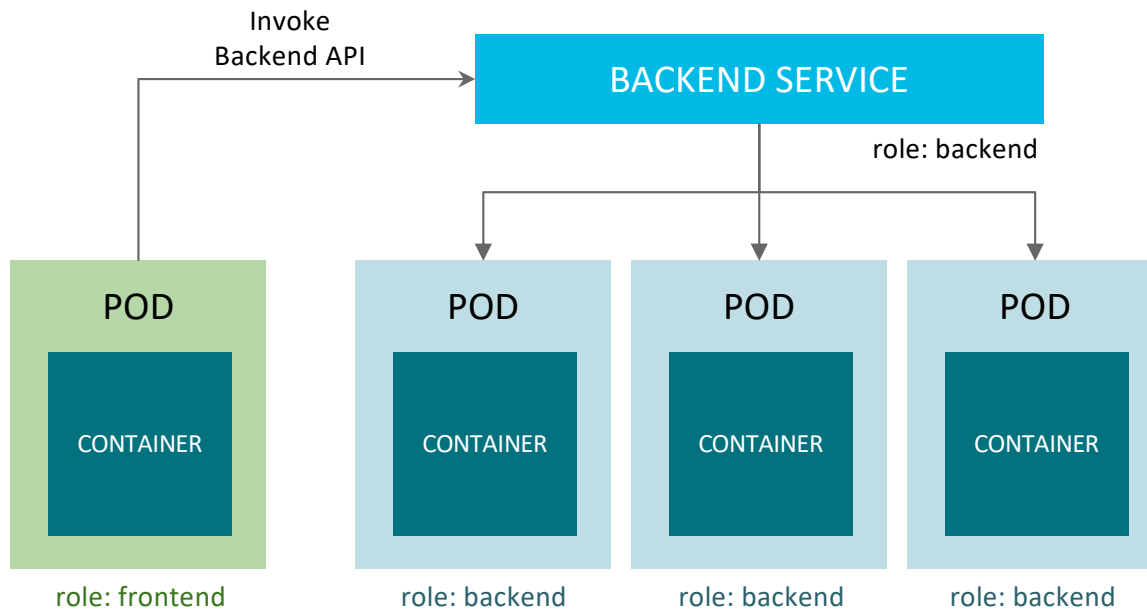


Services provide internal load-balancing and service discovery across pods

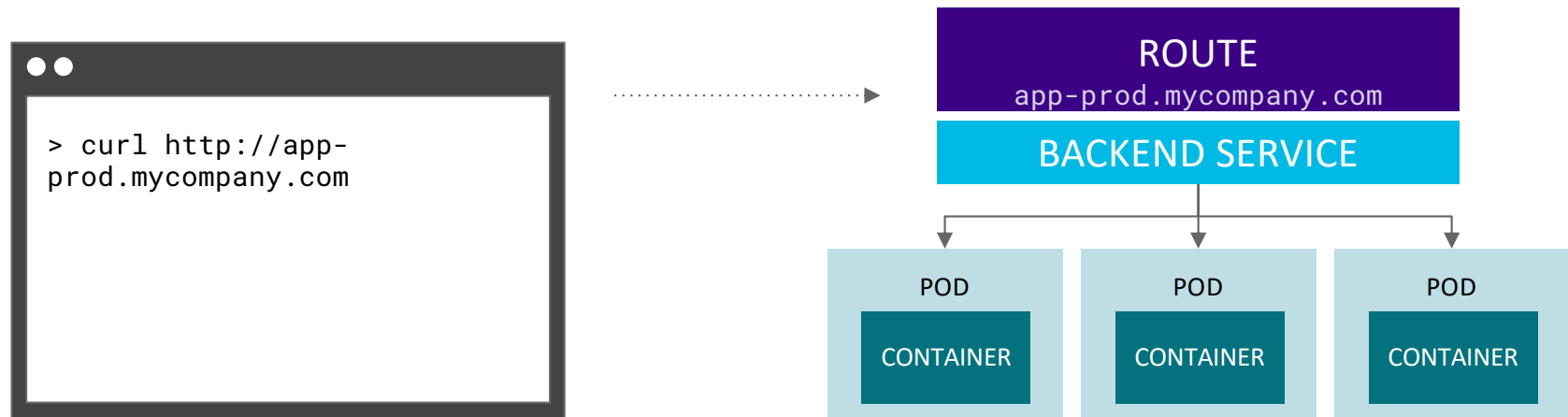


Roughly similar to an ECS service

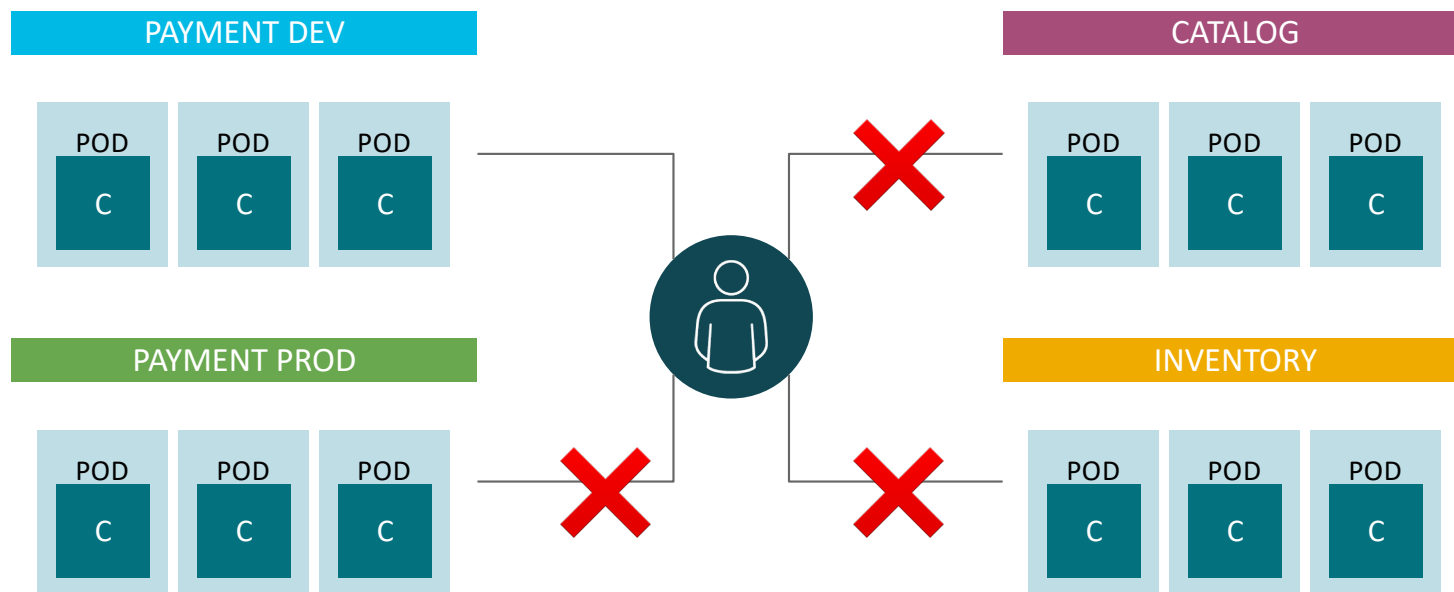
Apps can talk to each other via services



Routes add services to the external load-balancer and provide readable urls for the app

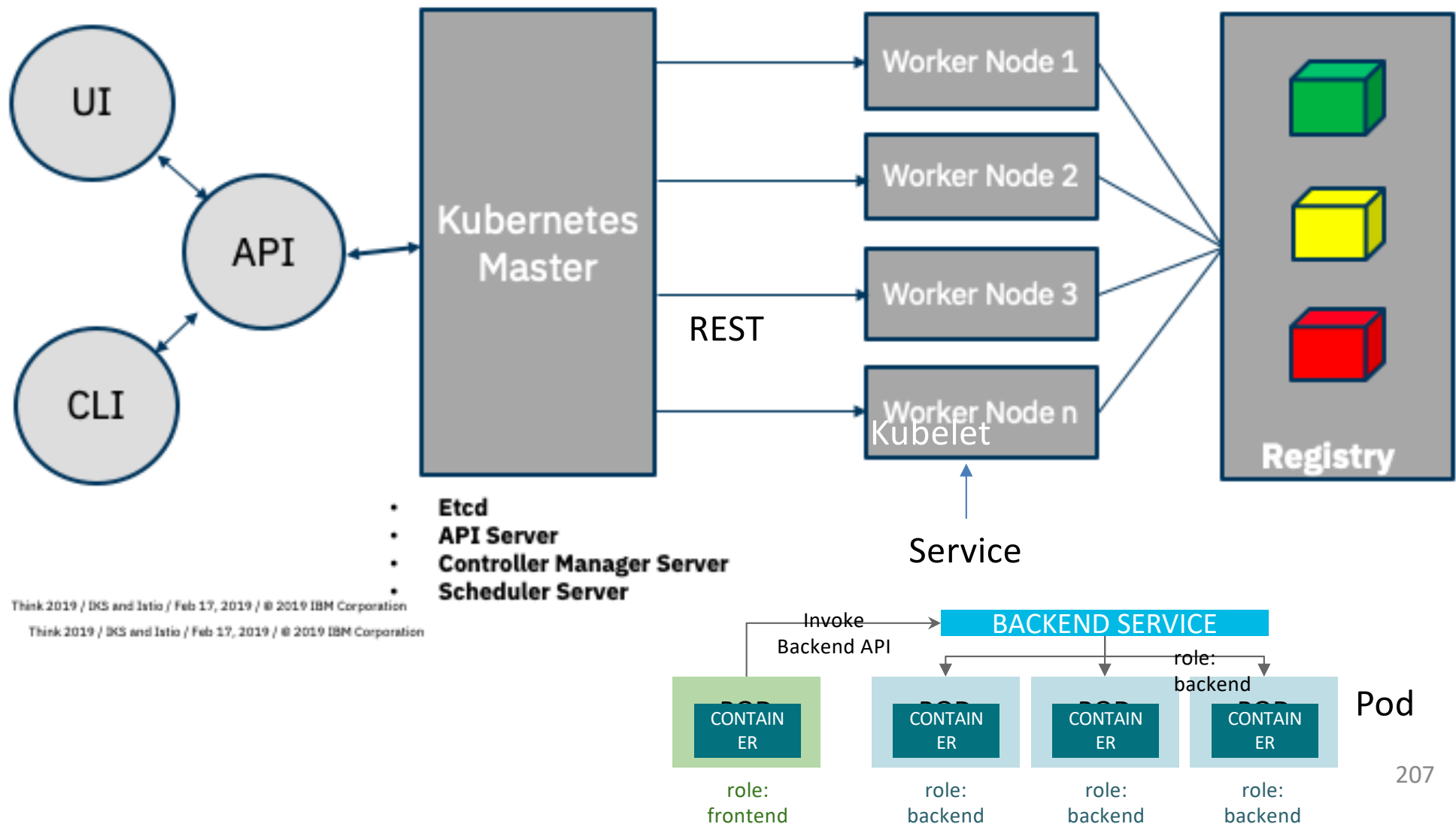


Projects isolate apps across environments, teams, groups and departments



- A user can only access projects that they have created or explicitly given access to

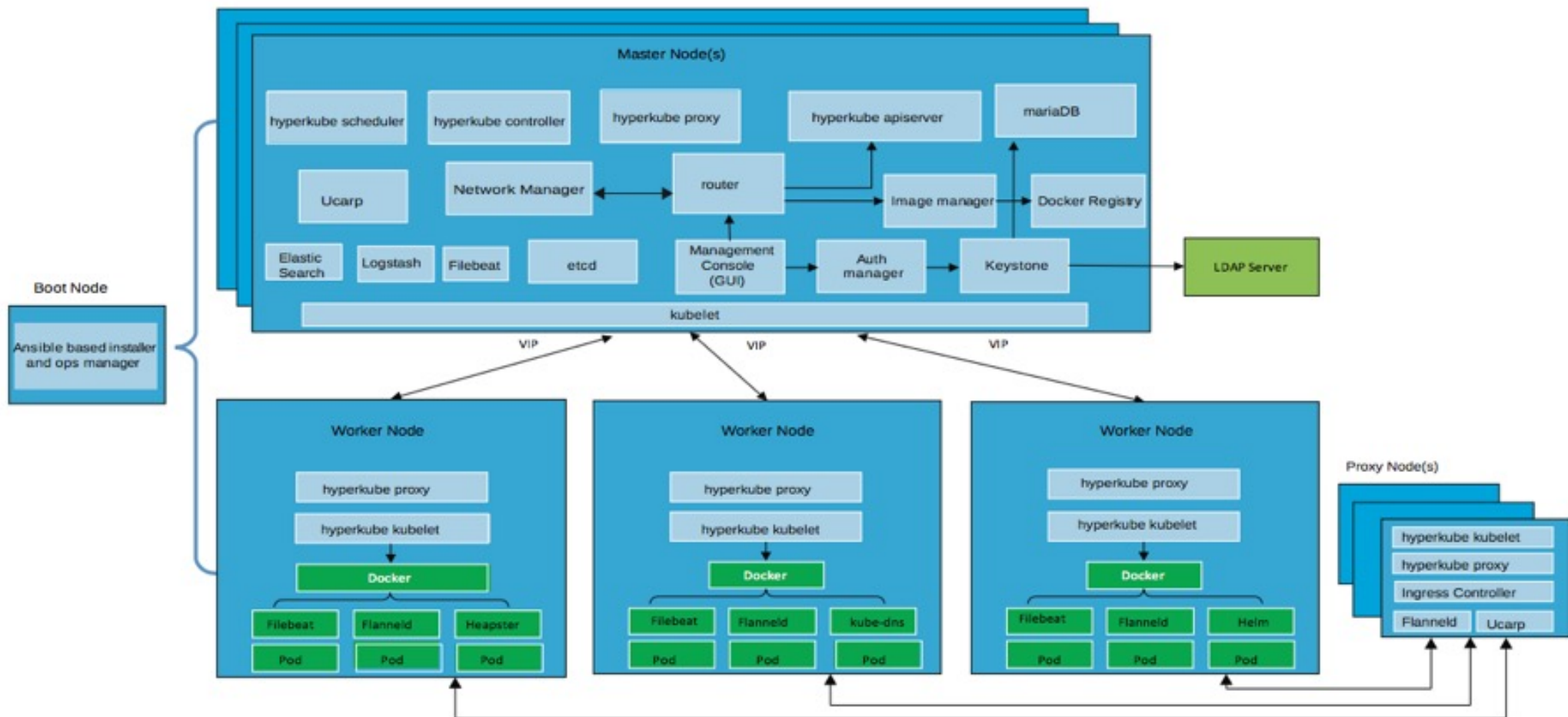
Kubernetes Architecture



Kubernetes (K8S) Components

- Nodes hosts that run Kubernetes applications
- Master node: Controls and manages the cluster
- kubectl (command line)
- REST API (communication with workers)
- Scheduling and replication logic
- Worker nodes: Host the K8s services
- Kubelet (K8s agent that accepts commands from the master)

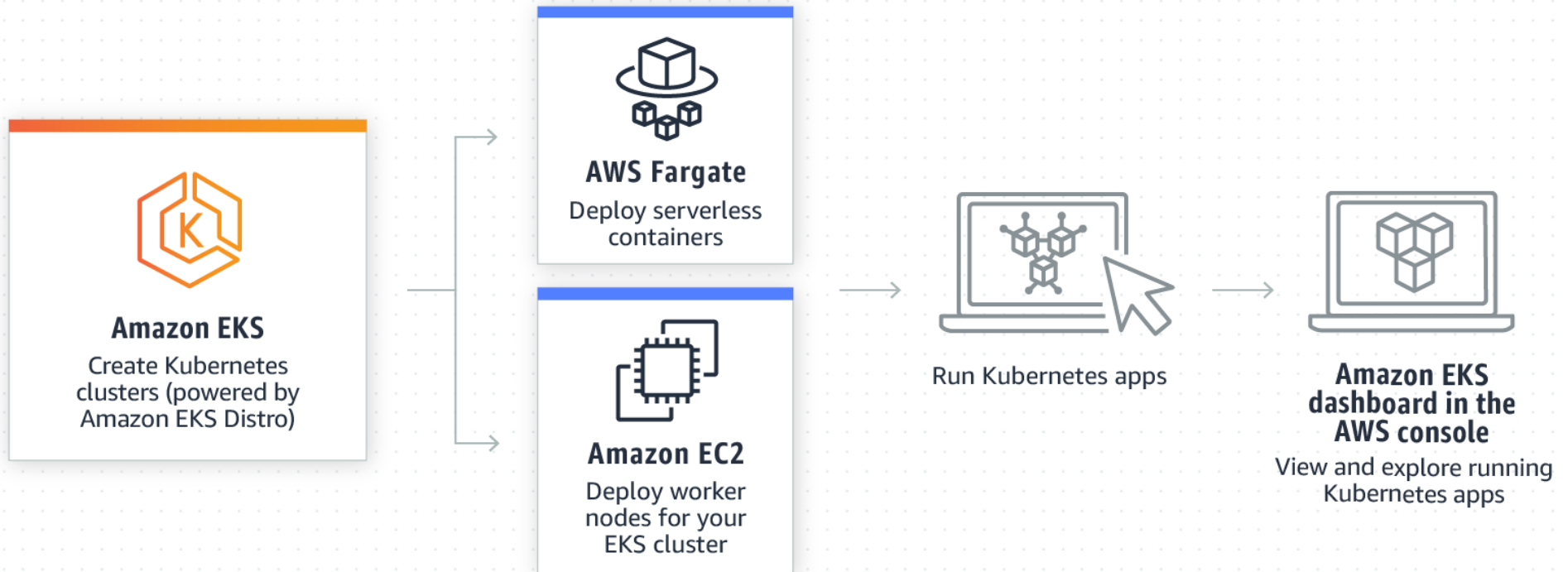
Detailed View of a Kubernetes-based 'Platform'





EKS

Elastic Kubernetes Service



- EKS provides a Kubernetes Control Plane in the Cloud or on premise (AWS outposts or self-managed infrastructure)
- EKS exposes a Kubernetes API endpoint. You can work with your favorite tools to manage your cluster

Elastic Kubernetes Service

- Fully managed **Kubernetes Control Plane**
<https://docs.aws.amazon.com/eks/>
- Tutorial at <https://www.eksworkshop.com/>
- Provides open container orchestration, helpful for
 - Hybrid multi-cloud environments
 - Application migration
 - Integration with other open source tools
- Relatively expensive (\$ 0.10/hour)
 - Free in Azure and GKE
- Access from AWS console

Managing EKS Resources

- AWS EKS Console for manual administration
- **CLI:** `aws eks <command>`
 - e.g. `aws eks update kube-config --name mycluster --region my-region`
(update a config file)
- **eksctl** command line tool to create/delete clusters and cluster resources
 - Maintained in GitHub, needs version 0.40+ for Fargate
 - ~~Requires temporary AWS credentials in Cloud9~~
 - e.g. `eksctl create cluster --name mycluster --region my-region --fargate`
(create a Kubernetes Fargate cluster)
- **kubect**l ‘standard’ Kubernetes command line tool to manipulate services and resources (installed in CloudShell)
 - e.g. `kubect`l apply -f sample-service.yaml (deploy a service)

Using the EKS Console & kubectl

<https://docs.aws.amazon.com/eks/latest/userguide/getting-started-console.html>

- Establish prerequisites
 - Choose custom configuration, LabRole doesn't have permissions for AutoEKS
 - Create EKS IAM role that is allowed to manage the cluster
 - Follow guidance to create role and select EKS and use case EKS-Cluster
 - **With AWS Academy, use LabEKSClusterRole and LabEKSNodeRole**
 - Choose VPC (or create one), subnets etc. Some availability zones may be out of resource.
 - Install and configure kubectl <https://docs.aws.amazon.com/eks/latest/userguide/install-kubectl.html> in a terminal (or use Cloud Shell)
 - (If not using an AWS Academy account, install eksctl)
- In the console, create a **cluster** with defaults (from the console) – this takes a while
 - Availability zones may be out of resource
- Create a **kubeconfig** file and test the configuration

```
aws eks --region region-code update-kubeconfig --name cluster_name
kubectl get svc (test the configuration)
```
- In Azure, use `az aks get-credentials -n cluster -g clustergroup`
- Go to 'compute' tab and create **managed node group** or a Fargate profile
 - Managed node groups seem to work better than Fargate. Default is 2 nodes.
 - Choose a namespace when creating the profile, e.g. eks-sample-app, matching the next steps

Deploying a workload with kubectl

<https://docs.aws.amazon.com/eks/latest/userguide/sample-deployment.html>

- Create a **namespace**
 - `kubectl create namespace eks-sample-app`
- Create a **deployment** manifest & **service** manifest (for simple nginx service)
 - ‘eks-sample-deployment.yaml’ & ‘eks-sample-service.yaml’ (use Azure image for Azure)
- **Apply** the deployment manifest to the cluster
 - `kubectl apply -f eks-sample-deployment.yaml`
 - `kubectl apply -f eks-sample-service.yaml`
- View the resources and details
 - `kubectl get all -n eks-sample-app`
 - `kubectl -n eks-sample-app describe service eks-sample-linux-service`
 - `kubectl -n eks-sample-app describe pod eks-sample-linux-deployment-65b7669776-m6qxz` (replace with id of one of your pods)
- Run a shell in one of the pods
 - `kubectl exec -it eks-sample-linux-deployment-65b7669776-m6qxz -n eks-sample-app - /bin/bash`
 - In the shell: `curl eks-sample-linux-service`
 - In the shell: `cat /etc/resolv.conf`
- In Azure, use `/bin/sh` in the command above
- Delete the namespace: `kubectl delete namespace eks-sample-app`

Notes for EKS with Fargate

- Make sure you create a VPC with public and private subnets. Only private subnets are allowed to hold pods running on Fargate.
- Create a Fargate profile

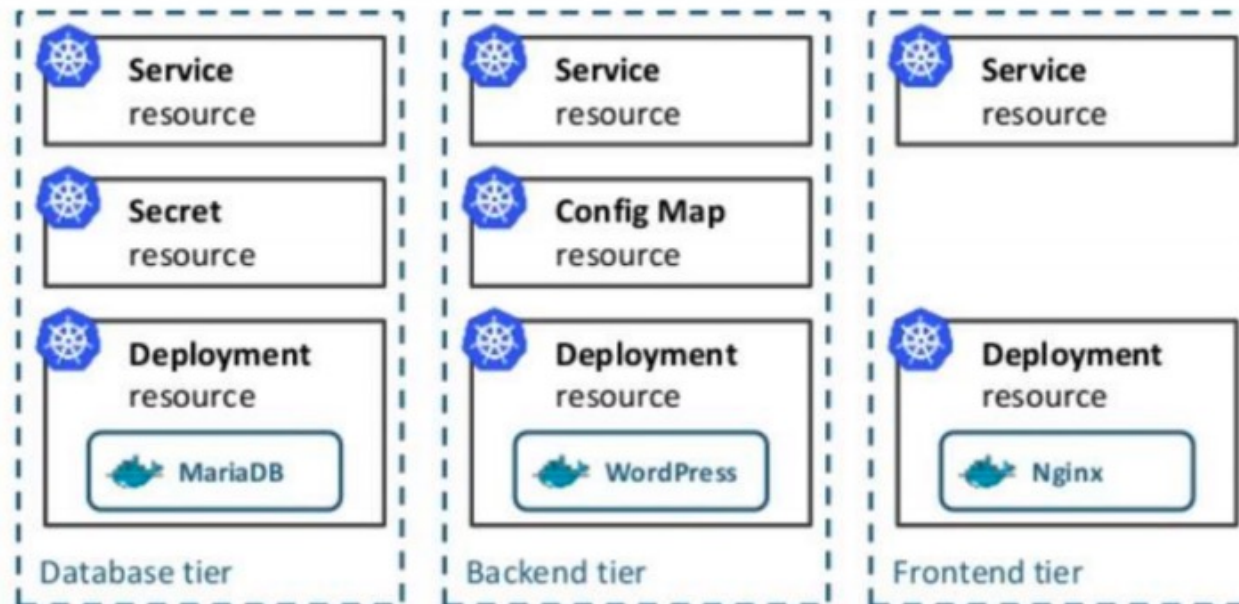
Getting started with EKSCTL

(Won't work with AWS Academy Learner Lab)

- Install eksctl and kubectl
 - <https://eksctl.io/installation/>
- Follow the instructions in <https://docs.aws.amazon.com/eks/latest/userguide/getting-started-eksctl.html>, e.g. for Fargate
 - `eksctl create cluster --name my-cluster --region us-east-1 --fargate`
 - `aws eks update-kubeconfig --name my-cluster --region us-east-1`
- Use a sample-service.yaml file following <https://docs.aws.amazon.com/eks/latest/userguide/sample-deployment.html>
- Deploy a workload with kubectl and eksctl
 - `eksctl create namespace my-namespace`
 - `eksctl create fargateprofile --cluster mycluster --name myprofile --namespace my-namespace`
 - `kubectl apply -f sample-service.yaml`

A Microservice in Kubernetes

1 Microservice == 1 POD + 1 Deployment + 1 ReplicationSet + 1 Service (+ 1 Ingress)



Amazon App Runner

- Similar to Elastic Beanstalk, but only for containers
- Dynamically creates a container image from code, if you haven't created one
- Takes care of load balancing and auto-scaling
- Can be considered a usability extension to Fargate
- <https://aws.amazon.com/getting-started/guides/deploy-webapp-apprunner/?ref=gsrchandson>
 - This tutorial creates an App Runner deployment from an image (nginx-web-app)
 - Needs an ECR access role, so won't work with Learner Labs

Options available to run containers on AWS



**AWS
App Runner**



**AWS
Lambda**



**Amazon
Lightsail**



**AWS
Batch**

Vertical Solutions



**Amazon
ECS**



**Amazon
EKS**



ROSA

Orchestration

Customer-managed (AWS Outposts)



**Amazon
EC2**



**AWS
Fargate**



Server



Edge

Capacity

Suggested Exercises

- Complete the exercises demonstrated during the session
- Read some blogs about containers, for instance
 - <https://aws.amazon.com/blogs/compute/building-deploying-and-operating-containerized-applications-with-aws-fargate/>
 - <https://cloud.google.com/solutions/best-practices-for-building-containers>
- Browse
 - https://docs.aws.amazon.com/decision-guides/latest/containers-on-aws-how-to-choose/choosing-aws-container-service.html?icmpid=docs_homepage_featuredsvcs

Further Suggested Reading

Implementing Microservices with AWS

<https://docs.aws.amazon.com/whitepapers/latest/microservices-on-aws/microservices-on-aws.html>

Module 10: Streaming Analytics

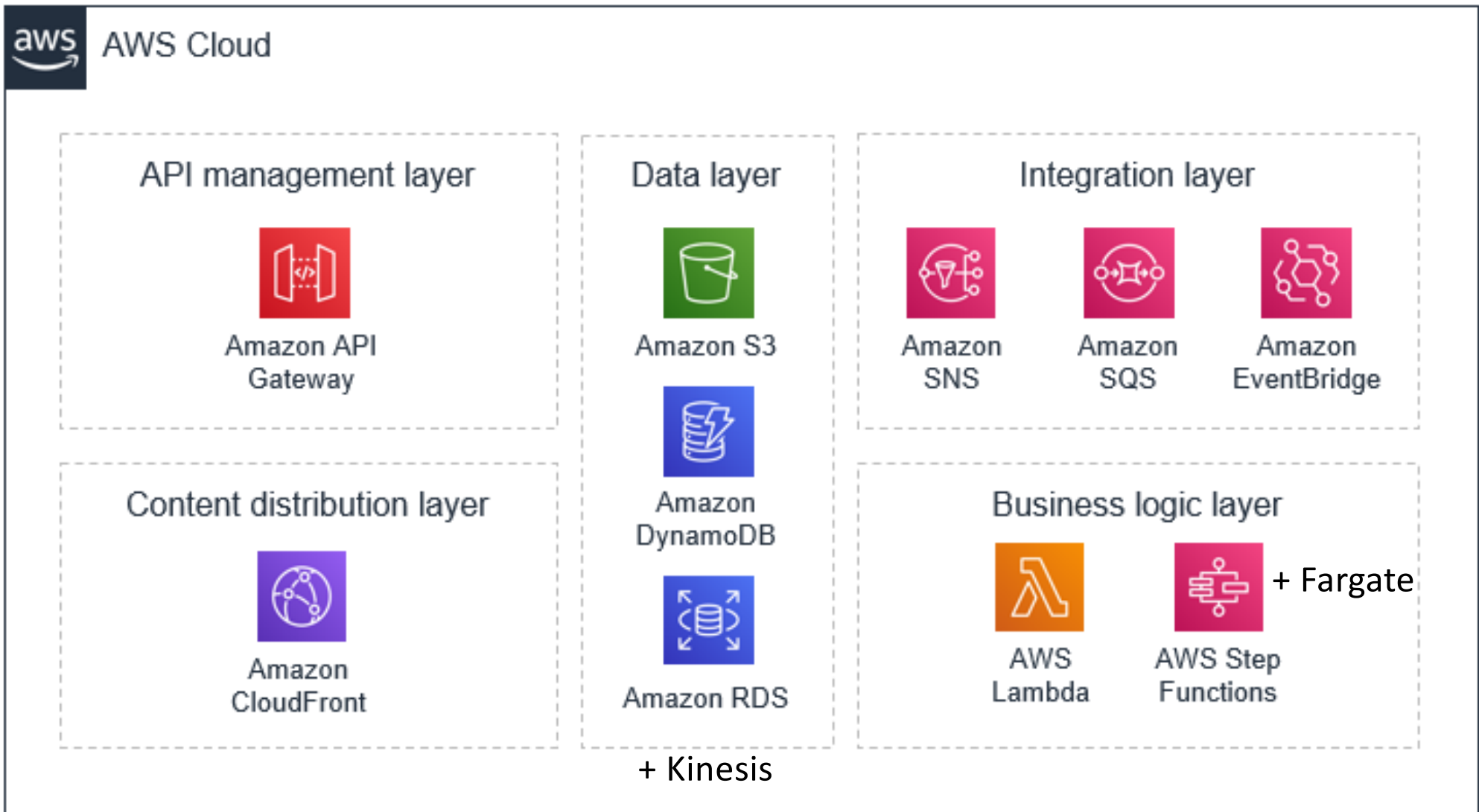
<https://github.com/aws-samples/aws-serverless-workshops>



The background of the slide is a photograph of a bright blue sky filled with several large, white, fluffy cumulus clouds. The clouds are scattered across the frame, with some appearing closer and more detailed, while others are further away and smaller. The overall lighting is bright and clear, suggesting a sunny day.

Serverless Stack: Streaming Analytics

AWS Serverless Platform



<https://docs.aws.amazon.com/lambda/latest/operatorguide/lambda-event-driven-paradigm.html>

Streaming Analytics

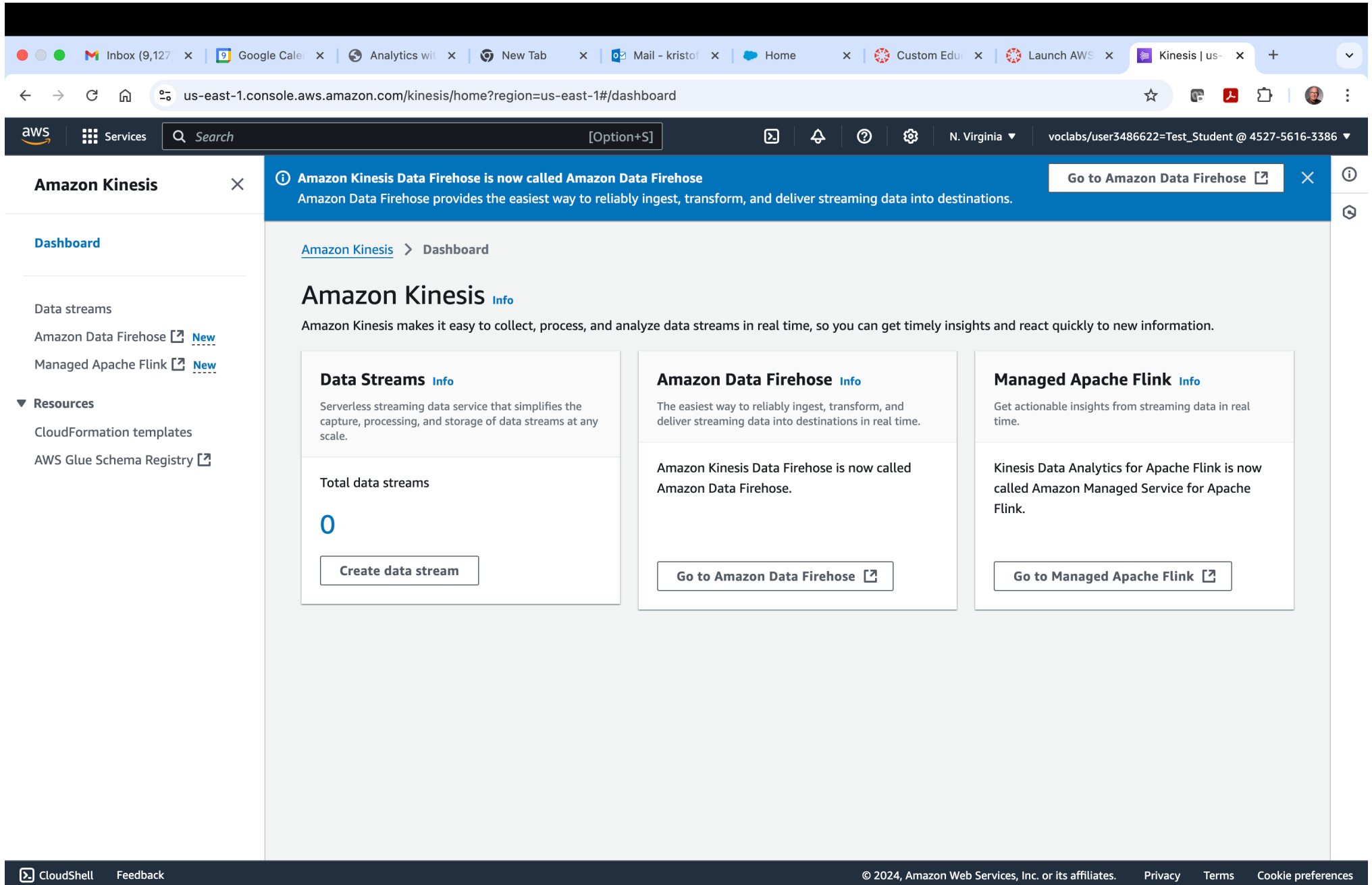
- Introduction to AWS Kinesis
- Serverless Data Processing Workshop
 - <https://data-processing.serverlessworkshops.io/>
 - Simulating an IoT Scenario with devices publishing status data
 - Lambda, Kinesis Data Streams, Kinesis Data Firehose, Glue, S3-Select, Athena

Amazon Kinesis

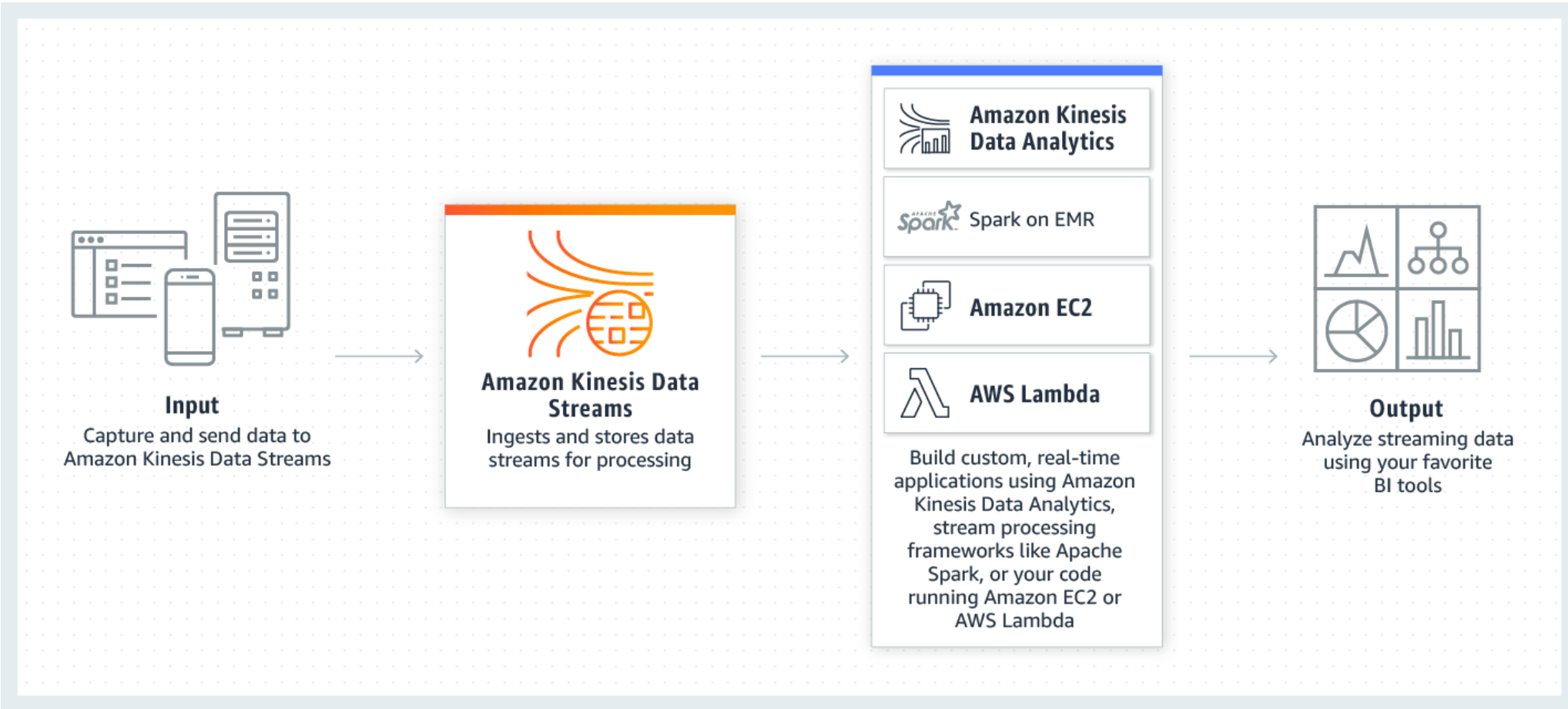
Handling and Analyzing Video and Data Streams

- Managed service, pay-as-you-go plus charge for provisioned capacity
- **Data Streams**
 - Continuously capture and temporarily store real-time data from a large number of sources, for instance IoT devices
 - ‘Shards’, configurable producers and consumers, fan-out
- Kinesis Video Streams
- **Amazon Data Firehose**
 - ‘Elastic’ service to deliver real-time data streams to destinations (like S3)
 - Allows some optional processing & data format conversion
 - Log and event analytics, clickstream analytics, security monitoring
- Apache Flink (succeeding Kinesis Data Analytics)
 - Real-time streaming ETL, analytics

Amazon Kinesis



Kinesis Data Streams



Tutorial:

<https://docs.aws.amazon.com/lambda/latest/dg/with-kinesis-example.html>

Kinesis Data Firehose



AWS Glue

- Serverless data integration
- Used to move and prepare data for analytics
 - Discover and organize data
 - Transform, prepare, and clean data for analysis
 - Build and monitor data pipelines
- Features
 - Data catalog
 - Data integration connectors for various sources
 - Job management & authoring for ETL
 - Apache Spark and Python
 - Codeless visualization

Amazon Athena

- Interactive serverless query service
- Uses Presto with ANSI SQL support
- Supports Jupyter Notebooks for analysis
- Works with a variety of standard data formats, including CSV, JSON, ORC, Avro, and Parquet
- Uses S3 as data store
- No need for ETL
- Targetted towards bulk queries (5\$/1 TeraByte)

Build a Serverless Data-Processing Application

<https://data-processing.serverlessworkshops.io/streaming-data.html>

- Extension of the Wildrydes serverless application scenario to aggregate and analyze sensor data from the ride-sharing (unicorn) fleet
- The project uses
 - [AWS Lambda](#) to process real-time streams,
 - [Amazon DynamoDB](#) to persist records in a NoSQL database,
 - [Amazon Kinesis Data Analytics](#) to aggregate data,
 - [Amazon Data Firehose](#) to archive the raw data to [Amazon S3](#),
 - [Amazon Athena](#) to run ad-hoc queries against the raw data
- Requires using CloudShell or an AMI to run producers and consumers

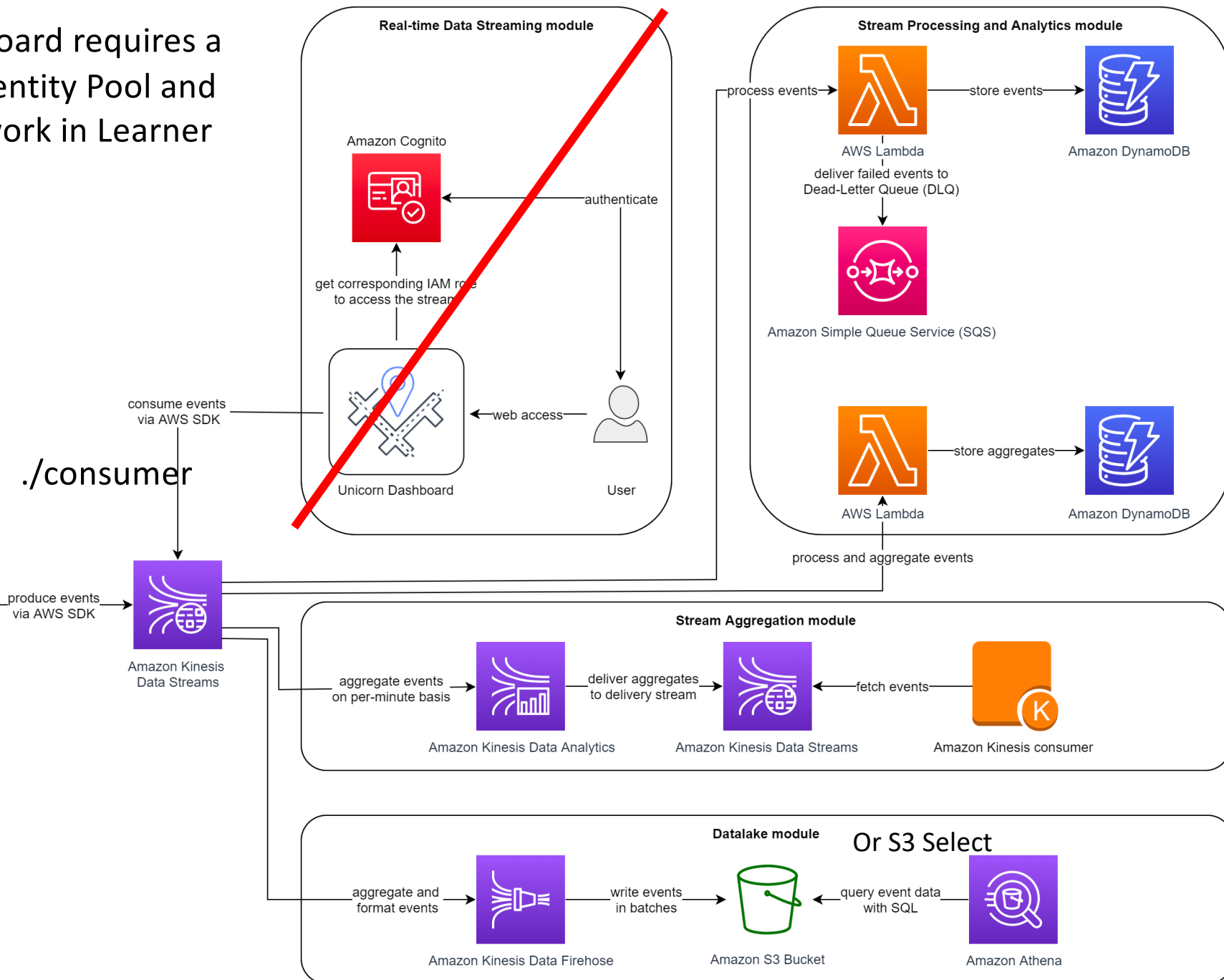
Streaming Analytics Project Architecture

The Dashboard requires a Cognito Identity Pool and does not work in Learner Labs.



Unicorn Fleet using Amazon Kinesis Producer

./producer

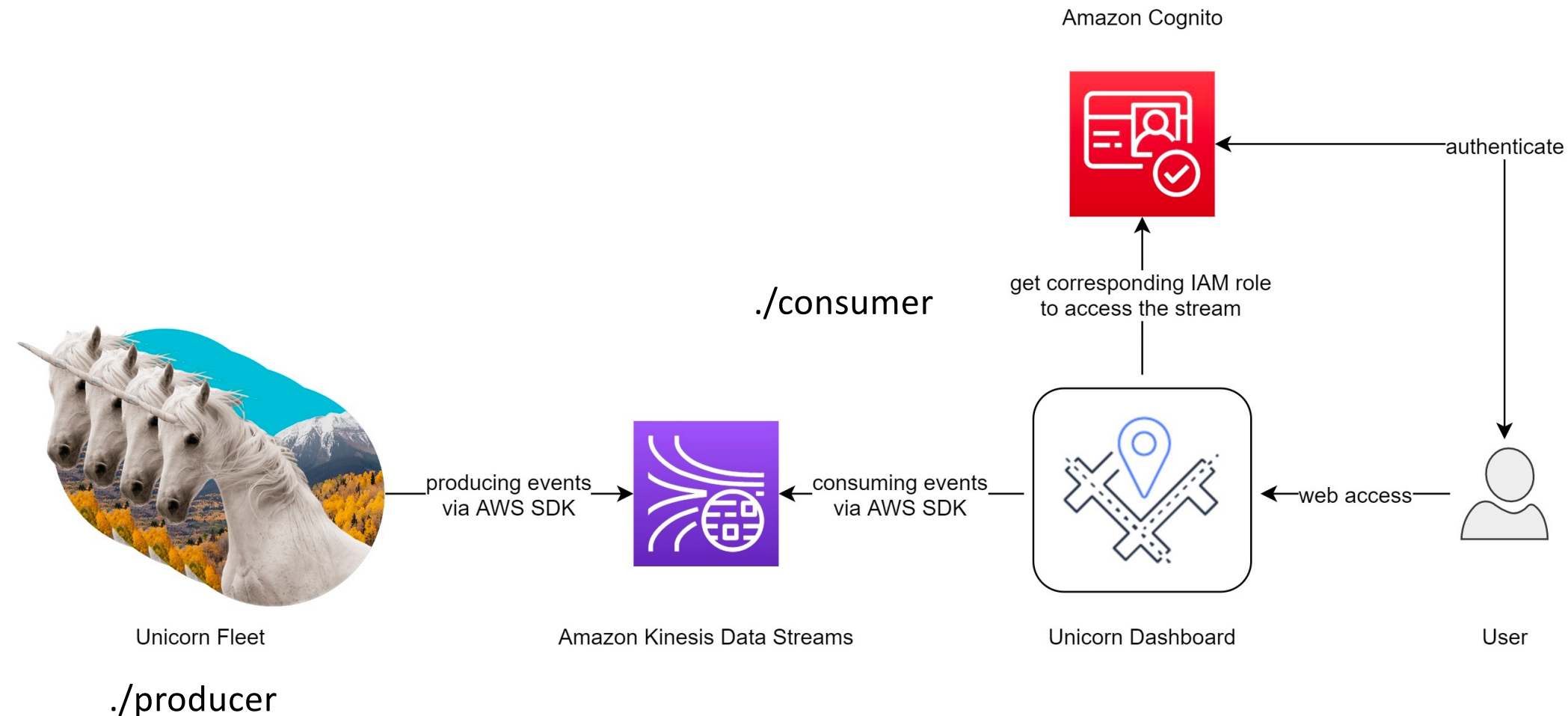


Modules

1. CloudShell (or instance) setup
2. Real time data streaming
3. Stream processing and analytics
4. (Stream aggregation)
5. Data lake

Module 2: Real-time Data Streaming

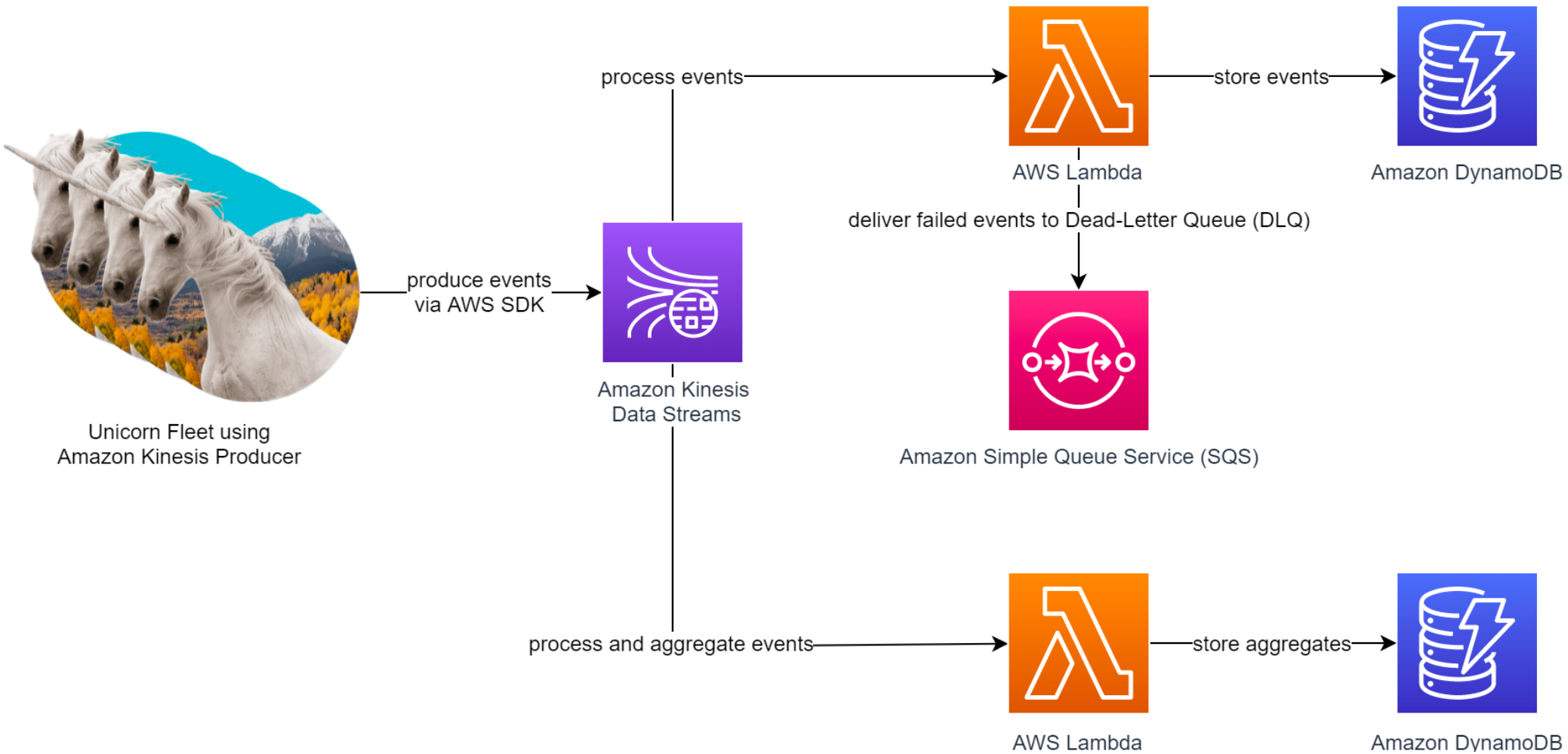
Dashboard does not work in Learner Lab



Steps for Modules 1+2

- Install producer and consumer programs in CloudShell or on an EC2 instance
 - Upload producer and consumer binaries from ILIAS, or manually build go programs
 - In an instance, use root and set your region with 'aws configure'
- Create a Kinesis Stream '**wildrydes**' with one shard in provisioned mode
 - Shard/hr cost is \$0.015
 - On demand mode is \$0.04/hr per stream
 - <https://aws.amazon.com/kinesis/data-streams/pricing/?nc=sn&loc=3>
 - Delete on completion
- In CloudShell (instance), run the producer (./producer)
- In another terminal, run ./consumer
- Click on the monitoring tab in the stream console to see the records arriving
 - GetRecord Success – average, you may have to reduce the viewing window
 - Alternatively, use Data Viewer and 'next records'

Module 3: Stream Processing



Steps for Module 3

- Create a DynamoDB Table **UnicornSensorData**
 - Partition Key: Name, Sort Key: StatusTime
- Create an SQS On-error destination queue **wildrydes-queue**
- **Don't** try to create an IAM role for Lambda, use **LabRole**
- Create Lambda function **WildRydesStreamProcessor**
 - NodeJS 20.X, LabRole
 - Create a .zip file with WildRydesStreamProcessor.js and aws-sdk and deploy
 - Code and zip file WildRydesStreamProcessor-package.zip is in ILIAS
 - Add two environment variables in the Configuration tab
 - TABLE_NAME in **Key** and UnicornSensorData in **Value**.
 - AWS_NODEJS_CONNECTION_REUSE_ENABLED in **Key** and 1 in **Value**.
 - Add the Kinesis stream wildrydes as trigger
 - Additional settings On-failure add ARN of wildrydes queue and make sure the trigger is enabled

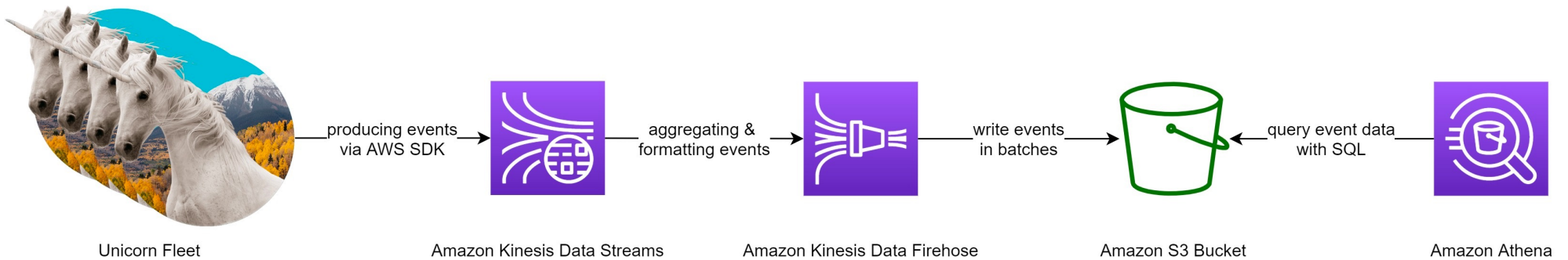
Steps for Module 3...

- Test the setup, from CloudShell
 - ./producer -name Shadowfax -stream wildrydes -msgs 20
- Verify by looking at Lambda monitor and logs
- Query the DynamoDB table
- For error handling, see
 - <https://data-processing.serverlessworkshops.io/stream-processing/03-error-handling.html>
 - This scenario injects a poison pill message

Steps for Module 3...

- For **tumbling window** (aggregation), see
 - <https://data-processing.serverlessworkshops.io/stream-processing/04-analytics-with-tumbling-window.html>
 - Tumbling windows are distinct time windows that open and close at regular intervals. By default, Lambda invocations are stateless—you cannot use them for processing data across multiple continuous invocations without an external database. However, with tumbling windows, you can maintain your state across invocations.
 - Create UnicornAggregation DynamoDB table (name, windowStart)
 - Create Lambda function WildRydesAggregator
 - Code is ILIAS, use the package including aws-sdk
 - LabRole
 - Environment variable (TABLE_NAME, UnicornAggregation)
 - Kinesis Stream wildrydes as trigger
 - Additional trigger settings:
 - Retry Attempts 2, Max Age of Record -1
 - Split Batch on Error, Tumbling Window 60
 - Test with ./producer and verify through logs and DynamoDB

Module 5: Data Lake



Or S3 Select

Steps for Module 5

- Create an **S3 bucket** *wildrydes-data-yourname*
- In AWS Glue, create a **database** “default”
- Create an AWS Glue **table** *wildrydes_athena*
 - Type S3 include path *s3:// wildrydes-data-yourname/*
 - **Format = Parquet**
 - Add columns
 - *name* (string)
 - *statustime* (timestamp)
 - *latitude* (float)
 - *longitude* (float)
 - *distance* (float)
 - *magicpoints* (int)
 - *healthpoints* (int)

Steps for Module 5 ...

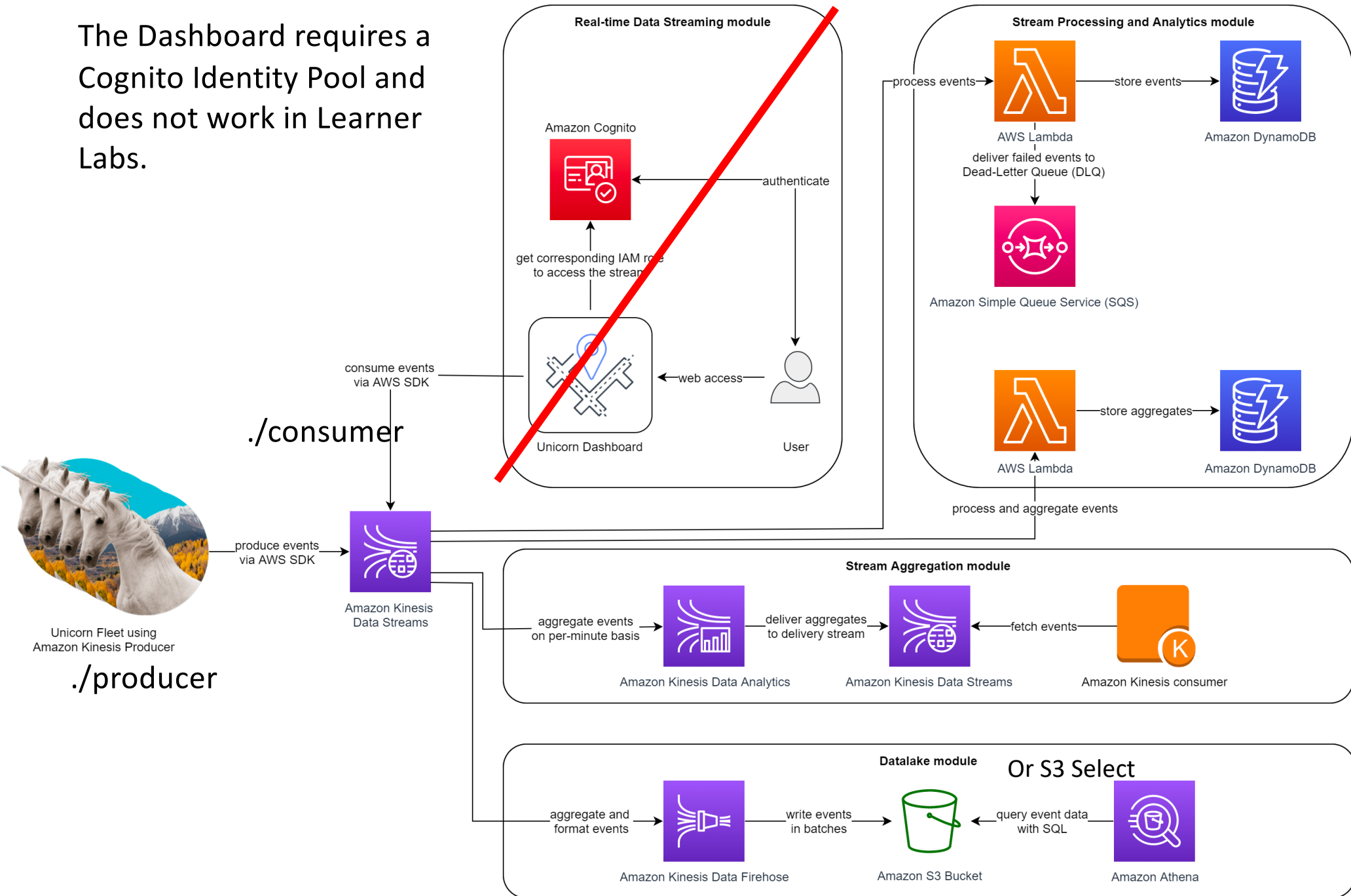
- Create an **Amazon Data Firehose** (delivery stream)
 - Name: wildrydes
 - **Source:** your Kinesis Data Stream
 - Enable record **formation conversion** (Parquet)
 - Choose your Glue table (wildrydes_athena)
 - **Destination:** your S3 bucket wildrydes-data-yourname
 - Buffer Interval: 60 seconds (under Buffer Hints)
 - Choose to use LabRole (Advanced Settings, Permissions)
- Test the setup
 - Navigate to an object in wildrydes-data, chose 'Query with S3 Select'
 - Input Format Parquet
 - Output Settings CSV and Comma
 - Run SQL Query

Module 5: Using Athena

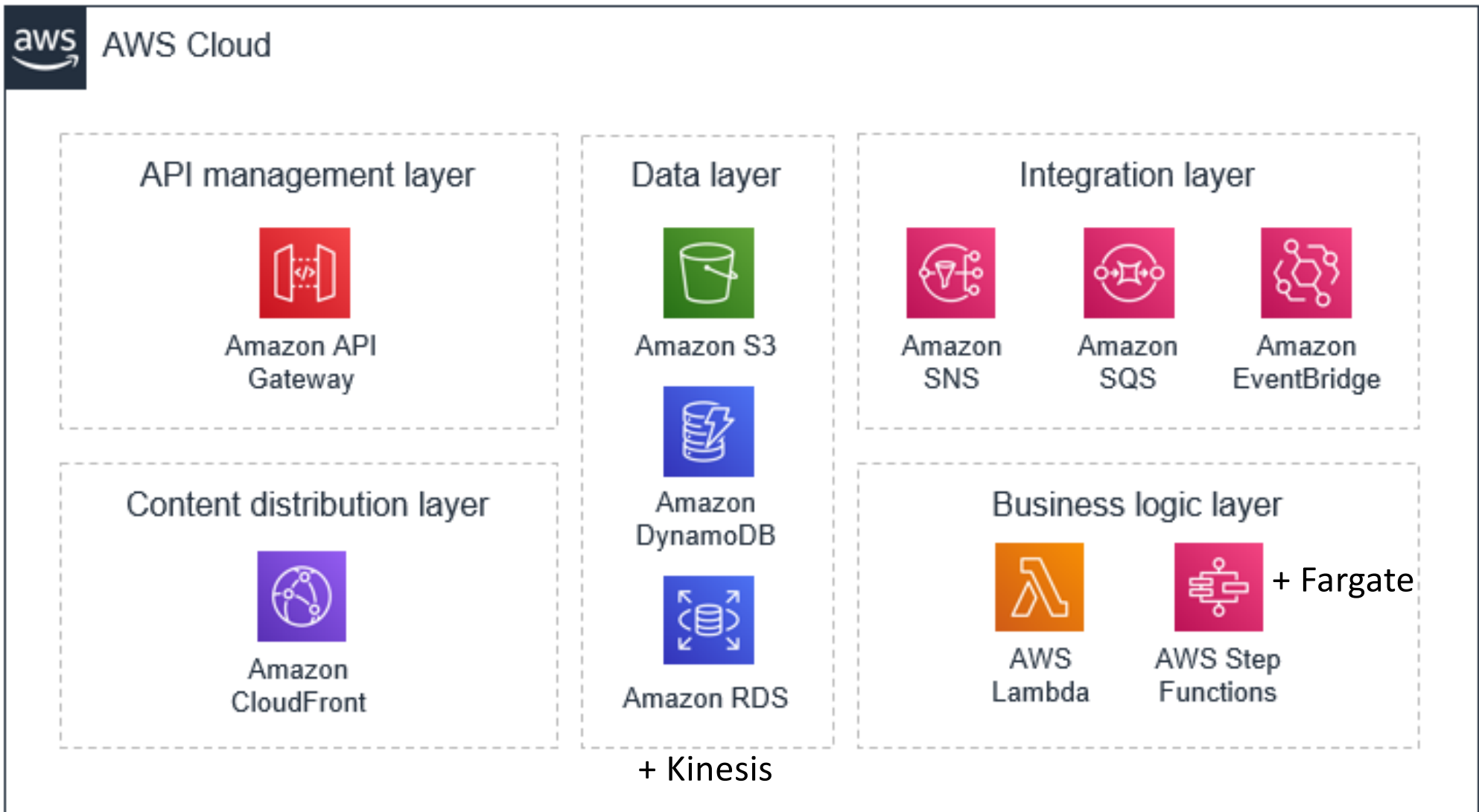
- Create an output bucket athena-query-output-yourname
- Navigate to **Athena Query Editor**
- In settings, select the bucket you just created
- Select database (default)
- Enter a query, e.g.
Select * from wildrydes_athena

Summary: Kinesis, Lambda, DynamoDB, Glue, Athena

The Dashboard requires a Cognito Identity Pool and does not work in Learner Labs.



AWS Serverless Platform

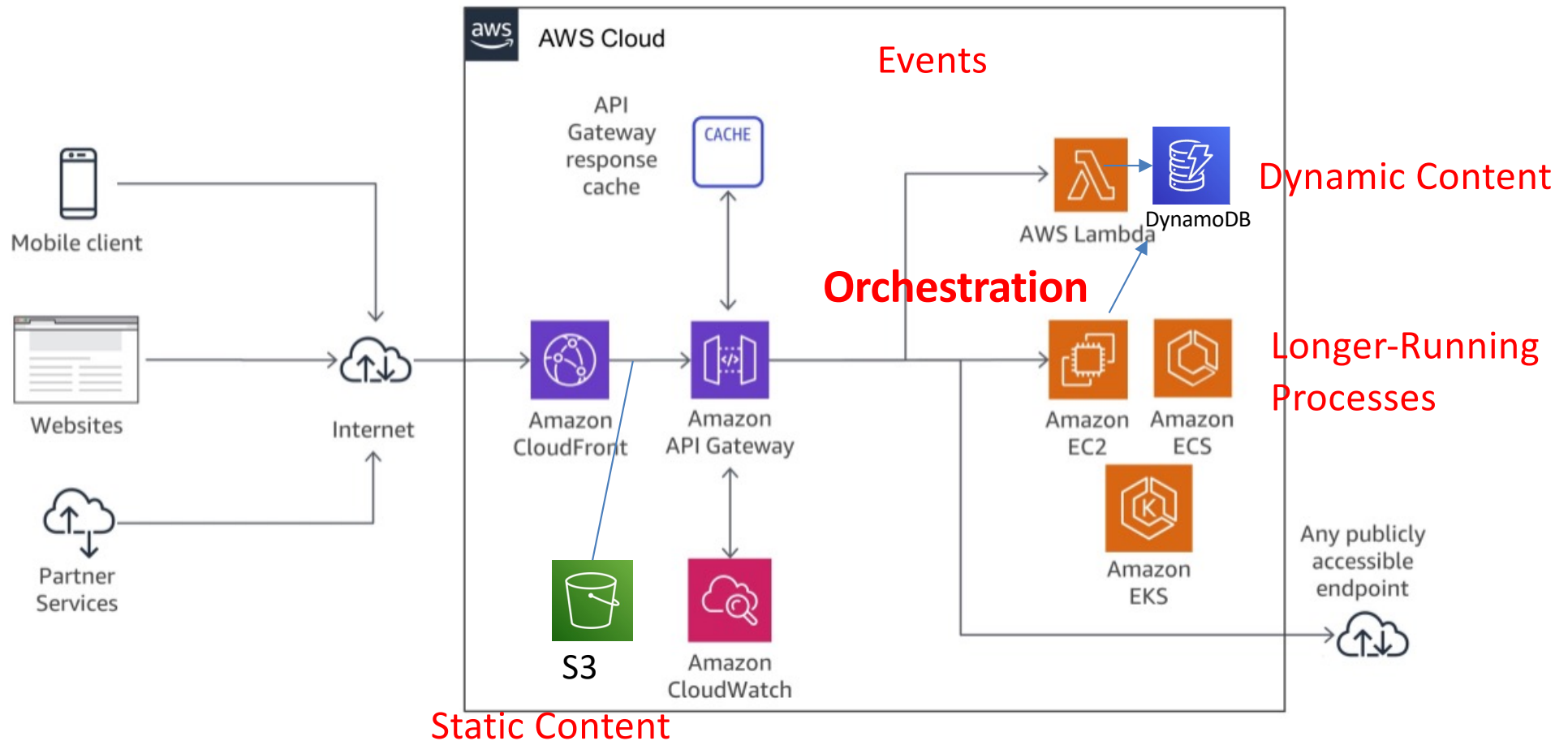


<https://docs.aws.amazon.com/lambda/latest/operatorguide/lambda-event-driven-paradigm.html>

Module 11: More Integration



Basic Structure of a Modern Application in AWS



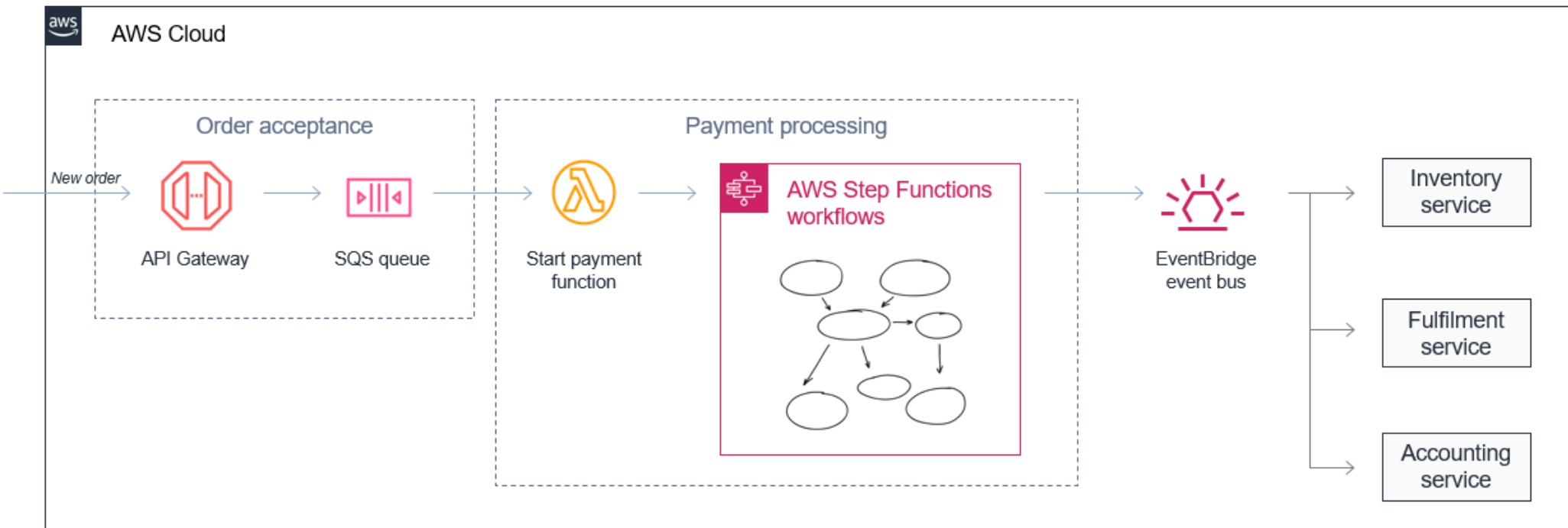
Source: Implementing Microservices in AWS

More Serverless Integration

- Step Functions
 - State Machines, Workflows
- EventBridge
 - Event Bus, Pipes

Assembling Microservices

Example: e-Commerce Flow

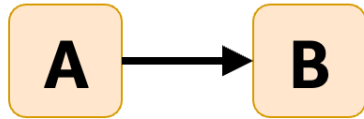


<https://docs.aws.amazon.com/lambda/latest/dg/concepts-event-driven-architectures.html>

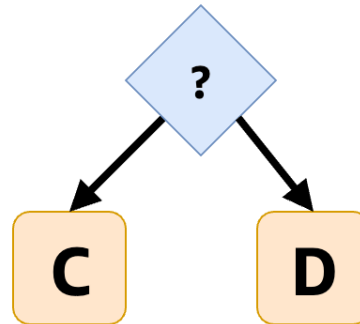
Step Functions

- Serverless function **orchestrator**
- Sequencing AWS Lambda functions and multiple AWS services into business-critical applications
- Create and run a series of **checkpointed and event-driven workflows** that maintain the application state
- Express (less than 5 minutes) and Standard workflow options
- Proprietary state machine language
- Visual interface
 - Define workflow in Studio, code is auto-generated
 - Alternatively, use VS Code (with snippet library)
 - Select Services and execute
- Priced per state transition
- <https://aws.amazon.com/step-functions/use-cases/>
- <https://docs.aws.amazon.com/step-functions/latest/dg/learning-resources.html#tutorials>

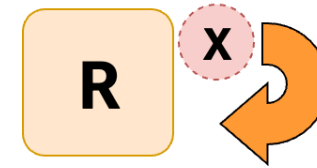
1 Orchestrate tasks



2 Choose tasks based on data

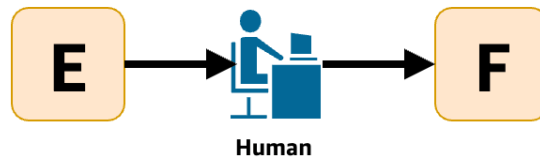


3 Retry tasks

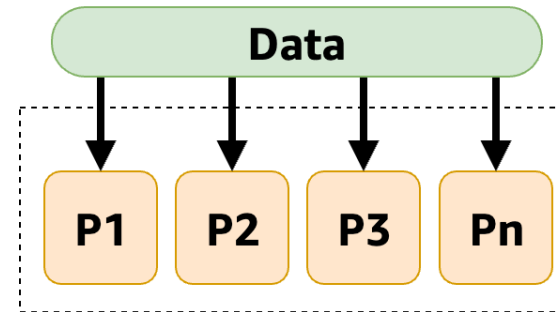


Use Cases

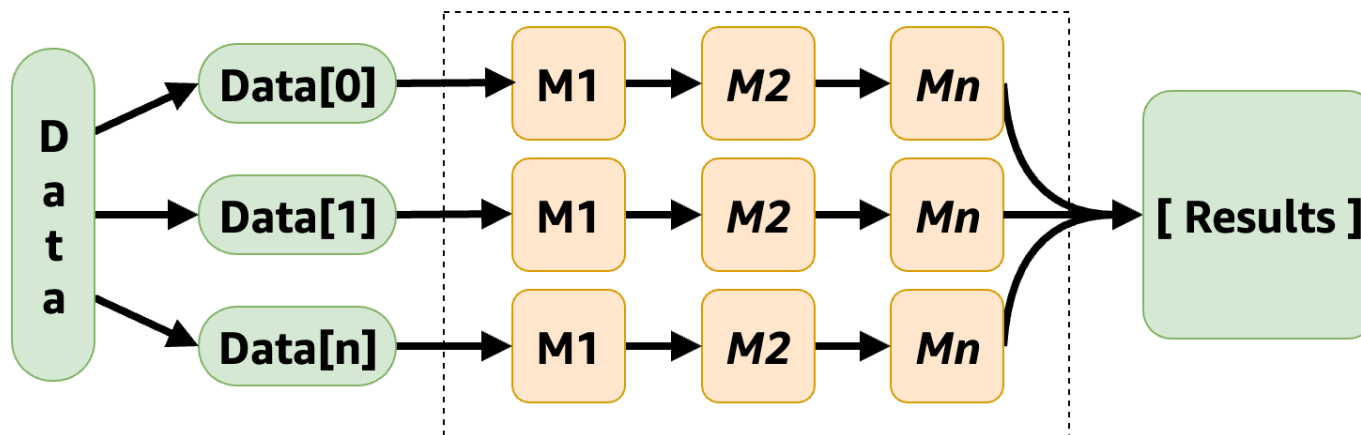
4 Add a human in the loop



5 Process data in parallel



6 Dynamically process data elements with a map



Source: AWS

Step Function Designer

The screenshot displays the AWS Step Function Designer interface in a web browser. The browser's address bar shows the URL: `us-east-1.console.aws.amazon.com/states/home?region=us-east-1#/v2/statemachines/getStarted`. The interface includes a top navigation bar with tabs for 'Design', 'Code', and 'Config'. Below this, a toolbar offers actions like 'Undo', 'Redo', 'Zoom in', 'Zoom out', 'Center', 'Duplicate', and 'Delete'. On the left, a sidebar lists 'MOST POPULAR' services (AWS Lambda, Amazon SNS, Amazon ECS, AWS Step Functions, AWS Glue) and a 'THIRD-PARTY API' section. The main workspace shows a 'Workflow Studio State Machine Graph' with the following steps: 1. 'Pass state Pass' (Start), 2. 'Choice state Hello World example?' (Decision), 3. 'Pass state Yes' (if `$$.isHelloWorldExample == true`), 4. 'Wait state Wait 3 sec' (if Yes), 5. 'Parallel state Parallel State' (if Yes), which branches into 'Pass state Hello' and 'Pass state World', 6. 'Fail state No' (if `$$.isHelloWorldExample == false`), and 7. 'Pass state Hello World' (if Default). All paths converge at an 'End' state. On the right, an 'Inspector' panel displays a 'Run Hello World' message with a 'Start' button. The bottom of the interface features a 'CloudShell' button, a 'Feedback' link, and a footer with copyright information for Amazon Web Services, Inc. (© 2023).

State Machine Configuration

The screenshot shows the AWS IAM console interface for configuring a State Machine. The browser address bar indicates the URL is `us-east-1.console.aws.amazon.com/states/home?region=us-east-1#/v2/statemachines/getStarted`. The console header shows the user is logged in as `voclabs/user2751656=Test_Student @ 4256-7131-6302` in the `N. Virginia` region.

The main configuration area is titled "State machine configuration" and includes the following sections:

- State machine name:** The name `MyStateMachine-f1jqh1umg` is entered in the text field. A note states: "State machine name cannot be changed after creation. Must be 1-80 characters. Can use alphanumeric characters, dashes, and underscores."
- Type:** Two options are available:
 - ☒ **Standard**: Durable workflows for ETL, ML, e-commerce and automation. They can run for up to 1 year, and history is stored in Step Functions for auditing and playback. Supported by a feature-rich console debugger. Recommended for new users.
 - ☐ **Express**: Low cost, high scale workflows for streaming data processing and microservice APIs. They can run for up to 5 minutes, and history can be streamed to CloudWatch Logs.
- Permissions:**
 - Execution role:** A dropdown menu shows `LabRole` is selected. A note states: "The IAM role that defines which resources your state machine has permission to access during execution. To create a custom role, go to the [IAM console](#)". A red arrow points to this dropdown.
 - A `View in IAM` button is available next to the dropdown.
- Logging:** This section is partially visible at the bottom of the configuration area.

At the bottom of the console, there are links for `CloudShell`, `Feedback`, and a footer with copyright information: "© 2023, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences".

State Machine Code

us-east-1.console.aws.amazon.com/states/home?region=us-east-1#/v2/statemachines/getStarted

MyStateMachine-f1jqh1umg

Design Code Config

Cancel Actions Create

Undo Redo Format Copy Commands View docs

```
1 {
2   "Comment": "A Hello World example demonstrating various state types of the",
3   "StartAt": "Pass",
4   "States": {
5     "Pass": {
6       "Comment": "A Pass state passes its input to its output, without performing any",
7       "Type": "Pass",
8       "Result": {
9         "IsHelloWorldExample": true
10      },
11      "Next": "Hello World example?",
12    },
13    "Hello World example?": {
14      "Comment": "A Choice state adds branching logic to a state machine. Choose the",
15      "Type": "Choice",
16      "Choices": [
17        {
18          "Variable": "$.IsHelloWorldExample",
19          "BooleanEquals": true,
20          "Next": "Yes"
21        },
22        {
23          "Variable": "$.IsHelloWorldExample",
24          "BooleanEquals": false,
25          "Next": "No"
26        }
27      ],
28      "Default": "Yes"
29    },
30    "Yes": {
31      "Type": "Pass",
32      "Next": "Wait 3 sec"
```

Workflow Studio State Machine Graph

```
graph TD
    Start((Start)) --> Pass1[Pass state Pass]
    Pass1 --> Choice[Choice state Hello World example?]
    Choice -- "$.IsHelloWorldExample == false" --> Fail[Fail state No]
    Choice -- "$.IsHelloWorldExample == true" --> Pass2[Pass state Yes]
    Choice -- Default --> Pass2
    Pass2 --> Wait[Wait state Wait 3 sec]
    Wait --> Parallel[Parallel state Parallel State]
    Parallel --> Pass3[Pass state Hello]
    Parallel --> Pass4[Pass state World]
    Pass3 --> End((End))
    Pass4 --> Pass5[Pass state Hello World]
    Pass5 --> End
```

CloudShell Feedback

© 2023, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Step Functions Tutorial with Lambda Task Implementations

- Build a Step Functions state machine to simulate a simple call center workflow
 - Use the LabRole (go to "config" first)
 - Use CallCenterStateMachine as name, with json function in ILIAS
- Create 5 Lambda functions, using LabRole
 - OpenCaseFunction, AssignCaseFunction, WorkOnCaseFunction, EscalateCaseFunction, CloseCaseFunction
 - Use x26.py functions in ILIAS
- Populate the state machine json file with the ARNs of the Lambda functions

Step Function Development

us-east-1.console.aws.amazon.com/states/home?region=us-east-1#/statemachines/edit/arn:aws:states:us-east-1:123456789012:stateMachine:my-state-machine

Definition

Define your workflow using [Amazon States Language](#). Test your data flow with the new [Data Flow Simulator](#).

Generate code snippet

Format JSON

```
1 {
2   "Comment": "A simple AWS Step Functions state machine that
3   automates a call center support session.",
4   "StartAt": "Open Case",
5   "States": {
6     "Open Case": {
7       "Type": "Task",
8       "Resource": "arn:aws:lambda:us-east-
9       1:877950311269:function:OpenCaseFunction:$LATEST",
10      "Next": "Assign Case"
11    },
12    "Assign Case": {
13      "Type": "Task",
14      "Resource": "arn:aws:lambda:us-east-
15      1:877950311269:function:AssignCaseFunction:$LATEST",
16      "Next": "Work on Case"
17    },
18    "Work on Case": {
19      "Type": "Task",
20      "Resource": "arn:aws:lambda:us-east-
21      1:877950311269:function:WorkOnCaseFunction:$LATEST",
22      "Next": "Is Case Resolved"
23    },
24    "Is Case Resolved": {
25      "Type": "Decision",
26      "Resource": "arn:aws:lambda:us-east-
27      1:877950311269:function:IsCaseResolvedFunction:$LATEST",
28      "Choices": [
29        {
30          "Variable": "$?.Resolved",
31          "Boolean": "True",
32          "Next": "Close Case"
33        },
34        {
35          "Variable": "$?.Resolved",
36          "Boolean": "False",
37          "Next": "Escalate Case"
38        }
39      ],
40      "Default": "Escalate Case"
41    },
42    "Escalate Case": {
43      "Type": "Task",
44      "Resource": "arn:aws:lambda:us-east-
45      1:877950311269:function:EscalateCaseFunction:$LATEST",
46      "Next": "Fail"
47    },
48    "Close Case": {
49      "Type": "Task",
50      "Resource": "arn:aws:lambda:us-east-
51      1:877950311269:function:CloseCaseFunction:$LATEST",
52      "Next": "End"
53    },
54    "Fail": {
55      "Type": "Task",
56      "Resource": "arn:aws:lambda:us-east-
57      1:877950311269:function:FailFunction:$LATEST",
58      "Next": "End"
59    },
60    "End": {
61      "Type": "End",
62      "Resource": ""
63    }
64  }
65 }
```

Workflow Studio New

```
graph TD
    Start((Start)) --> OpenCase[Open Case]
    OpenCase --> AssignCase[Assign Case]
    AssignCase --> WorkOnCase[Work on Case]
    WorkOnCase --> IsCaseResolved[Is Case Resolved]
    IsCaseResolved -- Resolved --> CloseCase[Close Case]
    IsCaseResolved -- Not Resolved --> EscalateCase[Escalate Case]
    CloseCase --> Fail[Fail]
    EscalateCase --> Fail
    Fail --> End((End))
```

Step Functions Tutorial...

<https://aws.amazon.com/getting-started/hands-on/create-a-serverless-workflow-step-functions-lambda/>

- Execute the workflow by injecting a case (event)
{
 : "001"
}
- This event is passed to the Lambda function that gets invoked

Step Function Execution

us-east-1.console.aws.amazon.com/states/home?region=us-east-1#/v2/executions/details/arn:aws:states:us-east-1:877950311269:executi...

Fail state executed in step: Fail
▶ Cause [Show step detail](#)

Graph view **Table view**

Graph view [Data flow simulator](#) [Export](#) [Layout](#)

```
graph TD; Start((Start)) --> OpenCase[Open Case]; OpenCase --> AssignCase[Assign Case]; AssignCase --> WorkOnCase[Work on Case]; WorkOnCase --> IsCaseResolved{Is Case Resolved}; IsCaseResolved --> CloseCase[Close Case]; IsCaseResolved --> EscalateCase[Escalate Case]; CloseCase --> End((End)); EscalateCase --> Fail[Fail]; Fail --> End;
```

Fail

[Input and output](#) [Details](#) [Definition](#) [Events](#)

☐ Advanced view [Collapse all](#) [Expand all](#)

▼ **Input** [Learn more](#)

```
1 {
2   "Case": "002",
3   "Status": 0,
4   "Message": "Case 002:
5   opened...assigned...unresolved...escalating."
}
```

Reason Fail state executed

```
1 Engage Tier 2 Support.
```

☐ In progress ☒ Failed ☐ Caught error ☐ Canceled ☒ Succeeded

Example: Serverless Image Processing

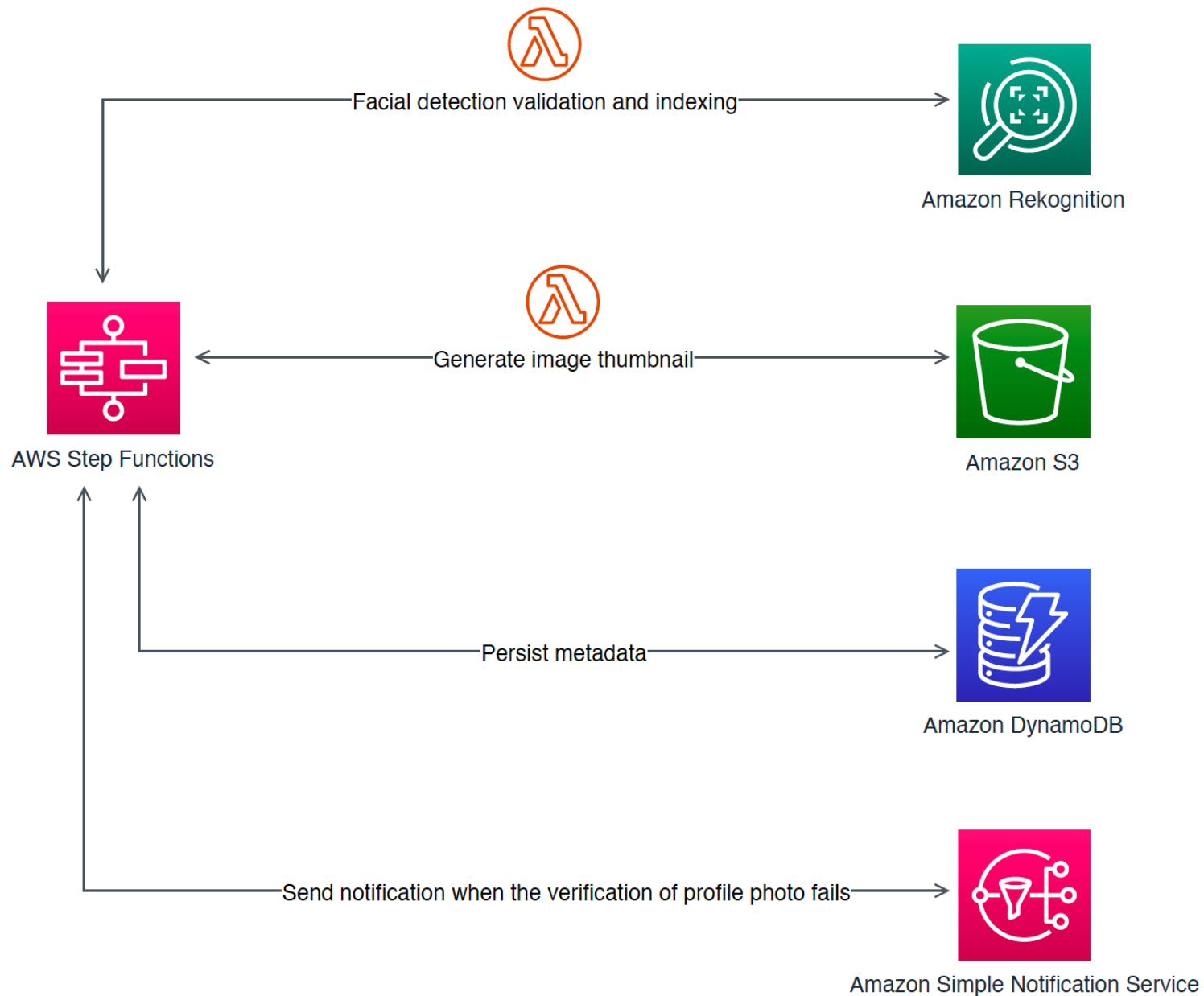
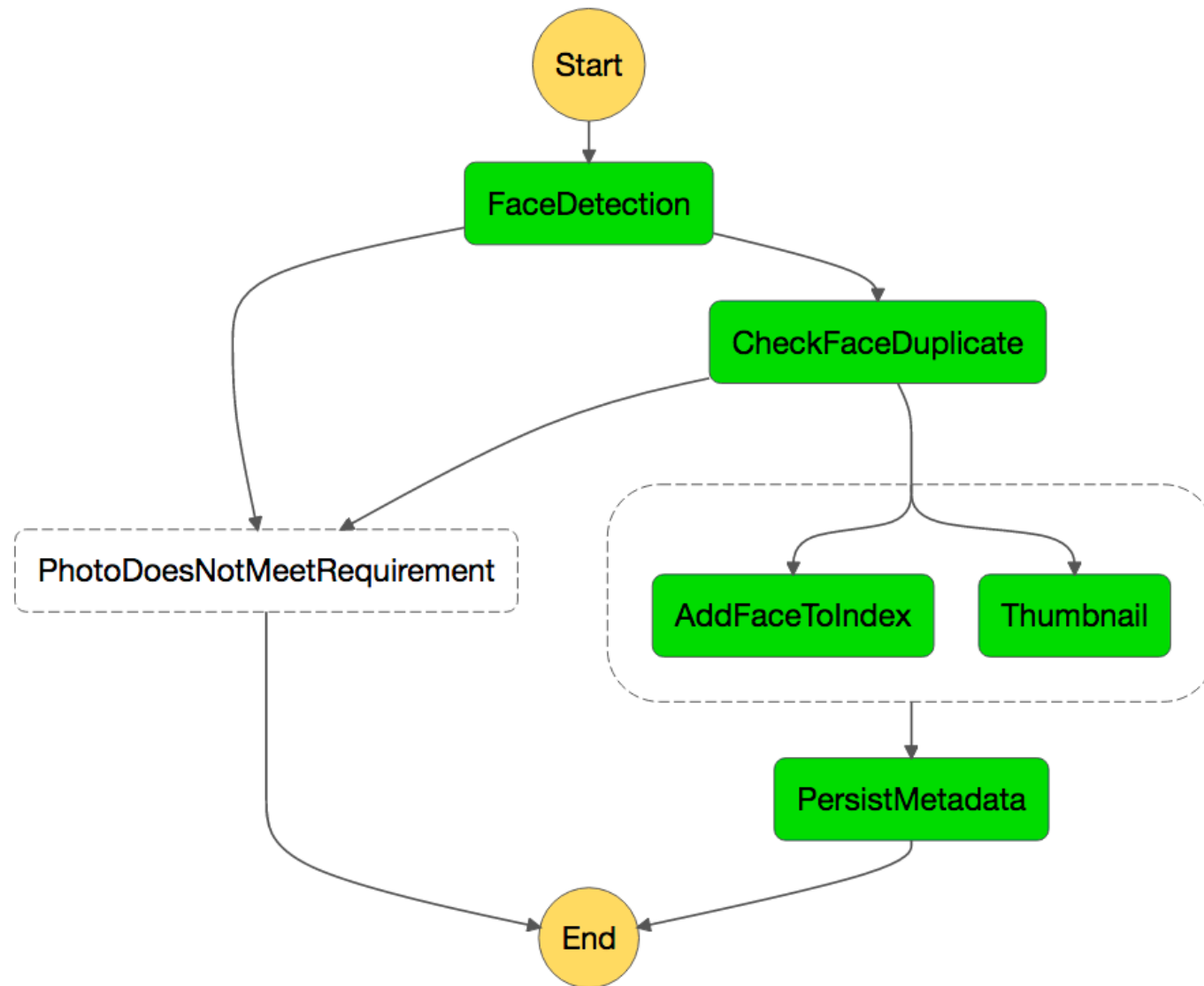


Image Processing Workflow

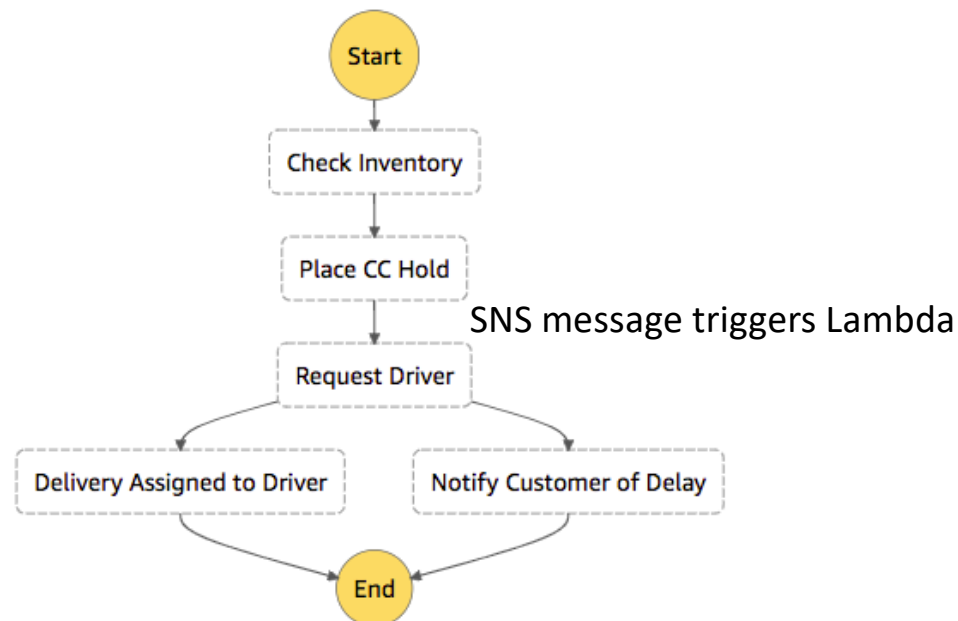


Step Function and SNS

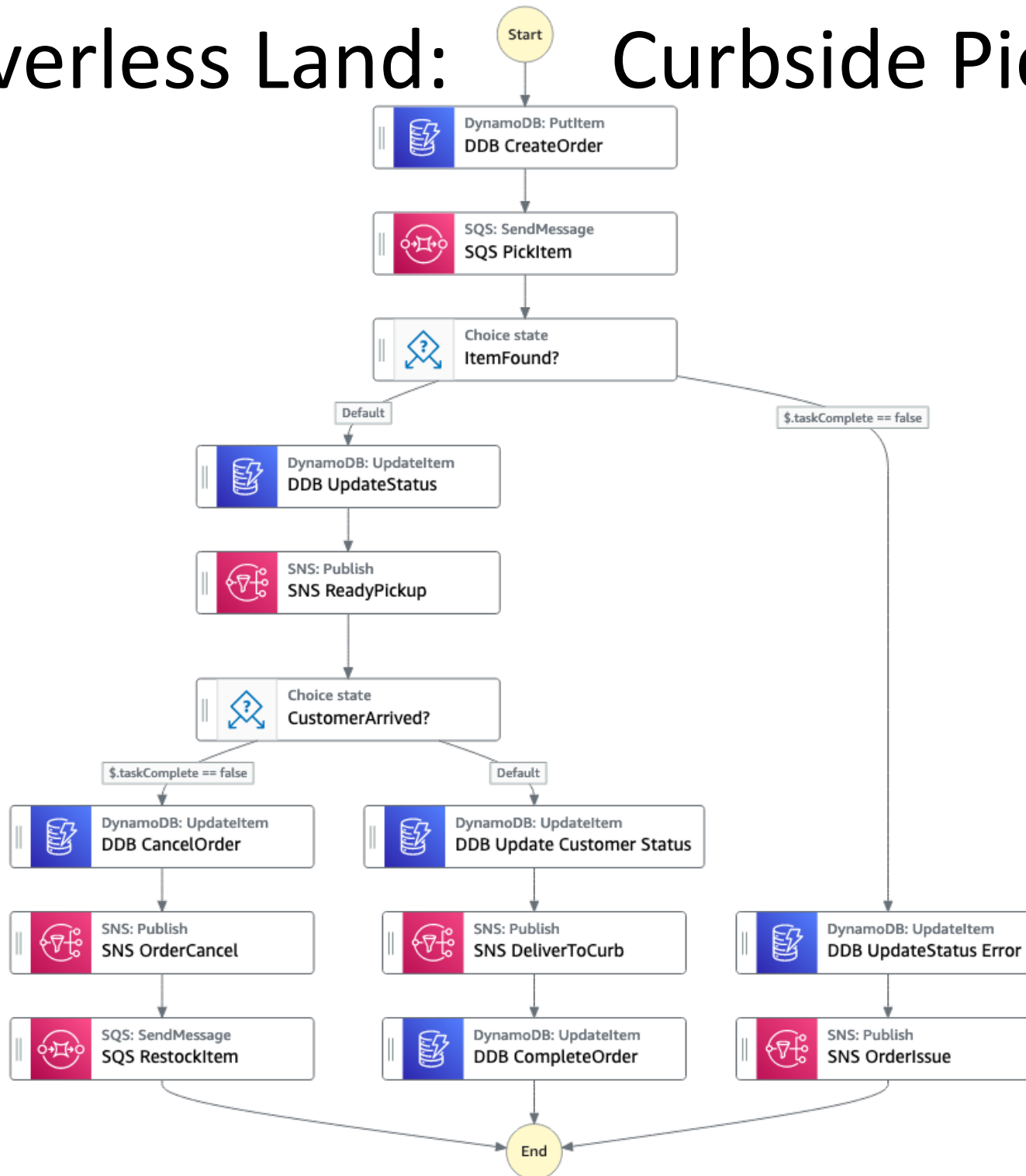
<https://docs.aws.amazon.com/step-functions/latest/dg/tutorials.html>

- Simulating a delivery service
- Code is in ILIAS, the doc describes the general mechanism
- A driver is requested via a SNS DeliveryRequest topic
 - Driver pool represented by Lambda functions subscribed to topic, e.g. driverSally and driverAlex, with labRole and NodeJs 18.X, subscribed to the topic. Use driverSally.zip file in ILIAS, modify name in index.js
 - Driver supervisor notified by email
- Delivery workflow through Step Function ProcessNewDelivery (with DeliveryRequest topic ARN)

`{"OrderNumber": "123"}`



Serverless Land: Curbside Pickup



Step Functions Activities

- Activities are an AWS Step Functions feature connecting tasks to workers
- Workers can be hosted in EC2, ECS and other execution environments and are (long-running) functions polling for tasks from a step function
- <https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-creating-activity-state-machine.html>
 - Create an activity get-greeting
 - Build a simple step function (getGreeting.json), replace the sample arn with your activity arn.
 - To avoid Java setup, use simple Python worker and replace the activity ARN with your own in the code
 - For Python sample worker <https://pypi.org/project/stepfunctions-activity-worker/>
 - Run 'pip install stepfunctions_activity_worker' in CloudShell or an AMI
 - Run the worker in CloudShell or an AMI
 - The worker will poll the step function
 - Execute the step function with

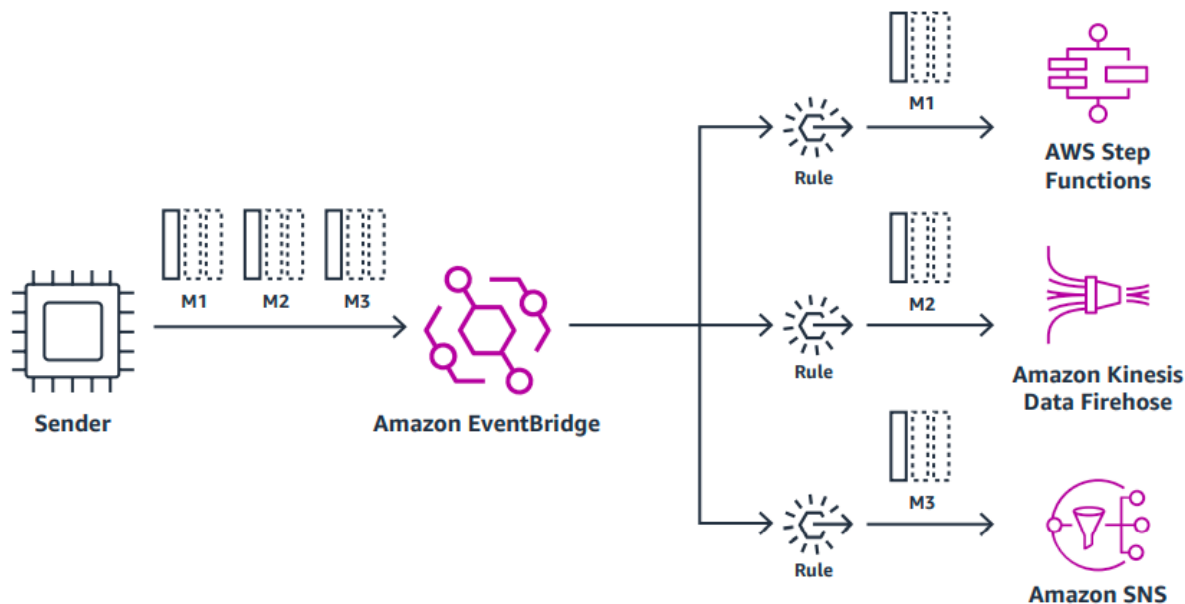
```
{“who”: “Alan Turing”}
```

More Serverless Integration

- Step Functions
- EventBridge

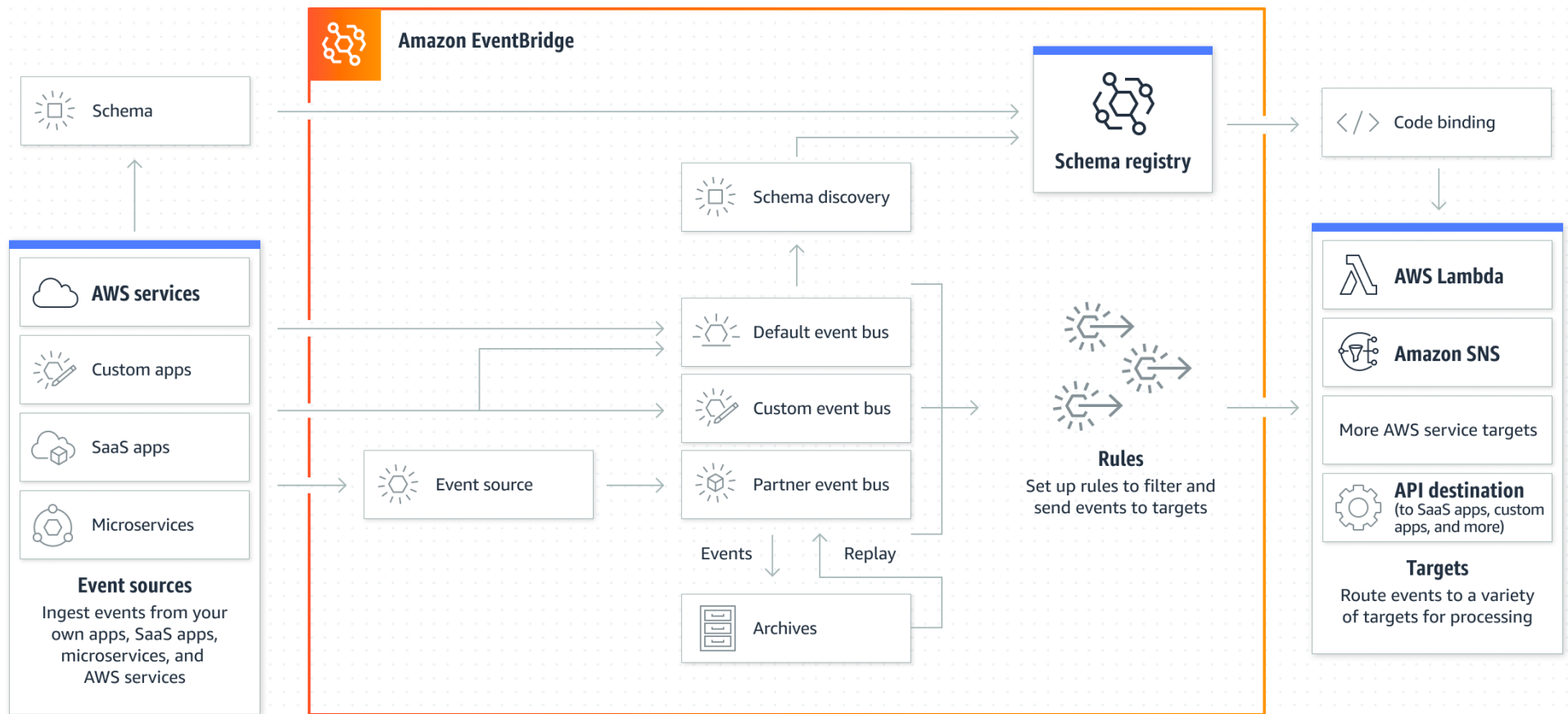
Amazon EventBridge

- Serverless event processing
 - <https://serverlessland.com/event-driven-architecture/intro>
- **Event bus**, enabling loosely coupled event-based architectures across AWS services and external SaaS services
- **Pipes**, point-to-point connections between wide range of producers and consumers with filtering, transformation etc.
- Built for high volume
- State events published by AWS services are free



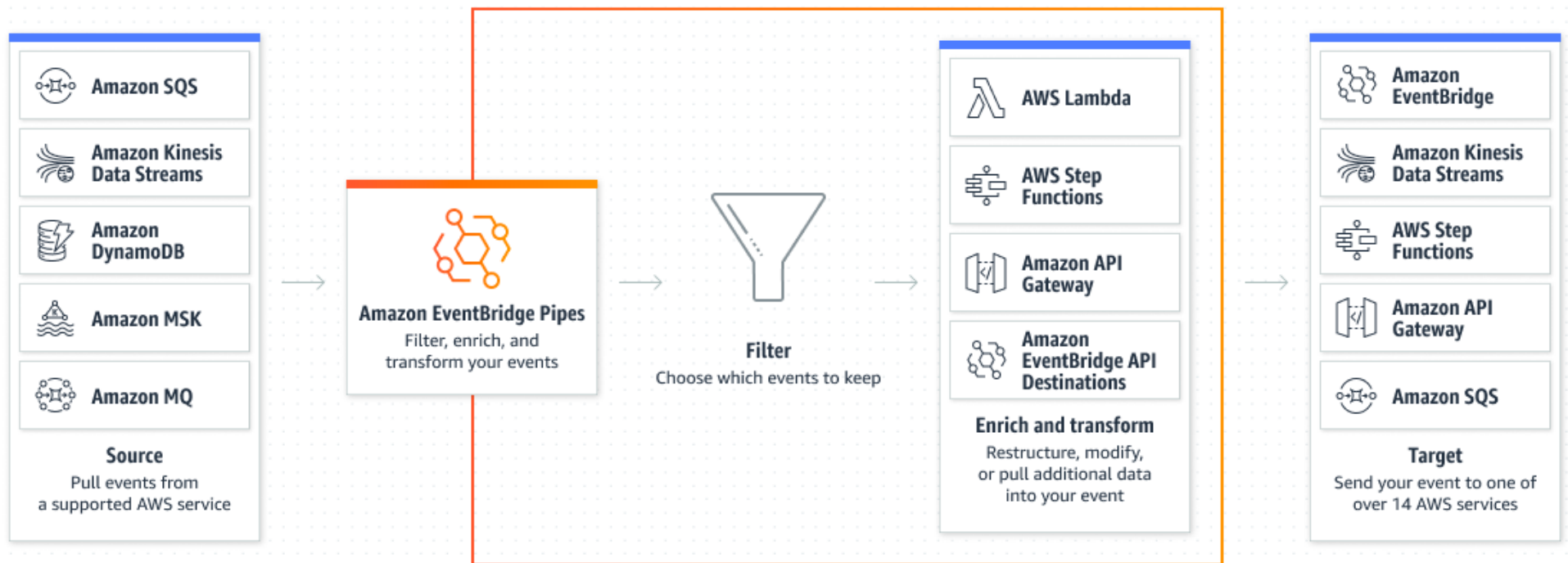
EventBridge Bus

Receive, Filter, Transform, Route, Deliver Events



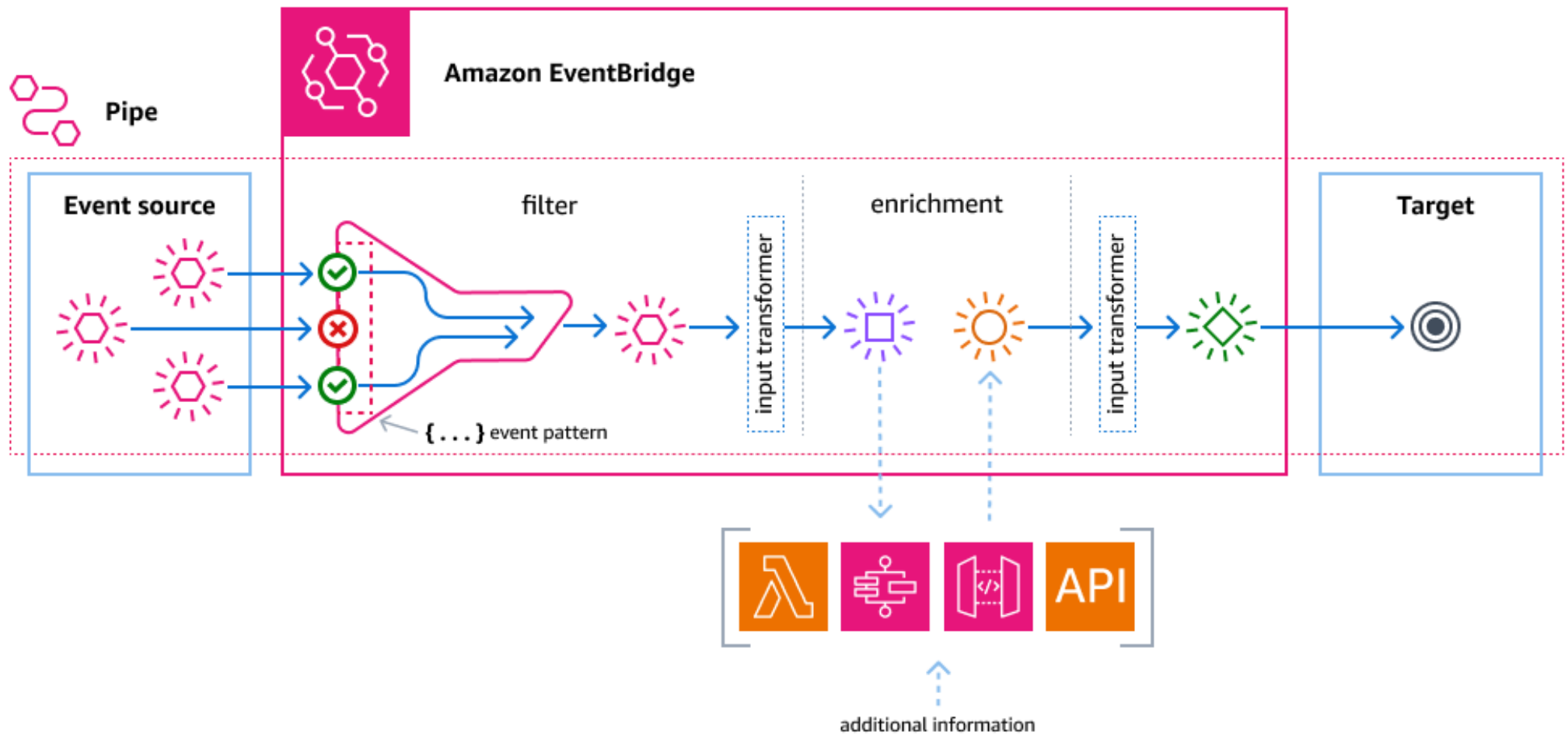
EventBridge Pipes

Connect event producers to event consumers with optional filtering, enrichment, and transformation capabilities.



<https://serverlessland.com/patterns?services=eventbridge-pipes>

EventBridge Pipe Concepts



Building a Pipe

- Name the pipe, and choose LabRole in settings
- Select a source, e.g. an SQS queue
- Select a target, e.g. an SNS topic
- Optionally, create a filter, e.g. by choosing a pattern

```
{  
  "body": {  
    "City": ["Seattle"]  
  }  
}
```

- Test the pipe

Filtering with Pipes

<https://repost.aws/knowledge-center/eventbridge-filter-events-with-pipes>

```
{  
  "messageAttributes": {  
    "priority": {  
      "stringValue": ["high", "urgent"]  
    },  
    "category": {  
      "stringValue": ["notification"]  
    }  
  }  
}
```

Schemas

Next

EventBridge Scheduler



Create schedule

Create schedule to get started



Schedule pattern

Use cron expression, fixed rate, time window, one-time, or specific date and time to set your schedule



Target

Select a target from over 200 AWS services



Set a payload

Define the data that you want sent to the target service when the schedule runs



Retry policy

Set up retry, dead letter queue, and encryption

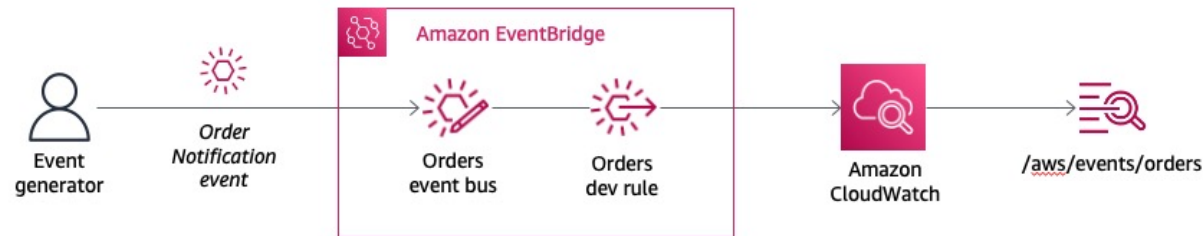


Manage and monitor

Manage schedules, schedule groups and tags

Simple Event Bus

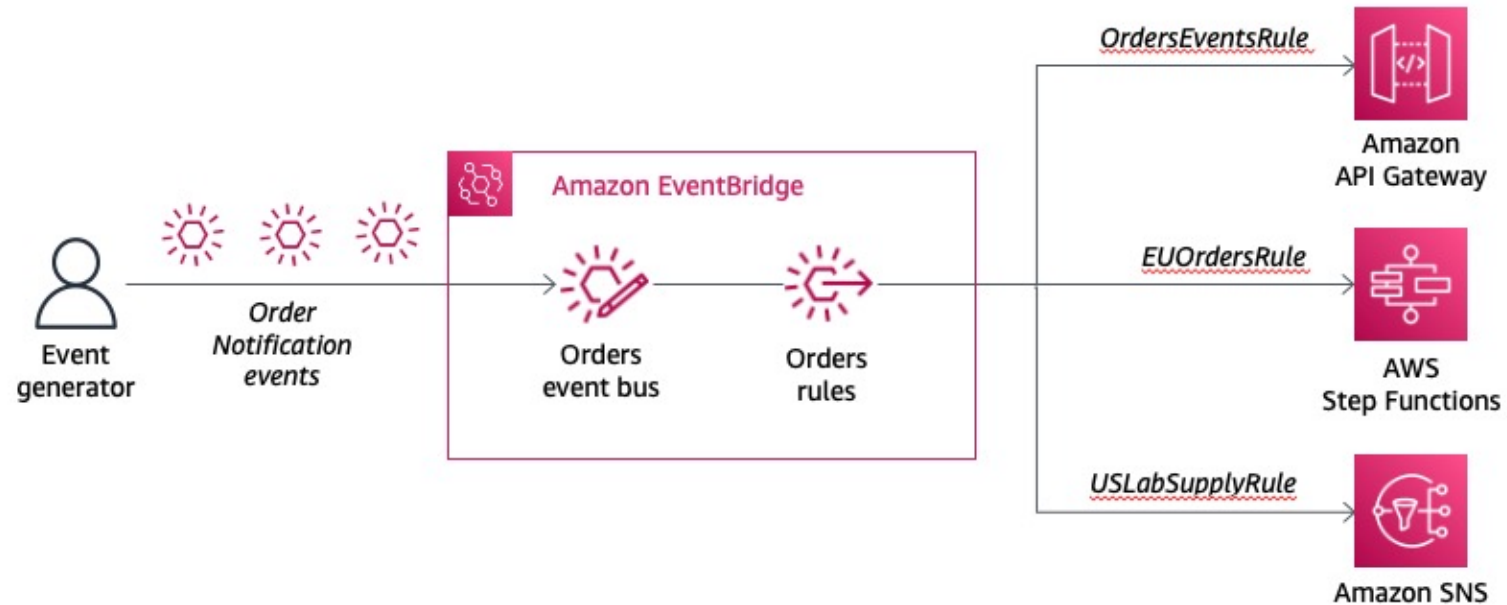
<https://catalog.us-east-1.prod.workshops.aws/workshops/63320e83-6abc-493d-83d8-f822584fb3cb/en-US/eventbridge>



- Create an **event bus 'Orders'** with defaults (you will get an error, ignore it)
- Don't use visual rule builder
- Create a **rule OrdersDevRule**
 - Add 'Catchall rule for development purposes' for **Description**
 - Select Rule with an event pattern for the **Rule type, event source 'other'**
 - Custom pattern { "source": ["com.aws.orders"] }
 - Target CloudWatch log group /aws/events/orders
- Test with **'send event' action on Orders**
 - Source com.aws.orders, detail type Order Notification
 - Payload

```
{
  "category": "lab-supplies",
  "value": 415,
  "location": "eu-west"
}
```
 - Send it, you will see the log event (but can verify separately in CloudWatch)

Working with EventBridge Rules



EventBridge Rule targeting SNS

- Create a topic New-Order (with access for everyone) and subscribe to it
- Build a rule USLabSupplyRule for the Orders event bus
 - Detail location us-east or us-west, detail category lab-supplies
 - Event Source: Other
 - Target SNS and the New-Order topic

- The custom pattern is

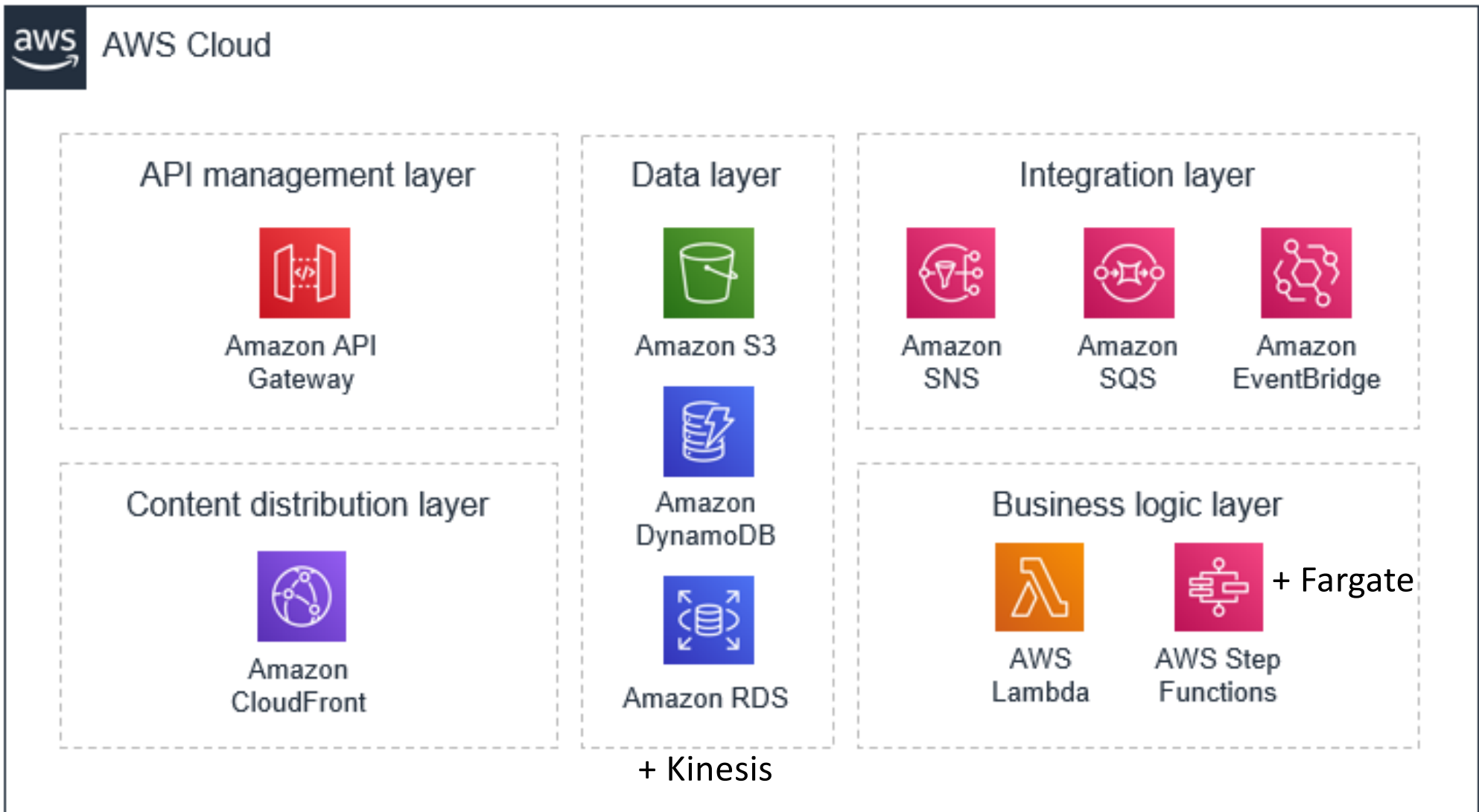
```
{  
  "detail-type": ["Order Notification"],  
  "source": ["com.aws.orders"],  
  "detail": {  
    "category": ["lab-supplies"],  
    "location": ["us-east", "us-west"]  
  }  
}
```

- Send an event with to the Orders Bus with Order Notification, com.aws.orders and various categories and locations
 { "category": "lab-supplies", "value": 415, "location": "us-east" }
- Check your subscription to New-Order

Using EventBridge to start a Step Function in Response to an S3 Event

- Create a bucket, e.g. krk-eb-tutorial, and enable Eventbridge notification for all events
- Create an Eventbridge Rule 'S3StepFunctions'
 - Use default bus
 - AWS Service S3
 - Event Type Amazon S3 Event Notification
 - Specific event 'Object Created'
 - Specify your bucket
 - Specify your target (Step Functions State Machine with name, e.g. helloworld)
 - **Specify LabRole** as the execution role
- Test the execution
- Use cases:
 - Start image processing workflow when an image is uploaded
 - Start order processing when order comes in

AWS Serverless Platform

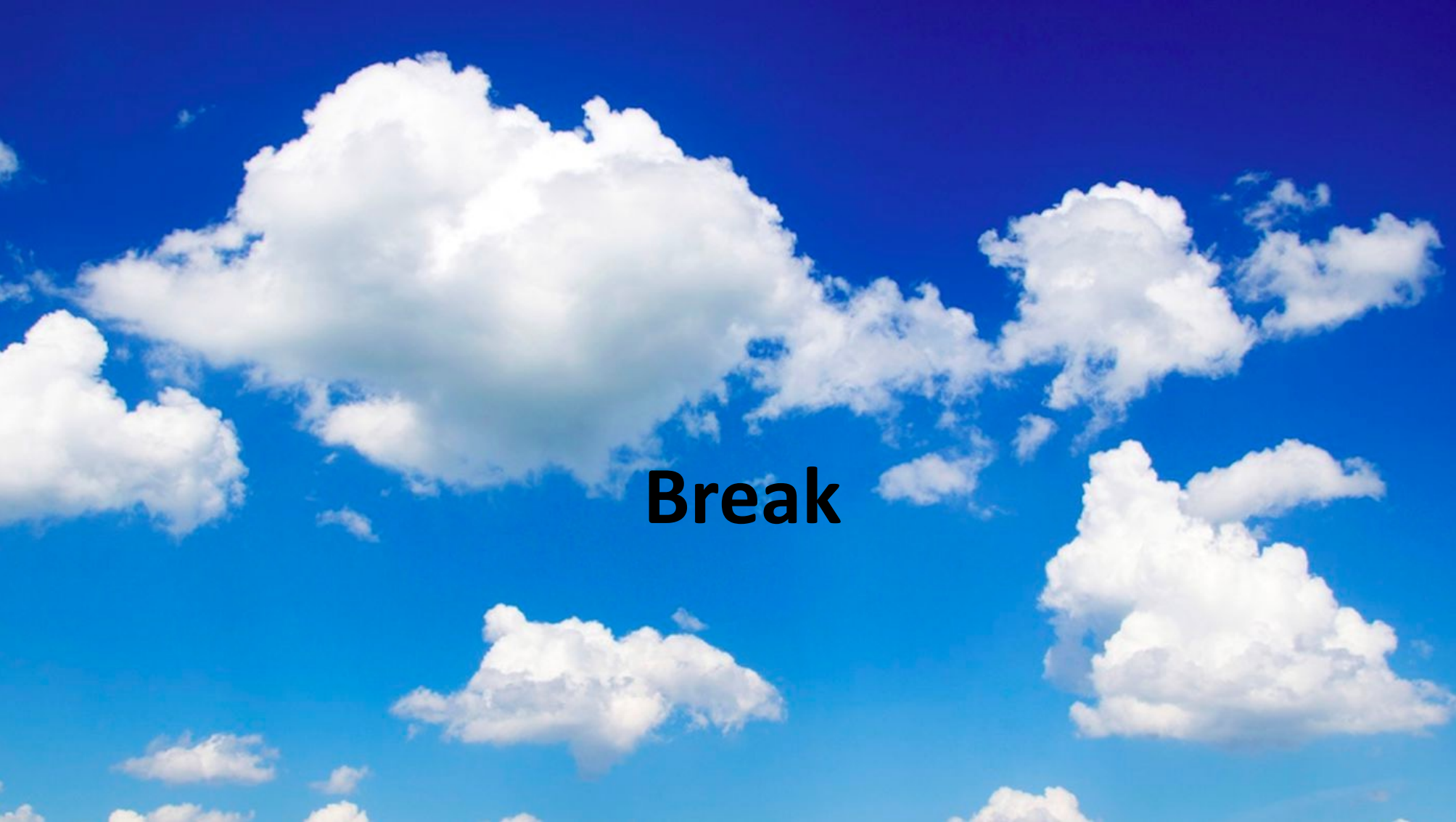


<https://docs.aws.amazon.com/lambda/latest/operatorguide/lambda-event-driven-paradigm.html>

12 Things to Know on Serverless

<https://aws.amazon.com/blogs/architecture/ten-things-serverless-architects-should-know/>

1. Microservices and API design
2. Event-driven architectures and asynchronous messaging patterns
3. Workflow orchestration in a distributed microservices environment
4. Cloud functions compute model (e.g. Lambda)
5. (Serverless) containers, especially Kubernetes (KK addition)
6. Cloud databases, especially NoSQL (KK addition)
7. *(Serverless) deployment and CI/CD patterns*
8. *(Serverless) identity management, authentication and authorization*
9. *End-to-end security techniques*
10. Application observability with logging, metrics and tracing
11. Best practices (Well-Architected)
12. Staying on top of technology evolution



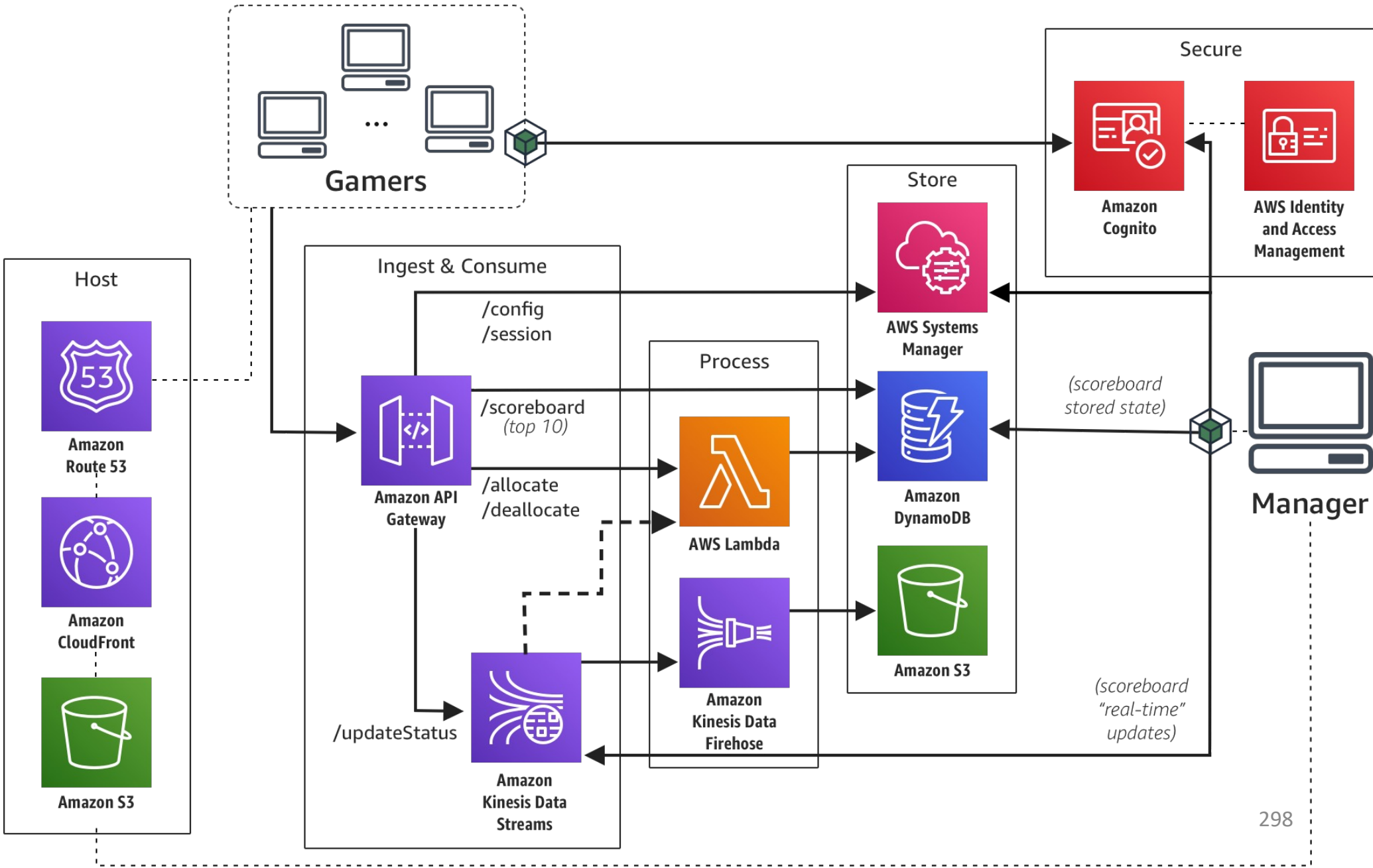
Break

Serverlessland Workshops

<https://serverlessland.com/learn?type=Workshops>

- Alien Attack
 - <https://catalog.us-east-1.prod.workshops.aws/workshops/fc36cb5a-de1b-403f-84dd-cc824390c548/en-US>
 - Gaming Analytics scenario
- Alien Attack Short Labs
 - <https://catalog.us-east-1.prod.workshops.aws/workshops/3ae476e4-e66d-4e78-b22f-6190c79ddee2/en-US>
- Serverless Data Processing (WildRydes)
 - <https://catalog.us-east-1.prod.workshops.aws/workshops/76d4b4eb-bff7-40c6-a925-7f101ad3bd43/en-US>
- Innovator Island (Theme Park)
 - <https://serverlessland.com/learn/innovator-island#>
 - <https://catalog.us-east-1.prod.workshops.aws/workshops/74d0f3be-7108-4bba-8136-00617a988535/en-US>

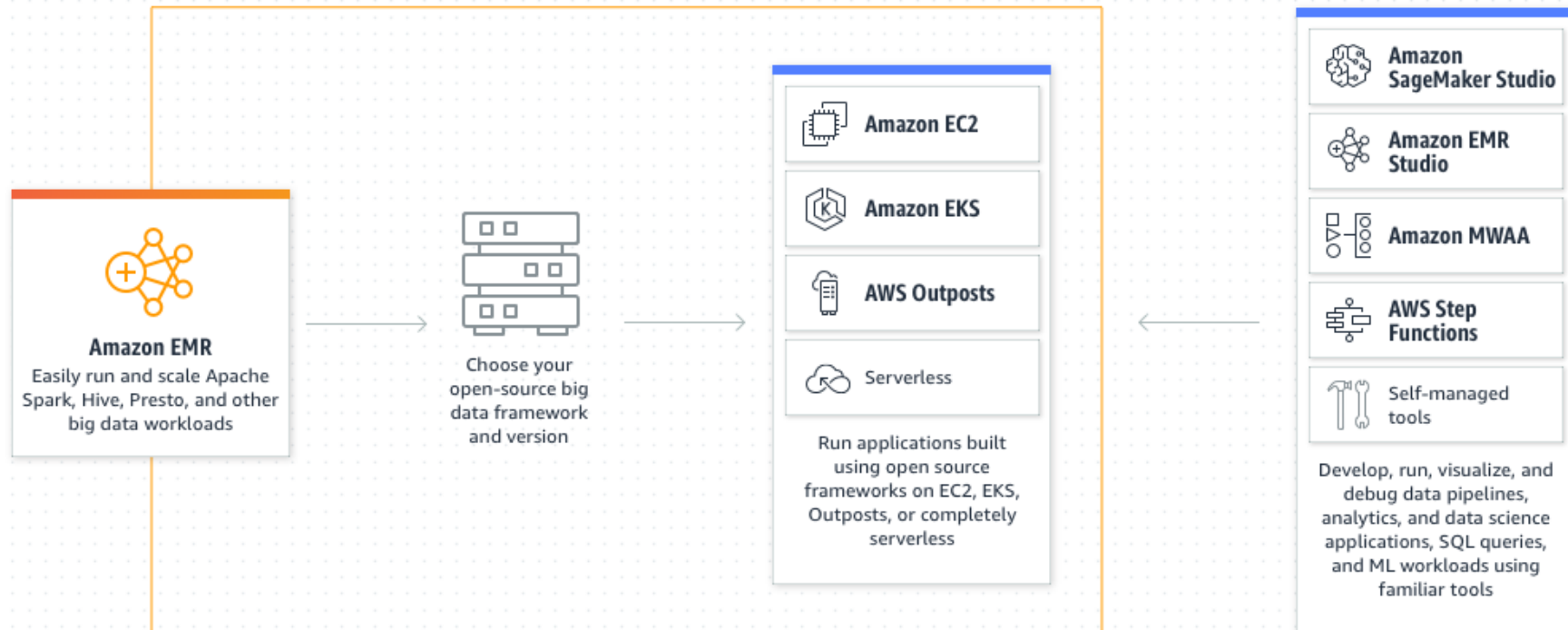
Alien Attack



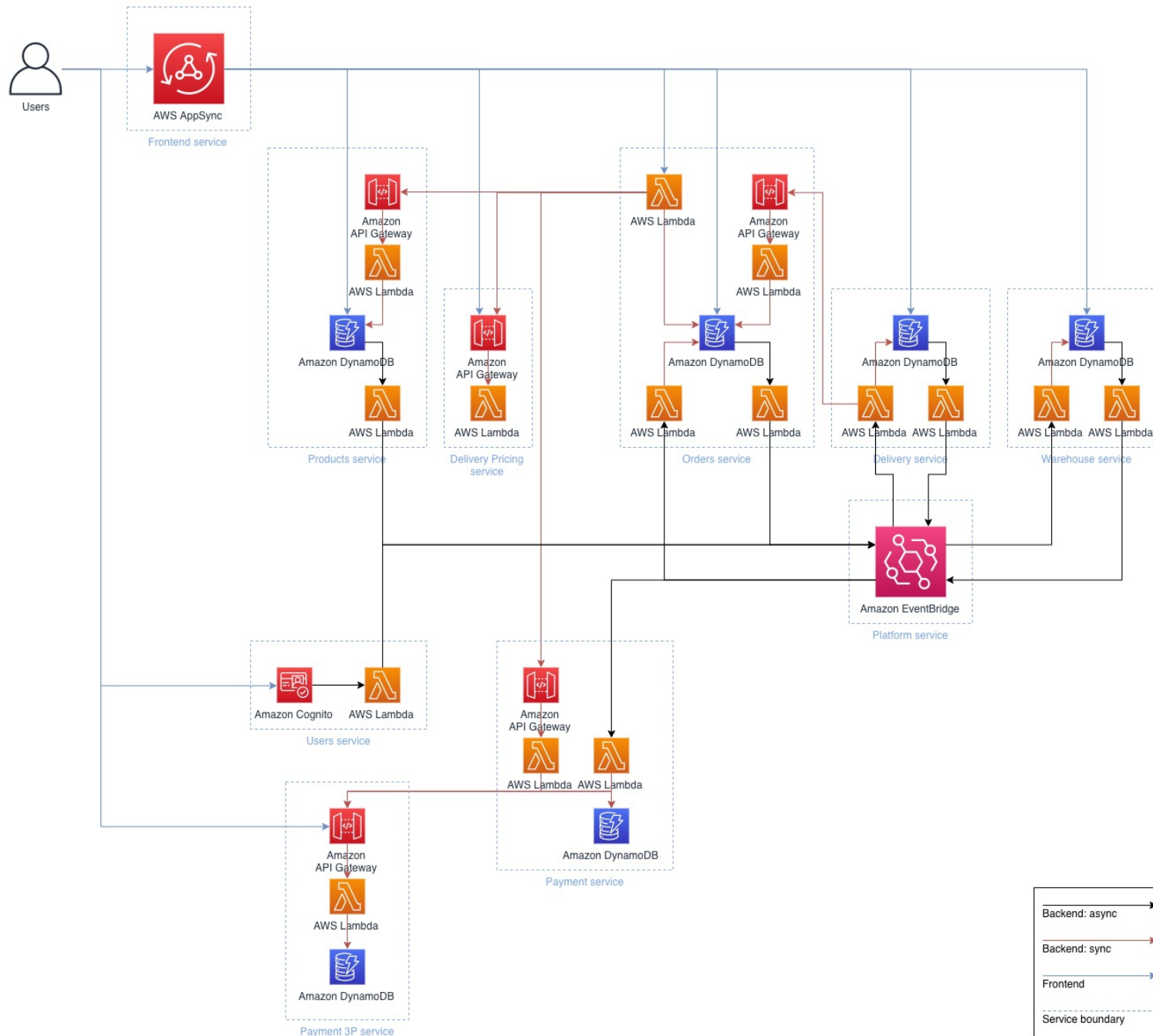


Appendix

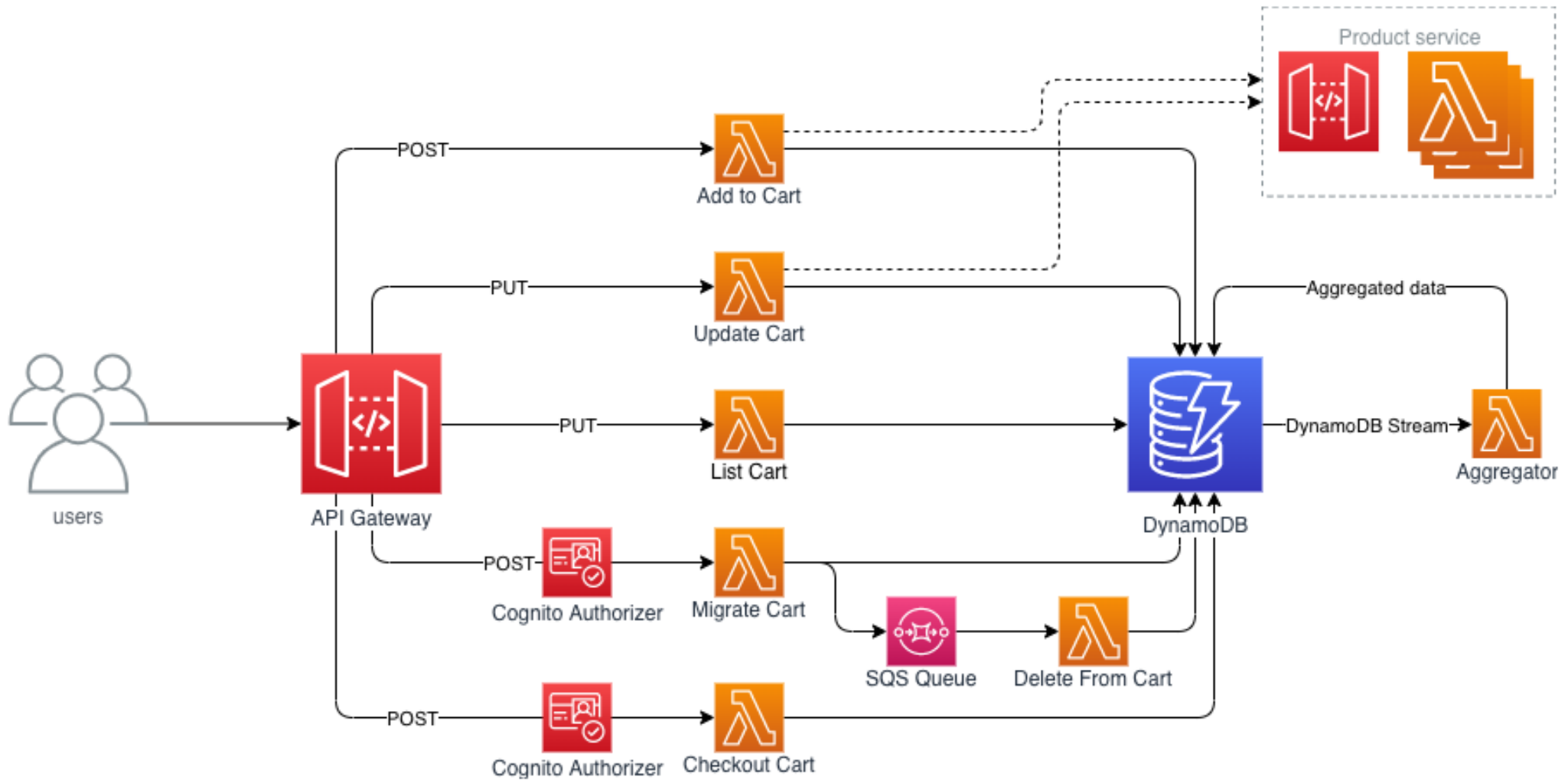
Amazon EMR (was Elastic Map Reduce)



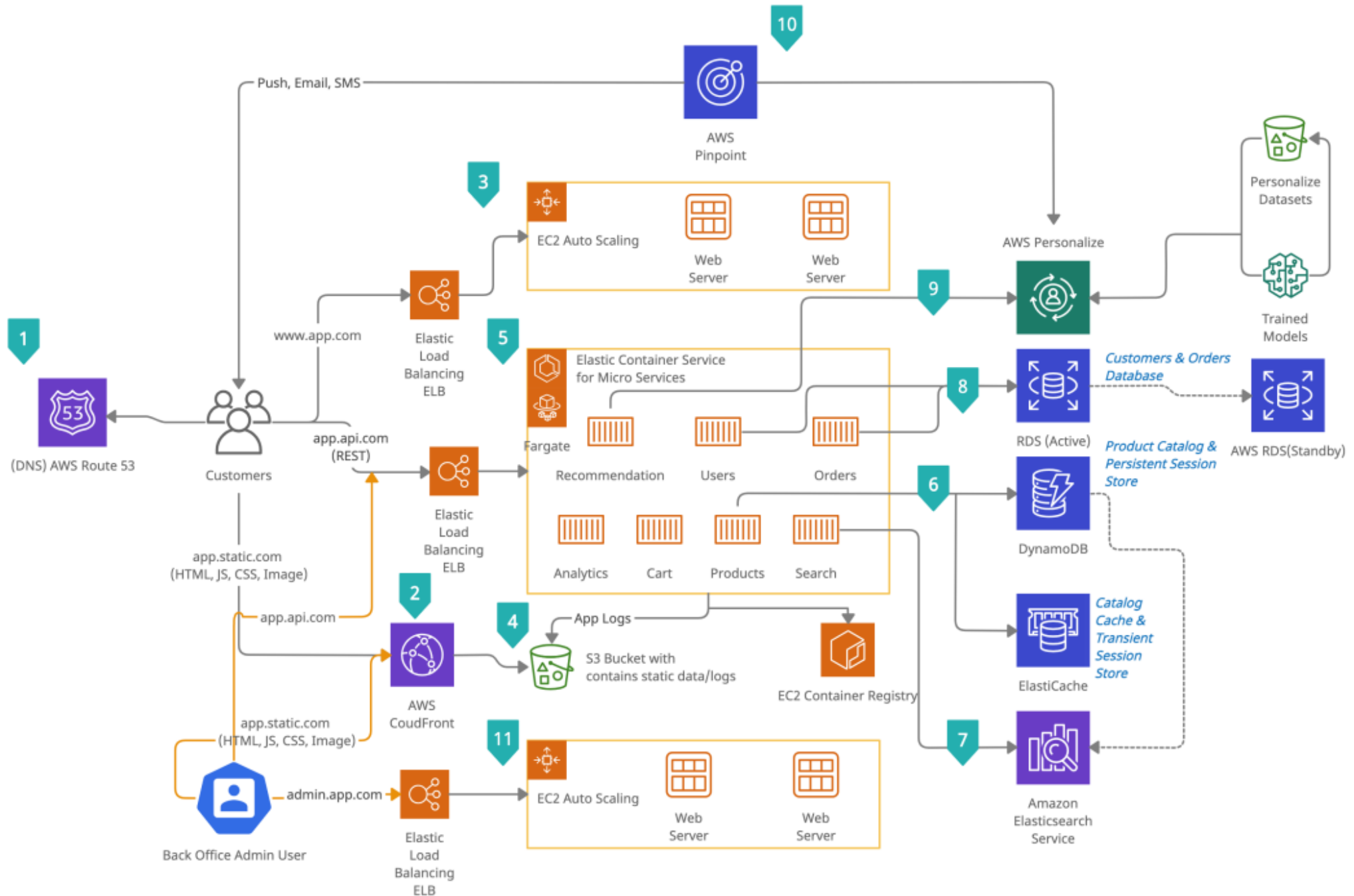
<https://github.com/aws-samples/aws-serverless-e-commerce-platform>



<https://github.com/aws-samples/aws-serverless-shopping-cart?tab=readme-ov-file>



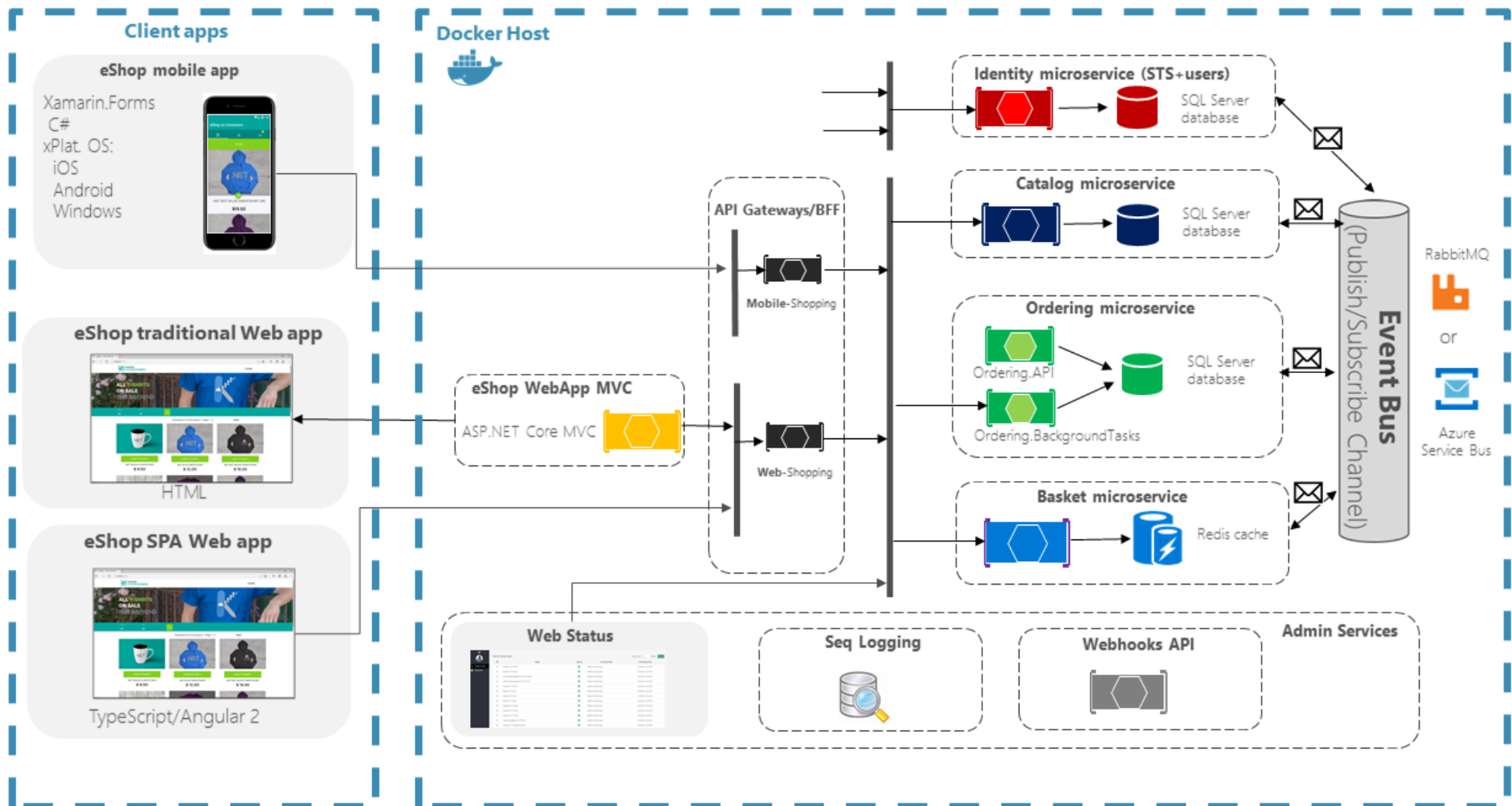
<https://www.linkedin.com/pulse/e-commerce-application-architecture-aws-cloud-suvendu-mohanty>



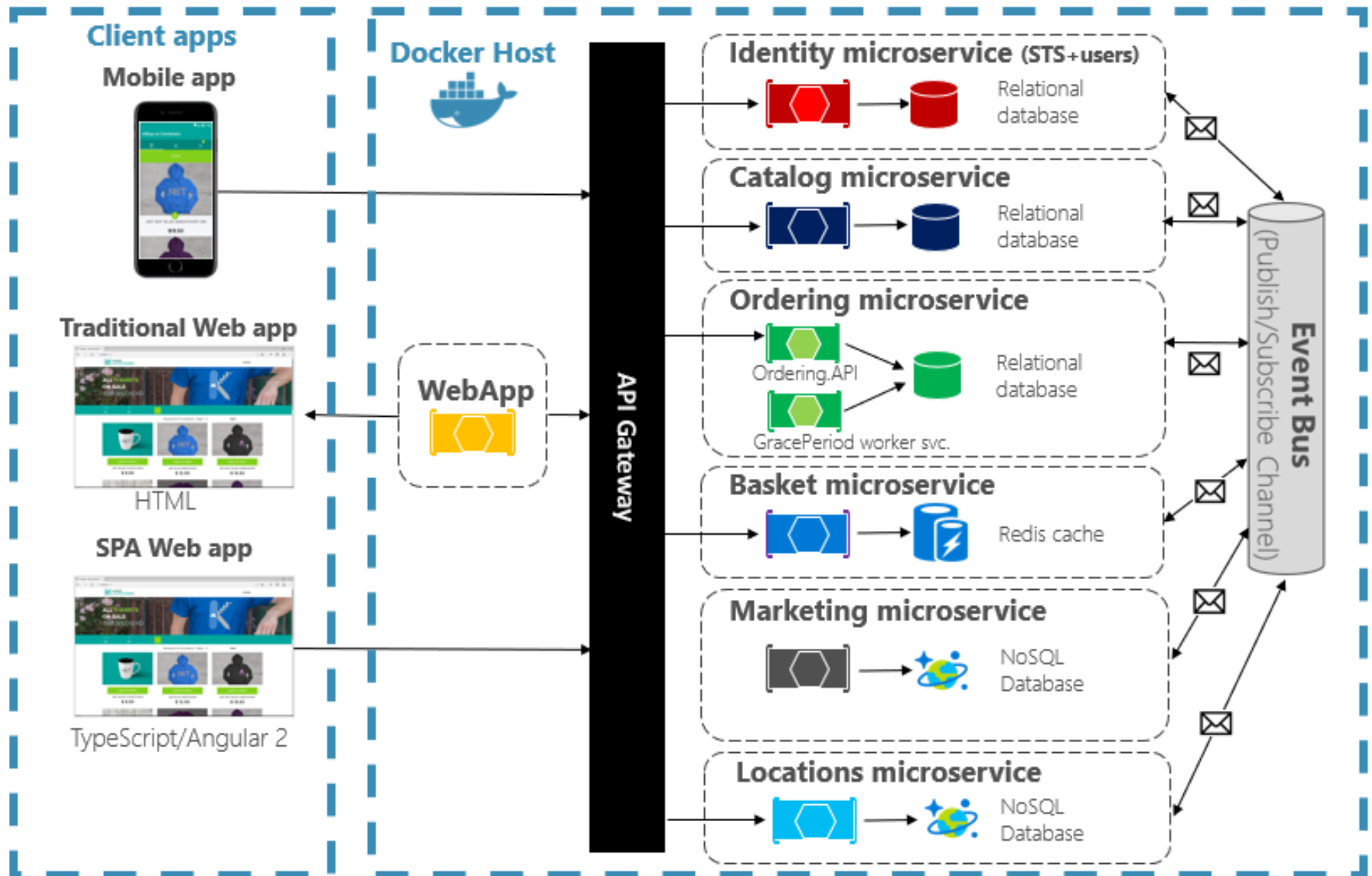
<https://learn.microsoft.com/en-us/dotnet/architecture/cloud-native/introduce-eshoponcontainers-reference-app>

eShopOnContainers reference application

(Development environment architecture)



OpenLegacy derived from MS



Shop — E-Commerce Application Architecture

AWS PRODUCTION (FARGATE + AMPLIFY)

